中国研究生操作系统开源创新大赛

项目说明书

作品名称：面向智能机器人的 AIOS

参赛队伍：Xenoblade

完成日期：2026 年 10 月

目 录

第 1 章 绪 论 **1**
1.1 目的与意义 1
1.2 项目背景 1
第 2 章 技 术 理 论 **4**
2.1 设计思想 4
2.2 技术路线 4
2.3 视觉推理与运动控制链路 4
2.3.1 JPU 硬件解码 5
2.3.2 VPSS 几何转换 5
2.3.3 TDMA 数据传输 5
2.3.4 TPU 硬件推理 6
2.3.5 运动控制追球 6
2.4 正式链路的函数级调用过程 7
2.4.1 初始化阶段 7
2.4.2 后台 UVC 采集阶段 7
2.4.3 前台摄像头与 JPU 阶段 8
2.4.4 VPSS 阶段 8
2.4.5 TPU 输入与模型前向阶段 9
2.4.6 后处理与控制阶段 9
2.5 内核优化：从“每个操作要付多少固定成本”入手 9
2.5.1 优化之前的总体画像 10
2.5.2 机制族清单 11
2.6 实时小核：单核平台的实时缺口与冗余缺口 12
2.6.1 问题：实时缺口与冗余缺口 12
2.6.2 平台事实：这颗核在哪、归谁管、能碰什么 13
2.7 板级 I/O：三条约束如何决定技术路线 14
2.8 代码原创说明 15
2.8.1 提交清单 15
2.8.2 原创性与引用边界 15
第 3 章 项目介绍 **17**
3.1 功能介绍 17
3.1.1 xxx 模块 17
3.1.2 xxx 模块 17
3.2 推理链路优化的工程实现 17
3.2.1 以分段探针驱动链路重构 17
3.2.2 将同步取帧改造成有界的最新帧流水线 17
3.2.3 JPU 硬件解码的具体实现 18
3.2.4 VPSS 几何转换的具体实现 18

3.2.5 TDMA 数据传输的具体实现 19
3.2.6 TPU 硬件推理的具体实现 19
3.2.7 加速器完成事件的统一调度优化 20
3.2.8 TPU 输出解析与检测后处理 20
3.3 机器人用户态程序优化 20
3.3.1 基于视觉尺度反馈的二次曲线渐进接近控制 20
3.3.2 可识别红色容器 22
3.3.3 摄像头完全遮挡后的脱困与视觉恢复 24
3.3.4 多目标识别与基于视觉距离的抓取排序 25
3.4 内核优化的工程实现 26
3.4.1 K1 系统调用返回路径：把指令缓存刷新收敛到少数事件上 26
3.4.2 K2 地址空间操作的机械开销：三处只删冗余的改动 27
3.4.3 K3 exec 的页物化：先减少要铺的页，再把剩下的摊到一次操作上 27
3.4.4 K4 缺页的每页固定成本：页索引与内存原语 27
3.4.5 K5 顺序缺页的窗口批量物化：四步 28
3.4.6 K6 文件读路径：一次多余的拷贝、一次逐页的事务、两处冗余失效 28
3.4.7 K7 存储写回与元数据：批量粒度、页缓存容量与访问时间 29
3.4.8 K8 启动等待：固定阻塞改成带截止时间的轮询 29
3.5 跨核通道：从寄存器到设备节点 29
3.6 小核接管底盘链路：三阶段落地 30
3.7 板级 I/O 与上板工作流 31
3.7.1 控制台与串口：单路 UART 下的可编程工作流 31
3.7.2 镜像通路：以 U-Boot UMS 取代串口文件传输 31
3.7.3 系统切换：以小核为中介的暖复位与双系统对照 32
3.7.4 执行器通路：SG2002 PWM 的编程模型 32
3.7.5 从网卡上电到板上 SSH 33
3.8 软件界面 （可选） 33
3.8.1 X 主要模块界面 33
第 4 章 实验测试 34
4.1 测试说明 34
4.2 测试结果 34
4.3 StarryOS 与 Linux 视觉性能对比 34
4.3.1 测试结果概述 34
4.3.2 各项指标的含义以及计算方式 35
4.3.3 表格结果解释 39
4.3.4 StarryOS 快于 Linux 的原因分析 39
4.3.5 StarryOS 慢于 Linux 的可能原因 40
4.3.6 对置信度结果的解释 42
4.3.7 综合结论 42
4.4 内核优化的测试说明 42

4.4.1 本线特有的测试口径 42
4.4.2 探针与验收门 44
4.5 内核优化的测试结果 46
4.5.1 总体对照 46
4.5.2 系统调用与进程间通信 46
4.5.3 进程创建与 exec 47
4.5.4 缺页与地址空间 47
4.5.5 文件读路径 49
4.5.6 存储写回与元数据 50
4.5.7 启动时间 51
4.6 小核链路的测试说明与方法 52
4.7 小核链路的测试结果 53
4.7.1 跨核往返延迟与稳定性 53
4.7.2 底盘链路接管的验收结果 53
4.7.3 控制周期的确定性：大核直控与小核接管的 A/B 对照 53
4.7.4 控制周期档位与代价 54
4.8 上板工作流与启动锚点的实测 55
第 5 章 实现难点说明 56
5.1 视觉推理链路优化难点 56
5.1.1 优化前的处理链路与问题模型 57
5.1.2 异步采集与最新帧调度机制 59
5.1.3 JPU 硬件解码优化 60
5.1.4 ION 共享物理缓冲区 62
5.1.5 扩展 VPSS 驱动，直接接收 JPU 输出的 YUV422P 63
5.1.6 修复 VPSS 完成后的调度长尾 65
5.1.7 模型输入契约校验与正确的 TPU 输入导入 67
5.1.8 TPU ioctl ABI、ION 地址与完成语义修复 68
5.1.9 TPU 任务分类与四段时间戳计时机制 69
5.1.10 TPU 执行链路重构与调度开销消除 70
5.1.11 面向短时 TDMA 的中断唤醒与竞态管理 71
5.1.12 TPU 驱动优化 73
5.1.13 TPU 输出直读与轻量化检测后处理 74
5.2 技术路线取舍与回退方案 76
5.2.1 强制 JPU 输出 NV12 76
5.2.2 普通模型输入直接物理地址绑定 76
5.2.3 P2 aligned 640 模型 76
5.2.4 固定跳帧或降低处理帧数 76
5.2.5 继续提高 compute 线程优先级 76
5.3 内核优化中的定位方法与难点 77
5.3.1 定位方法：分段计账、标度实验与受控 A/B 77

5.3.2 系统调用返回路径：一笔 2.17 µs 的冗余刷新 77
5.3.3 进程与 exec：账目与两条决定性判据 78
5.3.4 上下文切换：归因清楚了，但没动 78
5.3.5 缺页：从“87.7% 在 prepare”到一个 $O(n^2)$ 的索引 79
5.3.6 读路径：前两条线索为什么只值 10% 80
5.3.7 其余指标的剩余固定成本 80
5.4 被数据否掉的假设 81
5.5 已经归因清楚、但本轮没有改的项 81
5.6 一条尚未定位清楚的问题 82
5.7 实时小核：通道、兜底与边界 83
5.7.1 跨核通道的三处硬约束 83
5.7.2 失效兜底：小核挂了谁停车 83
5.7.3 小核还能做什么，以及为什么现在没做 84
5.7.4 负结果与取证纪律 85
5.7.5 方法：一个把结论改判的例子 86
5.7.6 边界与可复用产物 86
5.8 板级 I/O 的难点：三处硬约束与一处并发缺陷 86
5.8.1 单路控制台的代价：UART 焊盘是复用的 86
5.8.2 介质通路的两条边界：节点残留与空间统计 87
5.8.3 软复位复位了什么：只复位 SoC，不复位执行机构 87
5.8.4 PWM 的四处语义落差 87
5.8.5 无线链路的并发缺陷：累计传输约 9 MiB 后静默失效 88
第 6 章 总 结 **89**
6.1 视觉推理链路优化总结 89
6.2 内核优化总结 90
6.3 实时小核工作线总结 90
6.4 板级 I/O 与上板工作流总结 91
参 考 **92**

第 1 章　绪 论

1.1　目的与意义

伴随具身智能与 AI 大模型的快速演进，机器人正由受控环境走向开放而复杂的真实场景，其应用普遍包含高频视觉感知、实时决策与运动控制，对系统的实时性与资源利用率提出了严苛要求。这类系统通常把模型、调度、驱动与供电等模块集成在同一块 SoC（片上系统）内，操作系统既要不断向加速器投喂数据，又要协调多核之间的处理器时间分配。当前主流做法是在 Linux 等通用操作系统上搭建，其成熟的驱动与软件生态虽然便利，却也引入冗余服务、较长的执行路径、复杂的调度策略与较大的内存开销，在边缘算力受限时容易转化为端到端延迟与抖动上的负担。如果改由 Rust 编写、具备组件化特征的内核来承载同样的系统，内存安全与模块化程度可望提升，但必须以通用负载性能不受损为条件。本项目所采用的载体是一台网球拾取机器人，其程序来自 `chenlongos/akars`，已经以 Rust 实现，能够同时运行在 Linux 与 StarryOS 之上。该机器人的执行回路由追球、抓取、回桶、寻桶、趋近和投放六个状态构成，评价指标取相机成帧到执行器指令之间的帧到指令延迟，与竞赛官方口径一致。

项目工作沿应用层与系统层两条线索推进。在应用层，目标是让这台机器人在 StarryOS 上完整跑起来，并对推理链路加以优化，借助取球循环的组织方式与数据布局的调整，把帧到指令延迟压缩到与 Linux 相当的水平，同时补齐 TPU 推理所依赖的关键外设，使推理通路真正贯通。在系统层，则把 StarryOS 当作一个通用操作系统来对待，在系统调用入口、进程创建与调度、缺页与内存映射、文件与存储 I/O 这些与具体应用无直接绑定的子系统上，与 Linux 逐项对照；本平台只有一颗通用核，没有可供内核使用的调频调压设施，因此频率一项不在对等范围内。机器人因而只是被选中的负载之一，真正被检验的是操作系统本身的通用能力。

为使两侧结论具备可比性，评测口径需要预先统一：各子系统配套专属的基准套件，并各自设立独立的 Linux 基线；双方执行的是同一份符合 Linux ABI 的可执行文件，因此同一负载在两侧呈现出的差别，只会来自操作系统本身。下文先交代机器人应用，随后按视觉推理与运动控制链路、内核优化、实时小核、板级 I/O 与上板工作流四条工作线逐条展开。凡属对等比较的子系统，均以相应的 Linux 基线为参照并说明所做的改动；至于板级 I/O 与上板工作流一类以功能使能与被测可达为目的的工作，只陈述其使能效果，不作为对等性能结论。

1.2　项目背景

赛题「面向智能机器人的 AIOS」要求把开源机器人的操作系统内核从 C 语言实现的 Linux 整体迁移到 以 Rust 编写的组件化内核，在内核之上补齐系统调用、文件接口与 NPU/USB/UART/摄像头等关键外设驱动， 打通 NPU 推理链路，并在端到端延迟、启动速度、推理帧数与资源、能耗比等维度超过同机 Linux 基线。 赛程分三步走：先在开发板上把内核启动起来并完成模型加载，再补齐 USB 感知与串口电机驱动、完成 网球的拾取，最后与 Linux 基线逐项对标做性能优化。本项目落在第三步，交付的是一份可复现的对标 结论，以及为得到它而做的内核改动。

硬件载体是辰龙 AKA-00 教育机器人，主控为 SG2002：应用核是 RISC-V C906 @ 1 GHz（与 ARM Cortex-A53 二者取一，由管脚选择），另有 C906 @ 700 MHz 协处理器， 片上 TPU 的 INT8 算力为 1 TOPS，板载 DDR 256 MiB。机器人程序是 `akars`，由同名的 C 语言版本移植为

Rust 二进制，链路为 USB 摄像头取流、MJPEG 解码与 letterbox、 YOLOv8 的 CVI 运行时推理、串口底盘与串口机械臂，六个状态构成拾球闭环。它运行在 musl 根文件 系统之上，依赖厂商预编译的 `libcviruntime` 与 `libcvikernel`，并按 Linux 的 `ioctl` 与 `mmap` 约定 访问相机与 TPU。算力与内存受限、软件栈以 Linux 二进制形式固化、闭环对延迟敏感，这三条约束 共同划定了「换内核」一事的可行边界。

Linux 成为这类机器人的默认底座，理由很实在：驱动与中间件完备，摄像头、USB、文件系统与网络 开箱可用；ABI 稳定，厂商以二进制分发的推理运行时可原样运行；工具链与调试手段成熟，开发门槛低。代价来自同一处，也就是它的通用性。为兼容各种场景，系统里常驻着一批与任务无关的服务与抽象，一次调用要穿过比实际需要更长的路径；调度器在通用负载之间做公平取舍，内存子系统的许多假设在 大内存机器上成立、在小内存设备上则是额外开销。这些开销在桌面与服务器上被充裕的算力吸收，在 边缘受限时直接表现为端到端延迟与抖动。本项目的实测支持这一判断：厂商 Linux 从上电到首帧推理 耗时 34.23 s，其中内核初始化只占 1.2 s，其余 31.99 s 几乎都花在用户态的初始化与服务上；同一块板上，StarryOS 的内核初始化是 9.01 s， 用户态只要 4.2 s，总的 14.66 s 反而更短。通用底座把相当一部分 开销放在了内核之外，而它省下来的内核工作时间有限。

不同系统形态各有取舍，对照见表 1。

系统形态	代表	优势	在本题约束下的局限
C 语言宏内核	厂商 Linux 5.10	驱动与中间件完备，USB、摄像头、文件系统与网络开箱可用；ABI 稳定，厂商以二进制分发的 NPU 运行时可直接运行。	通用性带来冗余服务与较长的执行路径；空系统调用、缺页、块设备等入口开销高；常驻内存占用大。
实时内核	FreeRTOS、RT-Thread、Zephyr、NuttX	确定性好、体积小、上电即用，适合承担确定性感知前端。	没有 Linux ABI 与完整的进程、文件语义，厂商预编译的相机与 NPU 运行时无法复用； SG2002 的小核固件本身也不含 USB 与 TPU 驱动。
组件化 unikernel	ArceOS	能力以 `crate` 组合，裁剪自由度最大，是 StarryOS 的组件底座。	面向单地址空间应用，不带进程、信号与 Linux 兼容系统调用，无法直接承载已经编译好的机器人程序。
Rust 宏内核	StarryOS（TGOSKits）	在组件层之上补齐 Linux 系统调用、进程与信号；内存安全、模块化，可直接运行符合 Linux ABI 的可执行文件。	性能与稳定性尚未对齐 Linux，系统调用入口、内存与启动仍有明确差距。

表 1: 主流系统形态在本题约束下的取舍

把上表收拢成一句：Linux 的长处几乎都来自「通用」，短板也几乎都来自「通用」；实时内核与组件化 unikernel 则各自用放弃兼容性换来确定性与可裁剪性。赛题给定的条件同时要求直接运行厂商预编译的 机器人二进制、覆盖 USB/UART/摄像头/TPU 四类外设、并与 Linux 基线逐项

对标，能满足这三条的形态 只有「Rust 组件化内核 + Linux ABI 兼容」，也就是 StarryOS 所在的位置。

上游 TGOSKits 已经提供了这个起点：一个含 184 个 workspace 成员的 三系统工作区（ArceOS、StarryOS、Axvisor）、4 种 64 位架构，以及可复用的组件与 驱动，其中包含 SG2002 的 TPU 与它依赖的 ION 分配器、USB 子系统与串口。板卡侧，`board-aka-00-sg2002` 一条配置下已登记启动、`lsusb`、`libuvc` 取流、WiFi 压测与 `tennis-yolo` 检测 等用例；其中 `tennis-yolo` 用例要求板上打印 `STARRY_AKA00_TENNIS_DETECT_OK`，说明「模型能被加载 并识别出网球」这一环在上游已经打通。缺口在别处。把内核换成 StarryOS 之后，空系统调用的开销 相对 Linux 高出 26.7 倍，模型加载耗时 2.99 s 对 Linux 的 0.07 s，取球过程的峰值内存 19316 kB 对 Linux 的 7604 kB。 系统层因而仍有明确的优化空间，这正是本项目介入的位置。

本作品的新颖之处有三。

第一是问题域。赛题给出两台参考平台，本队落在 SG2002 一侧：它只有一颗可运行 Linux 的 C906 应用核，没有 ARM 大小核，也没有可供内核使用的调频调压设施与 PMU，RK3588 上那套成熟的大小核调度与 DVFS 路径在这里无法复制。可优化的对象随之改变——系统调用入口的固定开销、缺页与内存映射、块设备的 写放大与 `fsync`、上电到首帧的启动链路，以及网络与 WiFi 的稳定性，成为左右端到端表现的主要因素。

第二是评测口径。既有工作多以「优化前／优化后」自证，本作品要求每条结论都带同板对照：同一块板、 同一份符合 Linux ABI 的可执行文件，在 StarryOS 与厂商 Linux 两条基线上各测一次；系统层的每处 改动再以同板 A/B 给出优化前与优化后的读数，并注明探针与口径，凡两侧口径不能对齐的指标一律拆开 分别报告。

第三是贡献归属。报告中的每条优化都对应一个可核对的提交，并标注该改动属于本队新增、上游既有， 还是对既有实现的修复；以功能使能与被测可达为目的的工作只陈述使能效果，不计入对等性能结论。 工作按四条线组织：应用层的视觉推理与运动控制链路，系统层的内核优化、实时小核，以及板级 I/O 与上板工作流。四条线共同的立场是，机器人只是被选中的负载之一，真正被检验的是操作系统的通用 能力。

第 2 章　技 术 理 论

2.1　设计思想

本作品研究的是同一台机器人、同一份可执行文件在两套操作系统上的差别，因此三条工作线（应用层视觉链路、系统层内核优化、系统层实时小核与板级 I/O）共用同一套方法论。这套 方法论只有三条，但它们决定了后面每一处的做法。

第一，**归因优先于改动**。任何一处“慢”都先拆成可测量的段——分段计时、位置直方图、标度实验——直到能把代价钉到某一行代码或某一条硬件语义上，再决定改什么。项目里收益 收益最大的几处改动均由分段数据直接指出，并非按经验“调优”得来：每次返回用户态 都执行一次指令缓存刷新、COW 页索引插入随区域线性变慢、`close` 路径里一次约 10 ms 的 同步元数据提交。反过来，几处“看起来该优化”的地方（缺页路径的 TLB 失效、页缓存容量 回收策略、无线链路的协议层）被数据否掉后就不再投入。

第二，**对照优先于自证**。所有性能结论都在同一块板、同一份符合 Linux ABI 的静态 ELF 上， 与厂商 Linux 各测一次；内核“优化前/优化后”的对比尽量在**同一次上电、交替顺序、多次 重复**下取得，以抵消“启动后第一次大分配更快”这类时间漂移。凡两侧口径不能对齐的指标（例如启动时间里的脚本段），一律拆开分别报告，不把差异记到内核头上。

第三，**可复现优先于结论**。探针、口径、回退开关与验收门都随代码提交：动到对外语义的改动都留运行期回退（指令缓存刷新策略、缺页预取窗口大小、访问时间策略），每个数字都 能追溯到一次具体运行的原始日志。这样做的直接好处是，后续接手的人不需要重问“这个数 是怎么测的”，只需要重跑同一组命令。

2.2　技术路线

全篇按**四条工作线**组织，它们共用同一块板与同一套评测口径：应用层的**视觉推理与运动 控制链路**把帧到指令延迟压到与 Linux 相当；系统层的**内核优化**把“每次系统调用、每次 进程创建、每次缺页、每次文件操作”这些固定开销逐项收窄；系统层的**实时小核**把厂商 固件里那颗“存在但未被使用”的 RISC-V C906L 变成可管理的第二个可编程执行体，并由它 接管底盘串口这条硬实时链路，补上单核平台在实时性与冗余上的缺口；**板级 I/O 与上板 工作流**则把 SG2002 的板载外设接通，并把“改内核—上板—取数”这一循环做成可重复执行、可在无人值守条件下运行的对象。以下按这个顺序给出四条线的技术路线。

本节首先介绍机器人视觉推理这一工作线的技术路线。它是项目技术体系的一部分，用于说明摄像头、图像加速器、TPU 与运动控制之间的协同关系；本章其余小节仍可继续接入系统调用、内存管理、存储与网络等系统层工作。

2.3　视觉推理与运动控制链路

本系统面向 SG2002 平台构建了一条由硬件加速器协同完成的网球视觉推理链路，整体采用“异步采集—硬件解码—硬件预处理—神经网络推理—检测后处理—运动控制”的处理流程。摄像头通过通用串行总线（Universal Serial Bus，USB）视频设备类协议（USB Video Class，UVC）输出分辨率为 640×480 的码流；该码流由 JPEG 压缩图像连续组成，格式称为 MJPEG（Motion

JPEG）。StarryOS 内核中的后台采集任务持续接收并组装 UVC 数据包，使用单调递增的帧序号发布最新完整帧。前台视觉线程只获取序号更新的图像；当图像采集速度高于推理速度时，系统允许旧帧被新帧覆盖，从而避免历史图像在队列中积压，保证机器人使用尽可能接近当前环境状态的图像进行闭环控制。

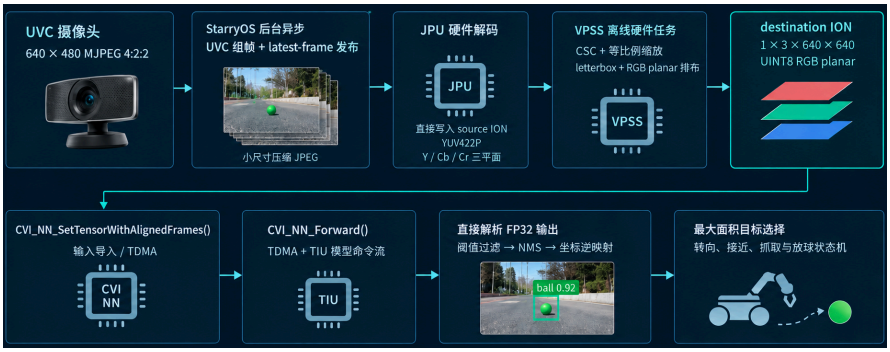


图 1: 网球视觉推理与运动控制链路

2.3.1　JPU 硬件解码

获得最新 MJPEG 帧后，系统使用 JPEG 处理单元（JPEG Processing Unit，JPU）完成硬件解码。当前摄像头的 JPEG 图像采用 4:2:2 色度采样，因此 JPU 将其解码为 640×480 的亮度—色度 4:2:2 平面格式（YUV 4:2:2 Planar，YUV422P），并通过直接存储器访问（Direct Memory Access，DMA）将亮度 Y、色度 Cb 和色度 Cr 三个平面直接写入 ION 内存分配器提供的共享缓冲区；ION 是该分配器的专有名称。该缓冲区同时具有中央处理器（Central Processing Unit，CPU）虚拟地址和硬件可访问的物理地址，可被后续视频处理模块直接使用，避免将整幅 YUV 图像复制到普通用户态临时内存。针对摄像头偶发的损坏 JPEG，驱动还实现了异常帧恢复机制；当 JPU 出现码流耗尽、超时或输出长度异常时，系统会丢弃当前帧、复位解码器并等待更新帧，而不会因单帧错误导致视觉线程永久阻塞。

2.3.2　VPSS 几何转换

JPU 解码完成后，YUV422P 图像被送入视频处理子系统（Video Processing Subsystem，VPSS）执行硬件预处理。VPSS 在一次离线任务中完成颜色空间转换（Color Space Conversion，CSC）、等比例几何变换、letterbox 填充以及红绿蓝（Red, Green, Blue，RGB）三平面排布。对于 640×480 的摄像头图像和 640×640 的模型输入，原始 4:3 图像保持纵横比并置于 640×640 画布中央，上下区域使用填充值补齐，从而避免直接拉伸造成网球外观畸变。处理结果以 8 位无符号整数（Unsigned 8-bit Integer，UINT8）形式存放在目标 ION 缓冲区中，并按照批次、通道、高度、宽度（Number, Channel, Height, Width，NCHW）布局组织为 `1×3×640×640` 的 RGB planar 数据。由于 VPSS 输出格式与模型输入契约一致，系统无需再由 CPU 执行逐像素 YUV 到 RGB 转换、图像缩放、通道重排或浮点归一化。

2.3.3　TDMA 数据传输

模型推理阶段使用 YOLOv8 网球检测模型。VPSS 生成的 RGB planar 物理缓冲区首先通过张量直接存储器访问（Tensor Direct Memory Access，TDMA）导入模型输入张量，随后由 CVITEK/SOPHGO 神经网络运行时调用 `CVI_NN_Forward()` 执行前向推理。运行时将模型编译生成的 TDMA 和张量指令单元（Tensor Instruction Unit，TIU）命令提交给张量处理器（Tensor

Processing Unit，TPU）驱动：TDMA 负责输入、权重、特征图和中间张量的数据传输，TIU 负责卷积、激活及其他张量运算。当前 TPU 工作频率为 700 MHz，片上互连域频率为 900 MHz。任务触发后，提交线程进入等待状态，并由中断请求（Interrupt Request，IRQ）在硬件执行完成时唤醒；驱动同时保留内存映射输入/输出（Memory-Mapped Input/Output，MMIO）轮询作为异常情况下的兜底机制。经过调度优化后，推理任务由提交线程直接串行执行，消除了旧实现中提交线程、TPU 工作线程和完成等待线程之间的多次调度交接。内核分段测试表明，当前约 39～41 ms 的 TPU 推理耗时主要来自指令触发至硬件中断之间的真实网络执行，而不是软件排队延迟。

2.3.4　TPU 硬件推理

TPU 前向计算完成后，用户态直接解析运行时管理的 32 位浮点数（32-bit Floating Point，FP32）输出张量。程序按照 YOLOv8 的通道优先布局读取每个候选框的中心坐标、宽度、高度和网球类别置信度，并首先依据置信度阈值剔除低质量候选框；当前默认阈值为 0.5。随后采用非极大值抑制（Non-Maximum Suppression，NMS）处理重叠候选框：程序按置信度降序排列候选框，并利用交并比（Intersection over Union，IoU）判断不同候选框的空间重叠程度，当 IoU 大于 0.5 时抑制置信度较低的重复框。由于模型输入图像经过 letterbox 处理，NMS 后还需要根据缩放比例和上下填充量执行坐标逆映射，将预测框从 640×640 模型坐标系恢复到 640×480 摄像头坐标系，为后续运动决策提供真实图像位置。

2.3.5　运动控制追球

运动控制层从经过阈值过滤和 NMS 的检测结果中选择面积最大的网球框作为当前追踪目标，以降低远处小目标对近距离抓取任务的干扰。系统使用检测框水平中心坐标表示网球相对于夹爪中心线的方向偏差，并以检测框面积占整幅图像面积的比例近似描述网球距离。当未检测到网球时，机器人执行原地旋转搜索；检测到远距离网球后，根据目标位于期望中心的左侧、右侧或中心区域，分别执行左转、右转或直行接近。目标接近抓取区域后，系统扩大中心容差并缩短转向脉冲，减少离散视觉反馈造成的左右过冲。当检测框面积比例达到 0.55、目标中心进入夹爪有效区域且连续两帧确认成立时，底盘停止并触发原有机械臂抓取序列。若网球面积比例达到 0.85，系统将其判定为距离过近，先执行轻微后退再开始抓取。由此，系统实现了从图像采集、硬件推理、空间位置估计到车辆转向、接近和机械臂抓取的完整视觉闭环。

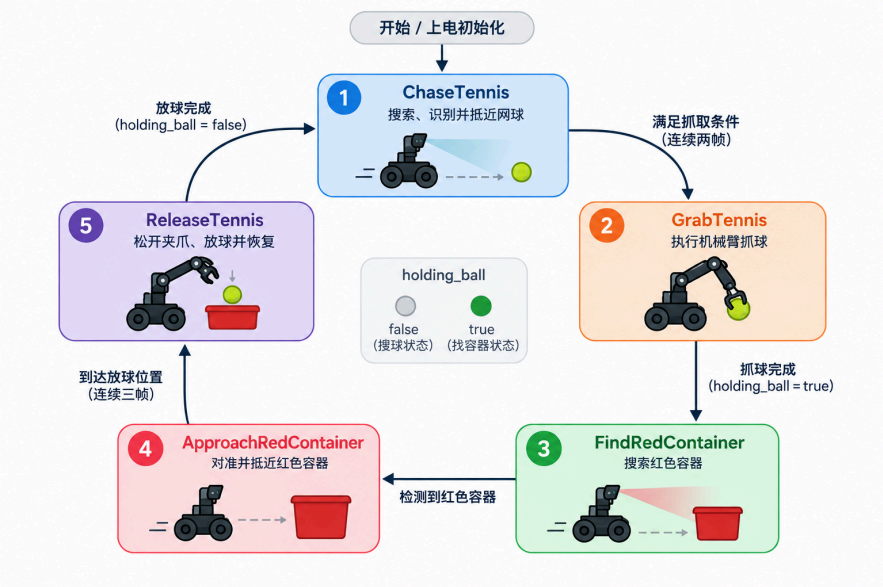


图 2: 网球追踪、接近与抓取控制流程

2.4　正式链路的函数级调用过程

2.4.1　初始化阶段

```
main()
  ├─ 解析模型、摄像头、VPSS、电机和机械臂参数
  ├─ YoloModel::open(model_path)
  │    ├─ CVI_NN_RegisterModel()
  │    ├─ CVI_NN_GetInputOutputTensors()
  │    ├─ 检查输入 shape/format/pixel_format/aligned/quantization
  │    └─ 输出 AKARS_TPU_INPUT 契约
  ├─ VpssRgbPipeline::open(output_width=640, output_height=640)
  │    ├─ IonAllocation::allocate(source=2 MiB)
  │    ├─ IonAllocation::allocate(destination=3×640×640)
  │    ├─ open(/dev/cvi-usb-camera0)
  │    ├─ open(/dev/cvi-vpss0)
  │    ├─ CVI_CAMERA_IOCTL_INIT
  │    ├─ CVI_CAMERA_IOCTL_RESET_CAPTURE_STATS
  │    └─ CVI_CAMERA_IOCTL_START_ASYNC
  ├─ 打开 /dev/ttyS1 底盘电机
  ├─ 打开 /dev/ttyS2 机械臂控制器
  └─ run_tennis_hunter()
```

2.4.2　后台 UVC 采集阶段

```
async_capture_worker()
  ├─ 等待一帧在途背压条件满足
  ├─ capture_frame()
  │    ├─ uvc_capture_one_frame_profiled()
  │    │    ├─ USB/UVC 包接收
  │    │    ├─ FID/EOF 组帧
  │    │    └─ 返回 JPEG 与采集 profile
  │    ├─ 检查 SOI/EOI/最小长度
```

```
  |    ├─ 错误时复位 FID 连续性并有限重试
  |    └─ 更新 CaptureStats
  ├─ 写入非活动 latest-frame 缓冲
  ├─ 发布宽高、长度、时间戳与 sequence+1
  └─ 唤醒等待新序号的消费者
```

2.4.3　前台摄像头与 JPU 阶段

```
VpssRgbPipeline::next_inner()
  ├─ 构造 CameraIonFrameRequest(abi=2, last_sequence, source_ion_fd)
  ├─ ioctl(CVI_CAMERA_IOCTL_GET_LATEST_YUV_ION)
  |    └─ CviUsbCamera::latest_yuv_to_ion()
  |         ├─ wait_for_sequence(last_sequence)
  |         ├─ 解析并验证 source ION
  |         ├─ 复制小尺寸压缩 JPEG，释放 latest-frame 锁
  |         ├─ JpuDecoder::decode_planar_into()
  |         |    ├─ 复制 JPEG 到 JPU 位流缓冲
  |         |    ├─ 清零 1,024 字节尾部余量
  |         |    ├─ 编程 BBC/DPB/采样格式寄存器
  |         |    ├─ 启动并等待 JPU 完成
  |         |    └─ 输出 Y/Cb/Cr 到 source ION
  |         ├─ 失败时丢弃 sequence、复位并等待更新帧
  |         ├─ 返回三平面偏移/长度/步长、jpu_decode_us
  |         └─ mark_consumed(sequence)
  └─ 用户态校验 format=YUV422P、640×480 和平面范围
```

2.4.4　VPSS 阶段

```
VpssRgbPipeline::next_inner(execute_vpss=true)
  ├─ 构造 VpssRunYuv422pRgb(abi=1)
  |    ├─ source fd + Y/Cb/Cr offsets/strides
  |    ├─ crop=(0,0,640,480)
  |    ├─ content=(0,80,640,480)
  |    ├─ destination fd + R/G/B offsets/strides
  |    └─ border + timeout=100 ms
  ├─ ioctl(CVI_VPSS_IOCTL_RUN_YUV422P_RGB)
  |    ├─ 解析 ION 并校验范围
  |    ├─ 配置 YUV422P 输入地址、采样与步长
  |    ├─ 配置 BT.601 CSC、SC_V1 scaler、letterbox
  |    ├─ 配置 RGB planar ODMA
  |    ├─ 启动任务并记录 hardware_start
  |    ├─ 3 ms 有界完成观察 / IRQ 等待
  |    ├─ 记录 hardware_done 和调试状态
  |    └─ 返回 status/IRQ/timestamps
  ├─ split_vpss_timing()
  └─ 返回 destination physical address 与 RGB planar 视图
```

2.4.5　TPU 输入与模型前向阶段

```
YoloModel::infer_vpss_rgb_timed()
  ├─ 验证 destination 物理区间和张量契约
  ├─ CVI_NN_SetTensorWithAlignedFrames()
  |    ├─ libcviruntime 构造 TDMA-only 命令
  |    ├─ ioctl 提交 / TPU direct execution
  |    ├─ run_one(prepare → fire → IRQ → finish)
  |    └─ 将外部 RGB planar 导入模型输入
  ├─ CVI_NN_Forward()
  |    ├─ 提交 compute 命令流
  |    ├─ TDMA 搬运权重/特征图
  |    ├─ TIU 执行卷积、激活和检测头
  |    ├─ IRQ 唤醒提交线程
  |    └─ 返回输出张量
  └─ 分别记录 preprocess_us 与 forward_us
```

2.4.6　后处理与控制阶段

```
YoloModel::get_detections()
  ├─ CVI_NN_TensorPtr(output)
  ├─ FP32 直接只读切片
  ├─ parse_yolov8_output()
  |    ├─ 按通道读取 cx/cy/w/h/score
  |    ├─ 置信度阈值优先过滤
  |    └─ 构造少量有效 Detection
  ├─ nms(iou=0.5)
  └─ correct_yolo_boxes(640×640 → 640×480)

Robot::handle_detections()
  ├─ 选择最大面积网球框
  ├─ 计算 cx 和 area_ratio
  ├─ 搜索 / 左转 / 右转 / 前进 / 近球微调
  ├─ 连续两帧确认抓取条件
  └─ 进入机械臂抓取与持球状态
```

视觉链路的路线确定之后，系统层的第一条线是把通用子系统的固定开销与 Linux 逐项对齐：同一份可执行文件在两套系统上的差别，几乎全部落在这类“每次操作都要付”的成本上，其量级与取值范围构成本节余下部分的出发点。

2.5　内核优化：从“每个操作要付多少固定成本”入手

内核层的工作目标很具体：项目立项时的横向对比显示，同一块板、同一份静态 riscv64 ELF 上，StarryOS 在**每次系统调用、每次进程创建、每次缺页、每次文件读**这些固定开销上比厂商 Linux 贵一到两个数量级，而在内存带宽、大块顺序写、冷启动这些**带宽或总量型**指标上并不落后。因此这一阶段不做功能补全，而是沿着“把每次都要付的固定成本降下来”这一条主线，逐个热点做归因、改机制、再实测。

2.5.1　优化之前的总体画像

优化起点（2026-10-04 的 `ours` 基线镜像）与 Linux 的对照如下。这张表决定了优化的 优先级：凡是差距在一个数量级以上的，都是“每个操作都要付的固定成本”；凡是基本持平 的，都是带宽型或总量型指标。

指标	Linux 5.10.4	优化前	差距
`getpid`（空系统调用）	422.5 ns	10 560 ns	25.0×
`pipe_wr`（4 KiB 往返）	4 489 ns	46 819 ns	10.4×
管道 ping-pong（含一次切换）	26.0 µs	445.7 µs	17.1×
`fork`+`exit`+`waitpid`	338.1 µs	7 497 µs	22.2×
`fork`+`execve`+`waitpid`	2 407 µs	43 537 µs	18.1×
上下文切换	10.97 µs	226.8 µs	20.7×
64 MiB 匿名首触（默认 4 KiB）	7 899 ns/页	1 302 123 ns/页	165×
ext4 `open`+`read`(1 KiB)+`close`	42 µs	10 320 µs	246×
8 MiB 顺序写 + `fsync`	790 ms	7 961 ms	10.1×
缓存命中大读（缓冲已触碰）	671.6 MB/s	374 MB/s	1.80×
缓存命中大读（每轮新缓冲）	271.1 MB/s	50.2 MB/s	5.40×
内存 `memcpy` / 顺序读带宽	1.024 / 1.488 GB/s	0.984 / 1.420 GB/s	1.04× / 1.05×
端到端冷启动（`bootm`→提示符）	33.2 s	13.21 s	我方更快 2.5×

表 2: 优化起点的横向对照。前九行是“每个操作都要付的固定成本”，差距都在一个数量级以上；末四行是带宽型或总量型指标，本来就不落后

这张表给出三点判断，后文的取舍都围绕它们展开：

1. **系统调用入口是最大的单点**。一个什么都不做的 `getpid` 要 10.56 µs，是同板 Linux 的 25 倍。它的处理体只取一个已在内存里的进程号，所以这 10.56 µs 几乎全部是 “进入内核、返回用户态”这条固定路径。
2. **调度与进程生命周期的固定开销排在第二**。`fork`+`exit` 22 倍、上下文切换 20 倍、管道往返 17 倍、exec 18 倍，全都在同一个量级上；这条链路要拆成“页表复制 / 首次入队 / 子进程析构 / 每次切换”四段分别处理。
3. **内存的“每页固定成本”比内存带宽问题严重得多**。 64 MiB 首触慢 165 倍，而 `memcpy` 带宽只差 4%。据此判定，差距来自每页一次事务的簿记开销；同板带宽类指标两侧接近。

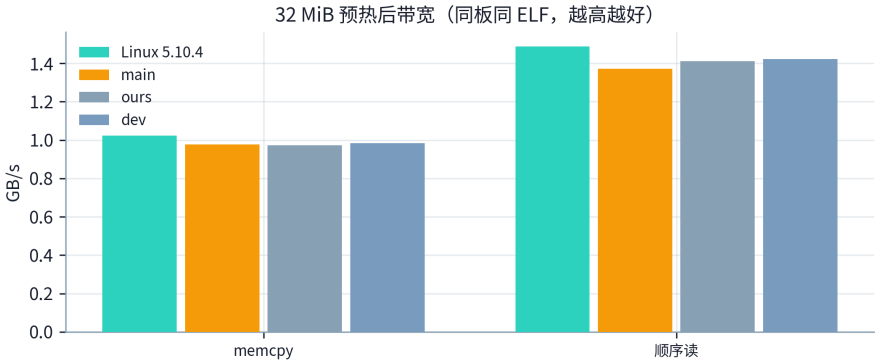


图 3: 同板对照下的内存带宽。`memcpy` 与顺序读在各方之间只差几个百分点——它说明本板的差距不在内存子系统，而在“每次操作要付多少固定成本”

还有两条“看起来很小、实际很贵”的发现：ext4 上每次 `open+read+close` 要付一次 约 10 ms 的同步日志提交（Linux 只在内存里改访问时间），以及启动期一个固定 2 s 的 DHCP 阻塞等待。

2.5.2　机制族清单

下表是本阶段实际落地的改动。这里按**机制族**归并，而不是按提交逐条罗列：同一族里的 几条改动作用在同一个数据结构或同一条路径上，彼此耦合、收益也相互叠加，拆开单看 容易重复计数。八族改动都满足两个约束：**机制上的改动都独立实现**，并且**每一族都有“优化前/优化后”的同板 A/B 数据**。

编号	优化族	族内具体改动
K1	系统调用返回路径	把“每次返回用户态都刷指令缓存”收敛到用户映像可能陈旧的少数事件上（`exec`、`PROT_EXEC`、切换地址空间）
K2	地址空间操作的机械开销	① 取消每个叶子的第二次页表走查；② 所有者发布去重由 $O(n^2)$ 改为 $O(n \log n)$；③ 地址空间退休不再逐页摘除
K3	exec 的页物化	① static-PIE 只物化带写权限的段，只读段改按需缺页；② 文件后端按连续 run 批量 pin 与域预留
K4	缺页的每页固定成本	① COW 页索引不再每次全量重建（去掉每次插入的两遍 $O(n)$ 存活计数）；② 内存操作从逐字节改字长循环
K5	顺序缺页的窗口批量物化	① 严格升序的私有匿名缺页按 64 KiB 窗口整段物化；② 窗口大小做成运行期旋钮并加短流爬坡；③ 窗内帧整块分配 + 子租约；④ 窗内索引按 run 批量登记
K6	文件读路径	① pin 住页缓存页后直接拷给调用方（去掉暂存中转）；② 用户内存准备整段化；③ 延迟失效路径不再逐页刷 TLB；④ 修复复用帧被旧墓碑遮蔽的身份查询
K7	存储写回与元数据	① 脏页按连续 run 合并写回；② 页缓存上限 512 → 4096 页；③ `close` 路径按 `relatime` 处理访问时间、不再同步提交
K8	启动等待	把启动期固定 2 s 的 DHCP 阻塞等待改成带截止时间的轮询

表 3: 内核改动按机制族归并。族内各条作用在同一条路径上，收益相互叠加，因此按族给出对照数据。各族的工程实现见 3.4 节，实测对照见 4.5 节，逐族提交记录见 2.8 节

除内核之外，本项目的系统层还包含两条独立的工作线，它们的路线同样在本章给出：一条是 把 SG2002 的板级外设接通、并把“改内核—上板—取数”这一循环做成可重复执行的通道 （板级 I/O 与上板工作流）；另一条是把厂商固件里那颗 RISC-V 实时小核从“存在但未被 使用”变成 StarryOS 可管理、可使用的第二个可编程执行体，并由它接管底盘串口这条 硬实时链路。三者共用同一套评测口径与同一块板。

内核优化处理的是“每次操作要付多少”这一面；它不改变另一类事实：单核平台无法承诺硬实时，也无法在自身崩溃时保住“停车”这个动作。第二颗核因此进入视野——把实时职责 与故障域一并移出通用核。

2.6　实时小核：单核平台的实时缺口与冗余缺口

SG2002 是一颗面向低功耗视觉终端的异构 SoC：除一颗 1 GHz 的大核之外，片内还有一颗 RISC-V C906@700 MHz 的“协处理器”（本文称 C906L）。厂商把它的固件固定在 DRAM 末尾 2 MiB，出厂镜像里也确实在跑，但它既不参与 StarryOS 的调度，也不被内核的任何子系统 管理，对操作系统而言是一颗“存在但未被使用”的核。本项目这条工作线做的事情，是把这颗核 从“厂商固件里自转”变成“可被 StarryOS 管理、可被应用使用”的第二个可编程执行体：先打通 核间通道并把它做成正式驱动，再让这颗实时核接管一条真实的硬实时链路（底盘串口控制）， 最后用同机、同负载、同链路的 A/B 数据回答它到底带来了什么。结论可以先用一句话概括：**收益不在算力，而在确定性与故障兜底**——底盘控制周期的抖动从负载下 p99 恶化 1.5 ms 降到 0 抖动（控制周期 100 ms，全部样本落在同一 tick 刻度），并且在大核失效或卡死时， 车能在心跳窗口内自主停车。

2.6.1　问题：实时缺口与冗余缺口

SG2002 在这一版板卡上只有一颗通用核在跑操作系统。C906 与 Cortex-A53 两颗大核由引脚 二选一，实战配置里不会同时出现；小核虽然存在，却被厂商固件独占。于是整台车的软件栈 ——摄像头取流、JPEG 解码、TPU 推理、跟踪与决策、串口下发——全部排在同一个核的时间片 上，实时工作与非实时工作共享同一个调度器。控制链路的延迟不再由代码决定，而由最拥挤的 那一刻决定。

这个缺口可以用数字说清。空载时大核直控的控制周期 p99 相对 100 ms 只偏 0.21 ms，看起来 完全够用；一旦有 3 个纯计算负载把核压满，p99 就恶化到 **1.54 ms**（约 8 倍），同一时刻串口 ioctl 的往返从 120–390 µs 涨到 **3 670 µs**。对“轮速闭环”这类机构而言，这种由负载 决定的长尾属不可承诺的时序问题，与性能高低无关：机构容差是否被突破，取决于当时有没有 别的程序在跑。小核接管之后，同一台车、同一条链路、同样的负载，控制周期在三种负载下 都是 0 抖动——全部样本落在同一个 tick 刻度上。

比实时抖动更严重的是**单点失效**。车在运动时，底盘板执行的是“最后一条速度指令一直有效”，它不会主动停车；如果唯一的通用核进入内核崩溃、关键进程被杀、或者被高优先级负载长时间 饿死，那么限速、看门狗与停车指令会随它一起消失——车保持最后一条 PWM 继续冲，直到撞上 东西或有人断电。该路径已在实车上复现：应用退出后不再下发任何指令，轮子一直转；整芯片软复位也不停，因为底盘板仍有电、寄存器仍保存着最后的速度。反过来，当链路交给 独立运行 RTOS 的小核，大核停止喂心跳约 500 ms 后小核就自己把两轮 `STOP` 并保持静默；即便大

核完全挂死，另一个只读判据（小核 PC 恒为 0、控制计数不再前进）也能让大核侧脚本 把 UART1 抢回来并补发停车序列。这条兜底链已经用“真的把核心停掉”的方式演练通过。

因此使用小核的理由可以归纳为三条，且都与算力无关。其一是**实时职责的卸载**：底盘串口链路的所有权移交小核后，周期性下发、状态回读与超时判定都不再依赖大核的时间片，大核 只需要交换目标与状态；对小核来说那只是每 100 ms 一次的串口任务，占用极小。其二是 **故障域的隔离**：控制与兜底跑在一颗不参与通用调度的核上，大核的崩溃、卡死与重启不再 同时带走“停车”这个能力。其三是**低成本的加法**：小核与大核同片集成，不增加 BOM、 不增加板卡与连线、与内核共享同一块 DDR 可直接共享内存，并且可以被操作系统当作一个 设备来管理——相比之下，外挂一颗 MCU 虽然也能解决实时与冗余问题，但要付出额外的器件、 接口、电源以及两套固件维护的代价。

同样要说明它的代价：小核算力低于大核、没有 TPU 访问、外设面窄，工具链与引脚归属都存在 未知项。据此对该核的定位是**只接实时与兜底职责**——底盘链路所有权、固定周期采样、停止与看门狗、上电快速就绪；推理、图像预处理与电机闭环仍然留在大核与底盘板上。

2.6.2　平台事实：这颗核在哪、归谁管、能碰什么

C906L 与大核不是同一个 hart。大核由引脚 `GPIO_RTX` 二选一（RISC-V C906@1 GHz 或 ARM Cortex-A53@1 GHz），小核固定为 RISC-V C906@700 MHz，无引脚可换。厂商的头文件 用 CPU0/CPU1/CPU2 区分三颗核：CPU0 = Cortex-A53、CPU1 = C906B（当前 StarryOS 运行核）、 CPU2 = C906L（小核）。设备树只描述一个大核，`boot-hartid=0`，因此内核既看不到也不会 调度这颗小核。

小核的固件有固定落点。厂商内存布局把 256 MiB DRAM 的最后 2 MiB 划给 C906L FreeRTOS， 起始地址 `0x8FE00000`，内核可见内存因此是 254 MiB；这与现有 SG2002 设备树的 `memory@80000000`（0xFE00000）与 `fast_image`（`0x8fe00000 + 0x200000`、 `ion-size = 0x1600000`）逐项对上。小核由 FSBL 在 M 模式释放一次，之后没有任何软件 再动它；操作系统与它之间只有一条核间通道。

这条通道是 `ap_mailbox`（`0x01900000`，4 KiB）。它把 64 字节的上下文内存切成 8 个 8 字节槽位，与 8 个中断位一一对应；每槽放一个 `cmdqu` 信封（8 字节：`ip_id`、7 位 `cmd_id` 与 1 位 `block`、两个方向有效位或超时、32 位 `param_ptr`），真正的负载经 `param_ptr` 指向共享内存。协议头文件是公开的，**不需要逆向**。控制面同样不神秘：厂商 Linux 走 SBI 的自定义 BASE 扩展（功能号 0x7/0x8/0x9），但这套服务在本项目的构建链里 并不存在；实测表明 **S 模式可以直接写小核的复位与入口寄存器**（`SEC_SYS_BASE`、`RSTC`）， `sg200x-bsp` 已经把等价序列封装成 `SecSys::start_secondary_core()`。也就是说，启动或 复位小核不需要改 M 模式固件，缺的只是通道驱动。

小核的外设面很窄。TRM 没有任何描述 C906L 外设归属的章节，只能从厂商 RTOS 固件反推： `driver/` 下有 `gpio`、`i2c`、`uart`、`pinmux`、`cif`、`jenc`、`sensor`、`rgn`， `task/` 下有 `isp`、`vi`、`vcodec`、`camera`、`audio`——换言之，厂商的设计意图就是“摄像头 + ISP + 视频编解码 + 音频 + 慢速外设的协处理器”。相反，小核固件里**没有 USB、 没有 TPU/NPU、没有 SD、没有以太网**，这说明**不应把它当算力或推理单元使用**。

项目	事实
大核 / 小核	C906@1 GHz 或 Cortex-A53@1 GHz（引脚选择）/ RISC-V C906@700 MHz 协处理器
CPU 编号	CPU0 = CA53、CPU1 = C906B（运行核）、CPU2 = C906L
固件落点	DRAM 末尾 2 MiB，`0x8FE00000`；内核可见内存 254 MiB
核间硬件	`ap_mailbox` @ `0x01900000`，4 KiB，8 槽 × 8 字节 `cmdqu` 信封
控制路径	S 模式直写 `SEC_SYS_BASE` / `RSTC` 即可复位与重载（已实测）
小核外设面	gpio / i2c / uart / pinmux / cif / jenc / sensor + isp / vi / vcodec / audio；无 USB / TPU / SD / 以太网

表 4: C906L 的平台事实。这组事实决定了后续所有设计：通道走 mailbox、复位靠 S 模式直写、职责只接实时与兜底

小核解决了实时与兜底, 但它与前述两条系统层路线都依赖同一件前提: 板子本身可被反复 刷写、反复上电，且取数过程可脚本化。板级 I/O 与上板工作流这条线就是为这一前提服务，它不主张性能优势，只主张被测可达与可重复。

2.7　板级 I/O：三条约束如何决定技术路线

板级 I/O 与上板工作流这一层承担两件事: 把 StarryOS 与 SG2002 的板载外设接通，以及为“改内核—上板—取数”这一循环提供可重复执行的通道。SG2002（SOPHGO CV18xx 系列）在 256 MiB DDR3 之外提供三颗处理器：大核为 RISC-V C906@1 GHz 与 ARM Cortex-A53@1 GHz 二选一（由引脚 `GPIO_RTX` 的外部电路决定），协处理器固定为 RISC-V C906@700 MHz（下称 C906L），另有 RTCSYS 内一颗独立供电的 8051 子系统。

与常见的“单核 SoC ＋ 网口 ＋ 串口”不同，本板把若干约束压到了栈底，而这三条约束 直接决定了本层的技术路线:

1. **C906L 的固件常驻 DRAM 末尾 2 MiB**（`0x8FE00000`），内核可见内存因此只有 254 MiB；而小核并非点缀——它是本板上唯一能被软件要求“复位整颗芯片”的执行者。这条事实决定了系统切换不能沿用“内核 `reboot(2)` 直接触发固件复位”的常规做法， 而要借道小核（见 2.2 节 中“实时小核”一条）。
2. **大核侧没有 `/dev/mem` 一类物理内存窗口**，任何直接写外设寄存器的板级动作都必须 以内核接口的形式存在，而不能像在通用 Linux 上那样从用户态投递。这条约束把 “把复位序列做成受控内核接口”从一个可选项变成了唯一解。
3. **交付阶段实际可用的控制台只有一路 115200 8N1 串口**，它同时承载内核日志与用户 shell。因此控制台策略是“只保留一路、把工具做厚”，而不是去开第二路串口 （器件树里 UART1–UART3 虽然可用，但分别落在 JTAG、IIC0 与 SD1 焊盘上，占用的 代价见 5.8 节）；同样因为这一路带宽有限，约 16 MB 的 FIT 映像不能经它传输， 镜像通路只能另找介质。

三条约束叠加之后，本层的工作重心落在**上板工作流本身**：把它做成可编程、可复现的对象， 而不是让某个驱动单独跑起来。据此形成的路线是四段:

工作流**:以“提示符出现在接收缓冲区末尾”为唯一判据, 把命令执行、启动锚点打点与 Linux 侧会话抓取做成三件可脚本化的工具； 介质通路**：用 U-Boot 的 USB 大容量存储（UMS）模式把整张 SD 卡以 U 盘形式交给 宿主机，FAT 启动分区用 mtools 写、ext4 根分区用 `debugfs` 写，

全程不需要主机提权、不需要板上网络；系统切换：**以大核侧的 `cmdqu` 命令队列把复位请求交给 C906L 固件，由固件按 FSBL 的序列完成整芯片软复位，使“回到 U-Boot”变成 shell 里的一条命令；使能链**：从无线网卡上电、固件下载、WPA2 关联到板上 `sshd` 打通一条登录通路，但把“串口可独立完成全部操作”作为硬约束——网络与 SSH 定位为加速项而非必要项。

最后一条取舍与“网络优先”的常规做法恰好相反，却更贴合无人值守的实际条件：现场热点为移动热点，主机随时可能离场。因此本层的一切设计都以“拔了串口线之后板子仍然可被反复刷写与被测”为准，这也正是其余各章相对 Linux 的持平与领先数据能够在真机上稳定测得的前提。

2.8　代码原创说明

本节说明本项目在内核侧所提交改动的归属与核对入口。总的立场是：**上游已有的能力不主张为新增贡献，本分支的改动都能落到一个具体提交上，且都带同板 A/B 数据**。

2.8.1　提交清单

下表按第 2.2 节给出的机制族（K1–K8）列出对应提交与关键验证。同一族可能落在多个提交上：它们作用在同一条路径上、收益相互叠加，因此按族给出对照数据，逐提交细节见仓库历史。

编号	优化族	提交与关键验证
K1	系统调用返回路径	`0a627ecb7`；宿主单测 + 板端全项 + 78 轮 `exec` 压力
K2	地址空间操作的机械开销	`10e67ae8d`（取消每叶第二次走查 + 去重复杂度）、`5491b9956`（退休不再逐页）；`try_clone` 1 747 → 1 528 µs，退休 3.57 → 0.41 ms
K3	exec 的页物化	`e13b6b962`（只读段直映）、`f5d357d4a`（按 run 批量 pin + 只物化可写段）；exec 内核侧 32 → 4.5 ms，缺页数 200 → 79 页/次
K4	缺页的每页固定成本	`130a98059`（页索引）、`760ef6d08`（字长内存操作，含一处未定义行为修复）；64 MiB 首触 8.3×，清零 6.8×
K5	顺序缺页的窗口批量物化	`bcd2d4e71`（窗口预取）、`371ffc0dd`（帧整块 + 子租约）、`792ffed5f`（索引按 run 批量，配套正确性修复 `dd8f6f0e1`）；防膨胀 1.00×/0.96×，块命中率 99.97%
K6	文件读路径	`cc56cd47e`（pin 直拷）、`11146c8a6`（整段物化）、`584b07fa7`（整段快路径 + 注册表 O(n) 化）、`c16994af2`（不再逐页刷 TLB）；三种分块校验和一致，TLB 自检 0 不匹配
K7	存储写回与元数据	`1db22090c`（写回批量）、`29e264fd0`（页缓存上限）、`a3cd50ece`（访问时间策略）；`fsync` 7.96 → 0.93 s，写吞吐 17.6×
K8	启动等待	`ce9b42a92`；用户态标记 −1.2 s

表 5: 内核改动按机制族归并后的提交清单。族内的多个提交作用在同一条路径上，单看任何一个都低估了整族的收益

2.8.2　原创性与引用边界

三条约束保证这些改动可以被独立核对。

第一，**每条结论都能落到一个提交上**，并且都带“优化前/优化后”的同板 A/B 数据；数据取法与口径见 4.5 节。

第二, **机制上的改动在本分支独立实现**, 且都遵循同一条约束: 不改变对外可观察的语义。其中三处确实动到了对外语义——返回用户态时的指令缓存刷新策略、延迟失效路径的 TLB 失效时机、文件的访问时间策略——都额外做了专项自检（多二进制 `exec` 压力、固定地址 反复映射的 TLB 一致性探针、多分块读校验和比对），并保留运行期回退开关。

第三, **与上游的关系是增量而非替换**。本分支的基线是 TGOSKits 的 `board-aka-00-sg2002` 配置; 改动以补丁形式叠加, 不改变其它板卡与应用的行为。上游已有 的能力——TPU 与 ION、USB 主机栈、串口以及板卡用例——在本项目中原样复用，不在本节 主张为新增贡献；本节与 4.5 节列出的收益，都只针对本分支明确改动的部分。

第 3 章　项目介绍

3.1　功能介绍

本项目交付的是一套可运行、可复现的机器人系统，它在结构上分成应用层与系统层两组模块。应用层是乒乓拾取机器人的视觉推理与运动控制链路，包含相机取流、硬件解码、几何变换、神经网络推理、检测后处理与追球控制；系统层包含三条线：内核优化（把每次操作都要付的固定成本降到接近 Linux 的水平）、实时小核（用第二颗核接管底盘串口，补上实时与冗余），以及板级 I/O 与上板工作流（把板子变成可无人值守刷写、可暖重启的目标平台）。三条线共用同一块板与同一份用户态 ELF，彼此不重叠。下面按这个顺序给出各模块的功能与实现。

3.1.1　xxx 模块

3.1.2　xxx 模块

本节从工程实现角度介绍视觉链路与机器人闭环。第 2.3 节已经说明各硬件模块的职责和数据流向，这里进一步说明从基线瓶颈出发的推进顺序：如何逐步完成驱动接口、共享缓冲区、完成事件调度和用户态后处理的落地，并给出每一步可以复核的实现位置与性能效果。

3.2　推理链路优化的工程实现

3.2.1　以分段探针驱动链路重构

项目早期虽然已经能够完成网球检测，但采集、软件预处理、TPU 输入和模型前向只以少量总耗时呈现，无法判断延迟究竟来自硬件执行、线程排队还是数据复制。起先在用户态的摄像头请求、VPSS 调用、`CVI_NN_SetTensorWithAlignedFrames()`、`CVI_NN_Forward()`和检测后处理边界插入单调时钟探针；随后在 StarryOS 内核中记录 JPU 解码起止、VPSS 任务入队与硬件完成、TPU 最终启动寄存器写入、完成中断和等待线程恢复等时间戳。由此，一帧视觉任务被拆成“摄像头等待、JPU 解码、VPSS、输入 TDMA、TPU 计算、后处理和其他开销”七个互不重叠的区间。

分段结果改变了优化顺序：代表性 VPSS 墙钟为 44.646 ms，硬件实际运行 1.480 ms；异常 TDMA 任务的内核总时间为 45.253 ms，真实搬运为 0.838 ms；TPU 计算路径中还包含 22.873 ms 的 worker 排队。早期瓶颈集中在完成事件、等待队列和缓冲区交接，相关软件延迟被计入了硬件阶段。后续每一项改造均要求同时观察用户态墙钟和内核分段时间，避免仅通过移动计时边界获得表面收益。

3.2.2　将同步取帧改造成有界的最新帧流水线

摄像头侧在 `cvi_usb_camera.rs` 中实现 `AsyncCapture` 与 `LatestFrameStore`。用户态通过 `CVI_CAMERA_IOCTL_START_ASYNC` 启动后台 UVC 采集任务，生产者持续接收 USB 数据包并组装 MJPEG 图像，但只有在 SOI、EOI 和最小长度校验通过后，才把完整帧发布给前台。`LatestFrameStore` 使用两个 JPEG 缓冲区：后台始终写入非活动缓冲区，写完后再切换发布索引并递增 `sequence`，因此消费者不会读到正在修改的半帧。

消费者在 CameraIonFrameRequest.last_sequence 中携带上一帧序号，内核只在新序号出现时返回；若前台 处理速度落后，则直接取得最新帧，无需按先进先出顺序清空历史队列。为了避免 UVC 线程在单核 C906 上 持续抢占 CPU，驱动又加入“一帧预取”背压：只有当前帧已被 mark_consumed() 确认消费后，后台才捕获 下一帧。该实现同时满足采集与推理重叠、帧龄受控、缓冲区容量有界和无效帧有限重试四项要求，解决了 同步取帧等待与历史帧积压两个不同问题。

3.2.3　JPU 硬件解码的具体实现

用户态在 VpssRgbPipeline::next_inner() 中构造 CameraIonFrameRequest，把 source ION 文件描述符、缓冲区 容量、上一帧序号和超时时间交给 CVI_CAMERA_IOCTL_GET_LATEST_YUV_ION。摄像头驱动首先等待严格大于 last_sequence 的新帧，再从 ION 分配表解析物理基址，检查目标区域是否位于分配范围内且满足低 4 GiB 的 DMA32 约束。为避免在 JPU 执行期间长期持有最新帧锁，驱动将几十 KB 的压缩 JPEG 复制到临时位流对象后释放 采集锁；614,400 字节的解码图像不会经过 CPU 中间缓冲。

随后，JpuDecoder::decode_planar_into() 把 JPEG 复制到 JPU 位流缓冲区，解析 SOF 中的色度采样因子，并按 当前摄像头的 4:2:2 格式配置位流控制器、解码图像缓冲区和 Y/Cb/Cr 平面地址。JPU 以 source ION 物理地址为 输出基址，通过 DMA 依次写入 307,200 字节 Y 平面和两个 153,600 字节色度平面。驱动等待完成 IRQ 后返回 y_offset、cb_offset、cr_offset、平面长度、Y/C 步长、帧序号和 jpu_decode_us，用户态无需推断 内存布局即可构造下一阶段的 VPSS 请求。

为解决长时间运行中的 Bitstream exhausted，位流有效长度按 256 字节页向上取整，末尾额外保留并清零 1,024 字节安全区域，允许硬件在解析 EOI 时进行有限预取。解码失败时，驱动不会反复提交同一损坏 JPEG，而是标记当前序号已消费、复位 JPU 并等待下一帧，最多尝试三个新帧；用户态还保留采集重启和摄像头硬复位 两级恢复。该实现使 JPU 平均解码时间稳定在 1.645 ms，并避免单个损坏帧阻塞整条视觉链路。

3.2.4　VPSS 几何转换的具体实现

JPU 输出为 YUV422P 三平面，而原 VPSS 接口主要面向 Y 与交织 UV 组成的 NV12 双平面格式。项目在内核新增 Yuv422PlanarFrame、SourceFrame::Yuv422Planar 与 VpssRunYuv422pRgb，并通过 CVI_VPSS_IOCTL_RUN_YUV422P_RGB 一次提交源 ION、目标 ION、三个输入平面、三个输出平面、裁剪区域、内容区域、边框颜色和超时参数。接口固定携带 ABI 版本和结构体大小，内核在写寄存器前检查所有平面范围、 步长、几何边界、地址加法溢出以及目标平面是否非法重叠。

硬件配置中，输入格式设置为 SCL_FMT_YUV422：Y 平面地址来自 y_offset，步长为 640 字节；Cb 和 Cr 分别 使用独立地址，步长为 320 字节，色度水平方向二分之一采样、垂直方向保持 480 行。VPSS 内部使用 BT.601 有限范围矩阵完成 YUV 到 RGB 转换，再把 640×480 有效图像放置到 640×640 画布的 (0, 80, 640, 480) 区域，顶部和底部各填充 80 行。输出端配置为 RGB planar 和 ODMA，R、G、B 平面偏移依次为 0、409,600 和 819,200 字节，步长均为 640 字节，结果直接写入 destination ION，内存布局正好对应模型的 NCHW 输入。

驱动记录任务入队、硬件启动、完成 IRQ 和 ioctl 返回时间，并保存 IMG、AXI、SCALER 与 ODMA 状态用于故障 诊断。项目还把 VPSS 输出与软件 YUV 转换逐像素比较：各通道均值一致，单像素最大误差不超过 1，R/B 交换 假设则出现明显误差，从而确认颜色矩阵、通道顺序和

letterbox 区域均正确。完成调度长尾修复后，VPSS 平均时间为 1.643 ms，CPU 不再执行颜色转换、缩放、边框填充和通道重排。

3.2.5　TDMA 数据传输的具体实现

模型注册后，程序首先读取输入张量契约，确认形状为 `1×3×640×640`、元素类型为 UINT8、通道顺序为 RGB planar、布局为 NCHW，并核对量化零点、均值、缩放系数和 aligned 属性。VPSS 写入 destination ION 后，用户态把三个颜色平面的物理地址、步长和尺寸封装为外部帧，调用 `CVI_NN_SetTensorWithAlignedFrames()`。 CVITEK 运行时据此生成 TDMA-only 命令，由 TDMA 把外部 RGB planar 数据导入模型要求的输入存储布局；该 阶段只完成输入张量准备，不包含卷积、激活以及模型内部权重和特征图搬运。

项目曾把 destination ION 物理地址直接写入输入张量，源缓冲和张量指针的字节哈希完全一致，但推理结果 为零检测。该实验说明运行时还负责设备内存类型、对齐步长、地址转换和命令流补丁，字节内容相同无法 替代完整张量描述。因此，正式链路保留官方导入接口，并把其耗时独立统计为“TPU 输入 TDMA 搬运”。

内核根据 `bd_desc_count=0` 且 `tdma_desc_count>0` 把输入任务识别为 TDMA-only。针对这类硬件时间不足 1 ms 的短任务，IRQ 处理路径将等待线程前插到 RR 运行队列，并把异常兜底轮询从 100 微秒改为 2 ms，防止轮询在 真实 IRQ 前移除等待者。代表性内核任务时间由 45.253 ms 降至 0.978 ms，正式测试的用户态 TDMA 平均为 1.114 ms；这 1.114 ms 是保持模型输入正确所需的真实硬件搬运和固定软件开销，并非被隐藏或取消的统计项。

3.2.6　TPU 硬件推理的具体实现

初始化阶段，用户态调用 `CVI_NN_RegisterModel()` 注册 640×640 INT8 模型，通过 `CVI_NN_GetInputOutputTensors()` 取得输入输出张量并完成上述契约校验。内核侧对照 SOPHGO 官方 CV181x 驱动补齐 `Cv181xTpuSoc` 生命周期：打开 TPU 与 FAB 时钟门控，按低有效语义完成 TDMA、TPU 和 TPUSYS 复位， 从 PLL、时钟选择器和分频寄存器计算真实频率。实机配置为 TPU 700 MHz、FAB 900 MHz；固件已处于目标 频率时，驱动只验证寄存器状态，不重复改写共享 PLL。

每帧输入导入后，`CVI_NN_Forward()` 把模型编译生成的 TDMA 与 TIU 描述符提交到 `/dev/cvi-tpu0`。正式路径 由原提交线程取得可睡眠的 `TPU_RUN_LOCK` 并直接执行 `prepare → run → wait → finish`：`prepare` 完成 地址解析、时钟和中断状态检查；`run` 写入描述符地址、DCM 和中断掩码，并以最终 `TDMA_CTRL` 启动写作为 硬件计时起点；线程随后在 IRQ 等待点睡眠；完成中断保存 TDMA/TIU 状态并唤醒同一线程；`finish` 检查错误位、 发布完成结果并直接返回运行时。该路径删除了“提交线程—worker—提交线程”的两次任务所有权转移，同时 保留旧 worker 实现作为 A/B 回退。

内核探针显示，正式模型 compute 平均 `fire_to_irq` 为 39.361 ms，占内核 compute 墙钟的 97% 以上；用户态 `CVI_NN_Forward()` 平均为 41.309 ms。两者差额主要包含运行时提交、系统调用、IRQ 后线程恢复和返回 用户态，说明 worker 排队等软件长尾已基本消除，当前主要成本来自模型内部 TIU 计算、TDMA 访存与存储系统访问。

上述四个阶段通过两级 ION 缓冲区连接：2 MiB 的 source ION 由 JPU 写入、VPSS 读取；按页对齐的 `3 × 640 × 640` 字节 destination ION 由 VPSS 写入、TDMA 读取。驱动从 ION 文件

描述符解析受管理的物理地址，按照“JPU 写入—VPSS 读取—VPSS 写入—TDMA 读取—TPU 完成”的顺序转移所有权，消除了 YUV422P 与 RGB planar 在设备之间的 CPU 整图复制。仍保留的压缩 JPEG 小缓冲复制用于解除锁依赖，因此整条链路准确描述为 “大图低拷贝”。

3.2.7　加速器完成事件的统一调度优化

JPU、VPSS、TDMA 和 TPU 的硬件执行方式不同，但都存在“硬件完成—IRQ 发布—等待线程恢复—返回用户态” 这一共同控制路径。VPSS 通过 3 ms 有界完成观察避免正常任务错过长时间片，TDMA-only 通过 IRQ 队首唤醒 避免不足 1 ms 的短任务排在 RR 队尾，TPU compute 则通过同步直通删除 worker 所有权交接。三种策略均保留 绝对超时、状态寄存器检查和轮询兜底，优化的是正常完成路径，不会以取消异常恢复换取低延迟。

优化对象	优化前	优化后	主要消除的开销
VPSS 单次墙钟	44.646 ms	1.637 ms	完成后调度等待
TPU `submit_to_run`	22.873 ms	0.002 ms	提交线程到 worker 排队
TPU `done_to_user`	2.958 ms	0.116 ms	worker 到提交线程交接
TDMA-only 内核任务	45.253 ms	0.978 ms	轮询竞态与 RR 队尾等待

表 6: 推理链路关键调度优化的代表性 A/B 结果

表中数据来自对应问题的独立 A/B 测试，用于说明各项改造消除的软件区间，不应相互相加作为同一轮端到端 结果。正式测试仍以统一测试程序输出的单帧阶段均值为准。

3.2.8　TPU 输出解析与检测后处理

输出侧通过 `CVI_NN_TensorPtr()` 直接建立 FP32 只读切片，按 YOLOv8 的 `[1, 5, 8400, 1]` 通道优先布局 读取坐标和置信度。低于 0.5 阈值的候选在构造检测对象之前即被丢弃，剩余少量候选再执行 NMS 和 letterbox 坐标逆映射，从而避免输出张量复制和大规模无效排序。最终后处理平均为 1.632 ms。经过上述数据面和控制面 改造，正式 640×640 模型的单帧视觉总时延降至 53.024 ms，平均置信度为 93.13%；同硬件 Linux 基线为 54.230 ms，表明 StarryOS 已在完整视觉链路上达到与 Linux 相当的水平。

3.3　机器人用户态程序优化

3.3.1　基于视觉尺度反馈的二次曲线渐进接近控制

早期机器人在发现网球后主要采用固定速度或近似线性分段速度向前行驶。当网球距离较近时，视觉采集、模型推理、运动指令下发和底盘制动之间存在响应延迟，机器人可能在接近目标后继续高速前冲，导致网球被撞离工作区域。为降低接近阶段的冲击，系统使用网球检测框面积比例估计目标远近，并通过二次函数连续调节前进速度。

设摄像头图像尺寸为 $ WH $，网球检测框的左上角和右下角分别为 (x_1, y_1) 和 (x_2, y_2)，则目标面积比例定义为：

$$A = \frac{(x_2 - x_1)(y_2 - y_1)}{WH}$$

在摄像头安装位置和网球尺寸固定的条件下，网球越靠近机器人，检测框面积比例 A 通常越大。虽然该值不是经过标定的真实物理距离，但能够作为接近控制中的视觉距离反馈。

设最大追踪速度为 $v_{\max}$，抓取面积阈值为 A_g，则二次减速模型为：

$$v_q(A) = v_{\max}\left(1 - \frac{A}{A_g}\right)^2, \qquad 0 \le A < A_g$$

当 $A \ge A_g$ 时，机器人停止前进并进入抓取确认：

$$v_q(A) = 0, \qquad A \ge A_g$$

早期算法对比采用：

$$v_{\max} = 45, \qquad A_g = 0.55$$

在该组参数下，原线性或分段控制策略与二次曲线控制的工程标定结果如下。表中的二次速度用于展示曲线的减速趋势，数值包含整数化处理。

网球面积比例 A	线性/分段策略（旧）	二次曲线策略（新）	运动状态
0.03	45，全速	40	略有减速
0.10	45，全速	30	明显变慢
0.20	35	17	中距离减速
0.30	18	8	接近目标，低速缓行
0.40	8	3	接近停止状态
0.55	0	0	停车并进入抓取

从表中可以看出，旧策略在网球面积比例由 0.03 增加到 0.10 时仍保持 45 的最高速度，机器人在已经接近目标后才开始明显降速。二次曲线则从网球刚被发现时就逐步降低速度，并在中近距离阶段快速进入低速状态，从而提前抑制底盘动量。

若令归一化接近程度为：

$$t = \frac{A}{A_g}, \qquad 0 \le t < 1$$

则二次速度函数可写为：

$$v_q(t) = v_{\max}(1 - t)^2$$

在 $ 0<t<1 $ 范围内：

$$(1 - t)^2 < 1 - t$$

因此，在最大速度和抓取阈值相同的情况下，二次函数的输出始终低于对应的线性函数。这意味着二次控制会比线性控制更早、更明显地降低车辆速度，尤其适合视觉反馈存在一定周期延迟的机器人接近任务。

考虑到底盘电机存在静摩擦和低速死区，正式程序还增加了最低有效速度 $v_{\min}$。最终控制函数为：

$$v(A)=\begin{cases}\max\left[v_{\min},\operatorname{round}\left(v_{\max}\left(1-\frac{A}{A_g}\right)^2\right)\right], & 0\le A<A_g\\ 0, & A\ge A_g\end{cases}$$

当前进一步限速后的参数为：

$$v_{\max}=27,\qquad v_{\min}=6,\qquad A_g=0.55$$

其中，最大追踪速度由 45 降低至 27，降低幅度为：

$$\frac{45-27}{45}\times 100\%=40\%$$

因此，前述对比表反映的是引入二次曲线时相对于旧控制策略的减速效果；当前程序又在此基础上将最大速度进一步降低 40%。同时，程序将实际输出速度限制为不低于 6，所以表中理论值为 3 的近距离阶段，在当前底盘上会执行速度 6，避免车辆因低于电机死区而停滞。只有面积达到抓取阈值 $A_g=0.55$ 时，速度才直接变为 0。

纵向接近速度只在网球位于允许的横向中心区域时生效。设网球检测框水平中心为 c_x，夹爪标定目标中心为 c_{target}，中心容差为 Δc，则完整控制决策为：

$$u(A,c_x)=\begin{cases}\text{TurnLeft}, & c_x<c_{\text{target}}-\Delta c\\ \text{TurnRight}, & c_x>c_{\text{target}}+\Delta c\\ \text{Forward}\ (\ v(A)), & \left|c_x-c_{\text{target}}\right|\le\Delta c\end{cases}$$

由此，机器人将横向对准与纵向接近分离：横向使用中心死区和短时差速脉冲完成纠偏，纵向使用检测框面积反馈和二次曲线控制速度。当面积达到 0.55、网球中心进入近球抓取窗口且连续两帧满足条件后，底盘停止并启动机械臂抓取序列。

该优化使机器人形成“远距离较快接近—中距离主动减速—近距离低速缓行—达到阈值停车抓取”的渐进式运动过程，减少了固定高速接近造成的制动过晚、近距离过冲和撞球现象。

3.3.2　可识别红色容器

当前 YOLOv8 模型只负责识别网球，并未训练红色容器类别。考虑到测试环境相对干净，且场地中只有一个显著红色接球容器，系统在用户态增加了基于 YUV 颜色阈值的轻量级红色目标检测算法，从而避免重新采集数据、标注并训练双类别神经网络。

红色检测直接作用于 JPU 输出的 640×480 YUV422P 图像，不需要先转换为 RGB。设像素位置为 (x,y)，其亮度和色度分量分别为 $Y(x,y)$、$U(x,y)$ 和 $V(x,y)$，则红色像素判定函数为：

$$R(x,y)=\begin{cases}1, Y(x,y)\ge 24\wedge U(x,y)\le 115\wedge V(x,y)\ge 155\\ 0, \text{其他情况}\end{cases}$$

其中，亮度下限用于排除暗部噪声，较低的 Cb 分量和较高的 Cr 分量用于描述红色区域。程序遍历图像后统计红色像素数量：

$$N_R = \sum_{x=0}^{W-1} \sum_{y=0}^{H-1} R(x,y)$$

当 $N_R < 200$ 时，当前帧被认为没有有效红色目标；当红色像素数量达到最低要求后，程序进一步计算红色像素质心：

$$c_x = \frac{1}{N_R} \sum R(x,y)x, \qquad c_y = \frac{1}{N_R} \sum R(x,y)y$$

同时计算红色区域外接矩形 $(x_{\min}, y_{\min}, x_{\max}, y_{\max})$。由此得到红色像素覆盖率、外接框宽度覆盖率和高度覆盖率：

$$\rho_R = \frac{N_R}{WH}$$

$$\rho_W = \frac{x_{\max} - x_{\min} + 1}{W}, \qquad \rho_H = \frac{y_{\max} - y_{\min} + 1}{H}$$

其中，ρ_R 用于描述图像中实际红色像素的占比，ρ_W 和 ρ_H 用于描述红色容器在水平方向和垂直方向对视野的覆盖程度。

红色容器只有在机器人已经完成抓球、`holding_ball=true` 时才会成为有效运动目标。空夹状态下，即使摄像头观察到大面积红色区域，用户态状态机也会将其映射为 `Ignore`，防止机器人在尚未抓到网球时错误驶向接球容器。

持球状态下，机器人根据红色区域的水平质心控制方向。当目标位于中心死区左侧时执行左转，位于右侧时执行右转，进入中心区域后向前抵近。由于红色容器在近距离时包围框较宽，程序使用自适应中心死区：

$$\Delta c_R = \min\left(\max\left(70, \frac{w_R}{4}\right), \frac{W}{3} \right)$$

其中，w_R 为红色外接框宽度。该方法能够避免宽大的红色目标已经覆盖画面中心时，仅因质心小幅波动而反复左右转向。转向脉冲限制在 25～60 ms，进一步减少过冲。

机器人向红色容器前进时，根据外接框面积比例采用三档速度：

红色外接框面积比例	前进速度	控制目的
小于 0.45	12	快速接近远距离容器
0.45～0.75	9	中距离减速
大于或等于 0.75	6	近距离低速抵近

最终停车条件为：

$$\text{Stop} = (\rho_R \geq 0.98) \vee (\rho_R \geq 0.55 \wedge \rho_W \geq 1.00 \wedge \rho_H \geq 0.98)$$

即红色像素几乎覆盖全画面，或者红色区域具有足够像素密度且外接框达到画面宽度 100%、高度 98%。停车条件必须连续三帧成立，系统才由 `ApproachRedContainer` 切换到 `ReleaseTennis`，从而避免单帧颜色噪声或检测框抖动触发误放球。

在红车搜索与抵近阶段，用户态调用 `next_yuv()` 直接取得 YUV422P 图像，不执行 VPSS 颜色转换和 TPU 推理。这种按任务状态裁剪视觉链路的方式减少了不必要的硬件处理，同时不会影响网球检测阶段的识别能力。

3.3.3　摄像头完全遮挡后的脱困与视觉恢复

机器人抵近红色容器后，容器会覆盖接近 100% 的摄像头视野。此时摄像头无法观察远处网球，刚放下的网球和红色容器还可能持续占据画面。如果程序立即恢复普通追球逻辑，可能出现三种异常行为：将刚放下的球重新识别为新目标、把近距离大目标判定为“过近”并反复后退，或者在看不到远处网球时高速旋转，导致下一目标从视野中快速掠过。

此时神经网络仍可正常工作，但机器人已经进入暂时不可观测状态。由于红色容器完全遮挡摄像头，系统无法仅依靠当前图像判断安全撤离方向。因此，用户态在放球后引入一段确定性的开环脱困动作，先恢复摄像头视野，再重新进入视觉闭环。

完整恢复过程如下：

```
确认车辆停止
    ↓
servo2张开至200°并回读确认放球
    ↓
servo2恢复至98°空夹位置
    ↓
左右轮以(-16,-16)直线后退500 ms
    ↓
制动并停止20 ms
    ↓
左右轮以(6,-6)原地顺时针旋转
    ↓
重新发现网球后恢复正常追踪控制
```

后退阶段左右轮速度相同，因此机器人沿原方向直线离开红色容器；后退结束后先执行制动和停止，再开始两侧轮速相反的原地旋转。将后退和转向划分为两个独立阶段，可以避免二者叠加成一边倒退一边旋转的弧线运动。

为了防止机器人重新识别刚刚放下的网球，状态机在放球后设置 20 个视觉帧的保护窗口。在该窗口内，即使检测器返回网球框，程序也暂时忽略检测结果，继续执行原地搜索：

$$N_{\text{guard}} = 20$$

保护窗口结束后，如果仍未检测到有效网球，机器人继续以速度 $(6, -6)$ 低速顺时针旋转。低速搜索状态由 `slow_search_after_deposit` 保存，不会在下一帧立即恢复到普通搜索速度 12。只有真正检测到新的网球后，该状态才被清除，并恢复正常的追球转向速度和二次曲线接近控制。

放球后阶段	左轮速度	右轮速度	持续条件
直线后退	−16	−16	500 ms
制动稳定	0	0	20 ms
低速顺时针搜索	6	−6	直到重新发现网球
正常追踪	动态控制	动态控制	目标有效时

机械臂动作执行期间，后台 UVC 任务仍在采集图像。用户态结束阻塞式机械臂动作后，不会继续处理动作开始前的历史帧，而是按照帧序号请求最新图像。因此，机器人完成后退和转向后能够基于较新的环境状态恢复视觉控制，避免旧帧积压进一步延迟重新定位。

这套机制将“完全遮挡时的开环脱困”和“视野恢复后的闭环追踪”结合起来，使机器人能够从红色容器近距离遮挡状态中稳定退出，并继续寻找下一颗网球。

3.3.4　多目标识别与基于视觉距离的抓取排序

当前 YOLOv8 模型属于单类别网球检测模型，但能够在同一帧中输出多个网球实例。需要区分“多类别识别”和“多实例识别”：模型并未同时学习网球和红色容器两个神经网络类别，而是能够同时检测画面中的多颗网球；红色容器由独立的 YUV 颜色算法识别。两种感知方式由用户态状态机统一调度。

YOLOv8 输出经过置信度阈值过滤和非极大值抑制后，可以得到网球检测集合：

$$\mathcal{D} = \{D_i = (B_i, s_i) \mid i = 1, 2, ..., n\}$$

其中，B_i 为第 i 个网球检测框，s_i 为对应置信度。默认置信度阈值为 0.5，非极大值抑制的交并比阈值为 0.5，用于删除同一网球产生的重叠重复框。

对于每个候选目标，程序计算检测框面积：

$$S_i = w_i h_i$$

并计算其相对整幅图像的面积比例：

$$A_i = \frac{w_i h_i}{W H}$$

在网球实际尺寸近似一致、摄像机参数固定的条件下，距离摄像头较近的网球通常具有更大的检测框。因此，系统采用最大面积优先策略选择当前控制目标：

$$i^* = \arg\max_i S_i$$

$$D^* = D_{i^*}$$

该策略利用单目图像中的目标尺度近似目标距离，不属于严格的三维测距。与单纯选择最高置信度目标相比，最大面积策略更符合抓取任务需求：置信度最高的球不一定距离最近，而优先抓取近处网球通常具有更短的移动距离、更大的图像特征和更稳定的抓取条件。

目标选择流程如下：

```
一帧中检出多个网球
        ↓
置信度阈值过滤
        ↓
NMS消除重复检测框
        ↓
计算每个检测框面积
        ↓
选择面积最大的网球
        ↓
对该目标执行转向、减速和抓取
```

系统在任意时刻只追踪一个主目标，以避免多个网球同时影响转向决策。选定目标后，其水平中心用于横向对准，面积比例用于二次曲线减速和抓取距离判断。机器人完成抓取后切换到红色容器状态，不再处理其他网球；放球并退出红色容器遮挡区域后，系统重新对当前画面中的全部网球执行检测和最大面积选择，从而开始下一轮任务。

完整多目标任务状态可表示为：

多球检测 → 选择最近目标 → 追踪与抓取 → 寻找红色容器 → 放球 → 重新检测剩余网球

用户态还提供 `--max-deposits N` 参数，用于指定需要完成的抓取和放回次数。每次经过机械臂位置回读确认的放球动作完成后，计数器增加 1；当完成数量达到 N 时，程序停止底盘并安全退出。该功能使机器人能够连续完成三球或更多目标的顺序抓取，并为多目标任务提供明确的终止条件。

需要指出，最大检测框面积只能作为距离的近似指标。当网球被遮挡、位于画面边缘或检测框尺寸存在明显误差时，面积排序可能不完全等价于真实距离排序。若后续需要更严格的最近目标选择，可进一步引入摄像头标定、目标直径模型、单目测距或深度传感器。当前方案计算成本低，不需要额外硬件，已经能够满足固定场地中的多网球顺序抓取需求。

视觉链路的工程实现集中在用户态与驱动边界：把同步取帧改造成有界的最新帧流水线，再把 相机、硬件解码、几何变换与神经网络推理串成一次硬件接力，机器人用户态只负责策略与 状态机。这些实现都不经系统层的内核路径，系统层三条线因此各自独立成节。

3.4　内核优化的工程实现

本节按机制族（K1–K8）说明每处改动**改了什么、怎么改**；每族对应的实测数据放在 4.5 节，定位过程与被否掉的替代方案放在 5.3 节。这样安排是为了让 “改法”与“证据”各归其位：本节每一条都能在前文的机制族清单里找到对应的提交与验证。

3.4.1　K1 系统调用返回路径：把指令缓存刷新收敛到少数事件上

问题出在返回用户态的路径上：内核在**每一次**返回用户态之前都执行一次指令缓存失效（riscv64 的 `fence.i`）。它在语义上只在“用户映像的代码字节可能陈旧”时才需要，而内核 本来就已经在两处正确的位置刷：把页变成可执行时（`PageObject::prepare_executable_mapping` 覆盖了全部 11 条装页表的路径）与 ptrace 改写正文后（`AddrSpace::sync_modified_text`）。

改法是把默认行为反转为“不刷”，只在四类事件上刷一次：`execve` 装载新映像、`mmap` 或 `mprotect` 把一段内存变成可执行、切换到另一个地址空间，以及显式写 `/proc/icache_flush`。最后一项同时是运行期回退旋钮，写 1 即恢复旧行为。因为这条动的是对外语义，验证上做了 三层：宿主单测、板端全项 harness、以及 78 轮多二进制 `exec` 压力（反复装载不同的可执行 文件并校验每次输出）。

3.4.2　K2 地址空间操作的机械开销：三处只删冗余的改动

`fork` 与子进程析构两栏的时间与内存带宽无关，其构成为同一类重复：本该一次做完的事 被逐项做了，本该 O(n log n) 的事写成了 $O(n^2)$。三处改动属于同一族，均只删冗余、 不改语义。

取消每个叶子的第二次页表走查：`occupied_leaf_records()` 本来已经拿到页表遍历时的 原始 PTE，却对每个叶子再调一次 `pt.query()` 复核；改成复用遍历结果后，单个叶子的克隆 成本从 99 µs 降到 82 µs。

所有者发布去重由 $O(n^2)$ 改为 O(n log n)：`publish_prepared_pte_owners()` 用线性 `Vec::contains` 检测重复键，而批量填充一次要发布几百个所有者；换成 `BTreeSet` 后这段从 120 µs 降到 99 µs。两处合计让 `try_clone` 从 1 747 µs 降到 1 528 µs、`fork()` 系统 调用从 4 739 µs 降到 4 418 µs。

地址空间退休不再逐页摘除：子进程退出时要退休整个地址空间，原实现逐页 `unmap`，每次 都要走查页表、再走查一次页表、更新地址空间账目；改成整段交给批量路径后，子地址空间 退休从 3.57 ms 降到 0.41 ms（8.7×）。这 3.2 ms 不在 `fork_exit` 的墙钟里（它由内存 回收线程异步完成），但直接减少了每次 fork+exit 的 CPU 占用。

3.4.3　K3 exec 的页物化：先减少要铺的页，再把剩下的摊到一次操作上

`execve` 的内核侧成本优化前是 32.3 ms，分段后 88% 落在一处：static-PIE 把每个 `PT_LOAD` 段整段物化。往物化里挖又得到三条否证——文件后端的分段计数全程为 0（说明段映射走的是 COW 而不是共享文件映射）、一次 51 轮 exec 压力前后的块层读数增量为零（说明 exec 完全不读 SD 卡）——因此每页的成本是“一次帧分配 + 三次 4 KiB 拷贝 + 页对象与索引 + 装页表”。

据此做了三刀：第一，**只物化带写权限的段**。重定位只写 `.got` / `.data.rel.ro` / `.bss`， 只读段完全可以按需缺页，于是只整段物化 `PT_LOAD` 里带 `WRITE` 的那些。第二，**按连续 run 批量处理**：物化时把“逐页一次 `io_lock`”改成整段一次 `pin_pages_run`、整段一次域锁预留， 并把每段一次的全表清理收成一次。第三，**单页缺页不再每次跑 O(表长) 的清理**：改成“命中 陈旧条目才清 + 每 512 次兜底”。

结果是 `exec_load` 从 31.7 ms 降到 4.5 ms（其中 `exec_populate` 从 28.3 ms 降到 1.2 ms），exec 内部的缺页数从约 200 页/次降到 79 页/次， `fork_exec_exit` 从 43 537 µs 降到 22359 µs。值得注意的是这一族的两条改动方向相反：第一条**减少要铺的页**，第二条**把剩下的页摊到一次批量操作上**，两者叠加才拿到全部收益。

3.4.4　K4 缺页的每页固定成本：页索引与内存原语

缺页分段显示 87.7% 的时间在 `prepare`， 而 `prepare` 里 58% 在 `pending_insert`——COW 页索引的插入。原因是该索引既在每次插入时扫描整个索引统计存活条目（每项一次原子读）， 又用有

序 `Vec` 做插入，两者叠加成 $O(n^2)$，成本随区域线性膨胀。改法是纯数据结构层面的：有富余容量时直接插入、跳过存活计数；已过期的弱引用条目改在容量增长路径上摊还清理；重叠检查只对存活条目生效。

同一族里还有一处与数据结构无关的改动：内核导出的 `memcpy`/`memmove`/`memset`/`memcmp` 全是逐字节 `for` 循环，而内核里所有清零与拷贝都链接到它们。改成 8 字节对齐 + `u64` 批量 + 尾部补齐（`memcpy`/`memcmp` 只在源目对齐一致时走字长），顺带修掉一处真实的未定义行为：`memmove` 的反向路径在 `i == 0` 时先算 `ptr.add(i - 1)`，地址下溢本身是 UB，编译器据此做的优化会让用户进程偶发丢输出并以 `SIGSEGV` 退出。

3.4.5　K5 顺序缺页的窗口批量物化：四步

前两族之后，“每页一次事务”仍是主要成本。动手之前先量上界：同一块板上，逐页缺页是 144 700 ns/页，而一次 `mmap(MAP_POPULATE)` 走既有的整段物化原语只要 62 800 ns/页—— 真正贵的是每页一次事务，不是内存带宽。据此实现这一族，共四步。

第一步，**按 64 KiB 窗口整段物化**，判据刻意保守：只在严格升序（本次缺页页正好接在上一次 窗口末尾）时预取，跨步与随机访问完全不命中；只对私有匿名且策略叶子为 4 KiB 的 VMA 生效， 文件与共享后端语义不变，`MADV_HUGEPAGE` 的 VMA 也不会被提前铺上 4 KiB 页表。

第二步，**窗口大小做成运行期旋钮并加短流爬坡**。窗口大小是“按窗付的固定开销”与“预取深度”的权衡，因此没有拍常数，而是加了 `/proc/fault_around`（写页数，0 表示关闭）再扫描定值，取 64 页（256 KiB）。固定大窗口的代价是“只顺序碰两页的程序也要付整窗内存”，因此又加了 爬坡：连续命中第 n 次才用 8/16/32/64 页。

第三步，**窗内帧整块分配 + 子租约**：连续未映射的匿名段先按“块大小阶梯”64→32→16→8→4→2 页尝试一次分配一整块，再用 `FrameLease::sublease` 把它切成 4 KiB 子租约，每个页对象拿 一个；拿不到连续块就逐页退回。整块命中率 99.97%，索引插入落在末尾的比例从 21% 升到 56%， 平均每次插入搬移的条目数从 102–115 降到 24。

第四步，**窗内索引按 run 批量登记**：既然一窗的物理帧连续，就把整窗的插入合并成“一次定位

1. 一次整段搬移“。这一步值得单独记一笔，因为第一版实现曾让板子在启动时硬挂（详见

5.3 节）；重做后的设计是三条：整段插入全有或全无、遇到空洞或区间内墓碑就退回逐页 登记、容量按整段预留以免在 IRQ 关闭的临界区里分配内存。

3.4.6　K6 文件读路径：一次多余的拷贝、一次逐页的事务、两处冗余失效

读路径的四条改动分别去掉四件冗余。

去掉暂存中转：缓存命中时原实现要把页内容先拷进一个暂存页，出了页缓存索引锁再拷给调用方；中间那次拷贝只是为了“拷给用户前必须放锁”。改成 pin 住页缓存页 → 出锁 → 用该帧 的内核直接映射直接拷给调用方 → 放下钉子。

用户内存准备整段化：riscv64 上没有可用的硬件用户访问探测指令，内核此前每次用户访问 都要逐页跑完整的缺页事务；改成“整段已映射”快路径，再进一步把用户内存准备改成整段物化 一次（内部按 VMA 分段并收进一次变更事务，任何一步不成立就回退到逐页循环）。

延迟失效路径不再逐页刷 TLB：四个后端的 `unmap_range` 都调逐页摘除，而它内部会每一页做一次本地 TLB 失效；但调用方已经进入延迟失效守卫并对整段登记过一次失效、提交时统一 发布，逐页那次是纯冗余。

修复复用帧被旧墓碑遮蔽的身份查询：页索引里已释放帧会留下过期弱引用条目，而重分配的帧 会插到同址墓碑之前，于是查询命中死墓碑、返回“无此身份”，明明有活主人却让 fork/COW 走错 分支。改成自该位置向前扫过所有仍包含该物理地址的条目、返回第一个活主人。

3.4.7　K7 存储写回与元数据：批量粒度、页缓存容量与访问时间

三处改动作用在同一条写回路径上：一处改每次写多大的块，一处改能攒多少脏页，一处去掉 `close` 里的同步提交。

脏页按连续 run 批量写回：写回快照拿到的本来就是连续的脏页，但回写实现仍然每页一次后端写——8 MiB 就是 2 048 次 4 KiB 写。改成把连续脏页合并成一次写，并把单次回写的批量上限从 16 页提到 64 页（256 KiB，回写期间的额外内存仍是常数级）。

页缓存上限 512 → 4096 页：8 MiB 档顺序写吞吐只有 2.41 MiB/s 的原因是页缓存上限只有 2 MiB，写满就要立刻回收、于是本地同步回写；把上限提到 16 MiB 后，写只落页缓存。

`close` 不再做同步元数据提交：文件对象析构时若句柄被读过（0 字节 `read()` 也算）就会 更新访问时间，而 ext4 的元数据更新末尾直接做一次同步日志提交——在这块 SD 卡上约 10 ms。改成按 Linux 语义处理：`noatime` 不写、`strictatime` 照旧每次写，其余按默认的 `relatime` （每个文件每 24 小时至多写一次）。判定必须用**单调时钟**而不是墙上时钟：本板没有 RTC 与 NTP，`date` 显示 1970 年，用墙上时钟套 `relatime` 判据会永远判成“已过期”、退化成每次都写。

3.4.8　K8 启动等待：固定阻塞改成带截止时间的轮询

优化前，网络初始化里有一段固定 2 s 的阻塞等待（等设备就绪），无论设备是否已经就绪都要 等满。改成带截止时间的轮询（上限 300 ms，一旦就绪立即继续）之后，上电到用户态标记从 10.81 s 降到 9.61 s、到 shell 可交互从 12.3 s 降到 10.8 s。启动路径上其余几块静默时间（USB 探测、无线固件加载、ext4 挂载）都牵动设备初始化的顺序与超时语义，本轮没有动，原因写在 5.3 节。

内核侧的八族改动作用在通用子系统的固定开销上，与实时性无关；需要另一颗核承担的 职责因此单独成一条线。

3.5　跨核通道：从寄存器到设备节点

小核要能被使用，先要有一条可靠、可编程的通道。本队把这条通道实现为四层：驱动层负责 寄存器与映射，锁与唤醒层负责中断上下文与任务上下文之间的交接，共享内存层负责大块状态 的传递，用户态接口层负责把能力暴露成设备节点。

第一层是**驱动**。驱动接管 `ap_mailbox` 的 8 个槽位与对应的中断位：发送时把 `cmdqu` 信封写进空闲槽并置位门铃；中断到来时只把槽位搬进接收队列，不做任何可能睡眠的操作。这套寄存器语义在协议头文件里是公开的，因此驱动的难点不在协议解析，而在映射与上下文 约束（两处踩坑见 5.7 节）。

第二层是**锁与唤醒**。中断上下文只能用 IRQ-save 自旋锁，持锁期间不能调用分配器，因此接收队列是固定容量的环形数组；等待路径按“硬中断 → 固定服务线程 → 等待队列”三段式 实现——硬中断只把槽位搬进队列并唤醒服务线程，服务线程在任务上下文里扇出等待队列，等待者自身还保留一条“超时前复查硬件队列”的自愈路径。这样既满足了中断上下文不能睡眠 的约束，也避免了在等待路径上做内存分配。

第三层是**共享内存**。mailbox 只传 8 字节信封，状态快照、事件环与命令回流走一块 1 MiB 的保留区（`0x8B200000`，放在 ION 窗口正下方，严格不与厂商给小核的 carve-out 重叠）， 由设备树的 `reserved-memory` 声明，驱动按 `cvitek,cmdqu-shm` 探测并强制非缓存映射， 用户态经 `mmap` 直接读写。布局当前为 v3：头部 192 字节按**写入方**拆成三条独立 cache line（常量 / 大核写 / 小核写），同步块拆成大小核两半，后接两条 64 槽 × 64 字节的环 （Ring A 大核到小核、Ring B 小核到大核），消息为 16 字节元信息加 48 字节载荷并带 CRC32。把头部按写入方拆行由一次实测教训直接促成（见 5.7 节 关于“度量假象”的记述）。

第四层是**用户态接口**。驱动注册 `/dev/cvi-rtos-cmdqu`（字符设备 10:58），提供发送（即发即弃）、接收、等待**指定**应答命令号的 `EXCHANGE`、`poll`/`epoll` 就绪语义与 调试体检接口；等待的应答命令号由调用方显式给出（厂商驱动把等待命令号写死成发送命令号，因此像 0x51 等 0x52 这样的非对称应答只能静默超时）。超时只通过 `errno` 返回（`ETIMEDOUT`），结构体不产生无定义的输出。

3.6　小核接管底盘链路：三阶段落地

拾球小车的真实链路是“SG2002 经 UART1 连底盘控制板”：底盘在 A18/A19 焊盘上（厂商 Linux 的 `/dev/ttyS1`，115200），电机闭环、驱动与编码器都在底盘板一侧，主控只下发目标速度； 机械臂舵机走 UART2。链路归属是用“协议指纹加双向拔插消元”在实车上定案的，不是靠读设备树 猜出来的。小核接管这条链路分三步，每一步都有独立的验收判据，实测结果见 4.7 节。

第一步是**独立掌握链路**：固件新增 UART1 轮询驱动与握手任务，默认关闭，由命令 `0x61` 打开后主动与底盘板做 `INIT` → `ACK` 握手，统计经共享环的 kind=6 消息上报。验收的判据是“拔掉排线时 `ack_rx` 冻结、插回后自动恢复”，它证明收与发两个方向都由小核独立完成； 同时在裸板上（没有底盘）会得到 `INIT` 持续增长而 `ACK` 恒为 0 的负向基线。两半配对， 排除了“软件自说自话”。

第二步是**协议层接管**：以 `mode=2` 走完一遍完整指令集，顺序是 `CONFIG(ppr=4680)` → `SET_SPEED(0,0)` → `SET_SPEED(1,0)` → `STOP(0)` → `STOP(1)`，每条命令等它的 ACK， 结果经 kind=7 上报。这一步的判据是“全程不含任何速度指令，车不会动”——协议自检必须能在 不驱动机构的前提下完成。

第三步是**周期服务与心跳看门狗**：命令 `0x62` 下发目标速度即进入 `mode=3`，小核默认每 100 ms 发一次左右轮 `SET_SPEED`，每 5 个控制周期回读一次 `GET_STATUS`（状态加双轮 RPM），结果经 kind=8 上报；控制周期可用 `0x63` 在 5–1000 ms 之间调整（按 5 ms 基础 节拍量化）。在同一层落地**心跳看门狗**：大核超过 500 ms（`0x64` 可调 100–10000 ms） 没有刷新目标，小核立即 `STOP` 两轮并保持静默，直到新目标到来。在此之上还有一层 **堵转与碰撞的软件保护**（kind=10）：目标 PWM 越过死区、两轮 $|\mathrm{RPM}| \le 5$、连续 3 次 命中（起步宽限 700 ms）即停车并锁存，只有目标值发生变化才解除。这条保护不需要任何 新增硬件。

三阶段与两层保护共同构成小核侧的职责边界：它拥有链路、拥有节拍、也拥有“停下来”这个 动作；大核只负责下发目标与读取状态。

小核侧的实现分两步：先把核间通道做成可编程的设备节点，再让固件接管底盘串口这条 真实链路。上述三条线都依赖同一件前提——板子可被反复刷写与被测，这也是最后一条线的 落点。

3.7　板级 I/O 与上板工作流

tty/pty 子系统提供字符设备与串口之间的行规程；USB 主机栈（DWC2）负责设备枚举与 传输；`cv181x-sdhci` 提供 SD 控制器访问；ax-net 提供协议栈与套接字；跨核通信由 `ap_mailbox`（基址 `0x01900000`，4 KiB，8 字节 `cmdqu` 槽位环）承载；SD1 上的 AIC8800 提供无线链路。本队在此骨架上补齐了 SG2002 的板级驱动与工作流工具，并修复了 若干时序、锁与引脚复用缺陷。按功能划分，本模块包含四组内容：单路控制台下的可编程 工作流、以 U-Boot UMS 为中心的镜像通路、以小核为中介的暖复位与双系统切换、 以及从网卡上电到板上 SSH 的使能链。执行器通路（PWM）单独分析，见后文。

3.7.1　控制台与串口：单路 UART 下的可编程工作流

本项目的控制台策略是“只保留一路、把工具做厚”，而不是增加串口——可以用的两路备用 UART 分别压在 JTAG 与 SD1 焊盘上，占用的代价见 5.8 节。围绕这一路 UART， 本节构建三件工具，其设计判据均来自串口本身的特性。

第一件是**命令执行器**。串口同一时刻只允许一个进程持有，且内核日志与 shell 输出在 同一字节流上交错，因此命令完成不能以行数或固定延时常量判定，只能以“提示符出现在 接收缓冲区末尾”为判据；该工具还需处理残留进程占用端口、命令长度上限带来的截断等 问题。

第二件是**启动探针**。在 U-Boot 中执行 `fatload` 与 `bootm`，并对每一行输出打上主机侧 时间戳，从而把“内核就绪”“用户态首个输出”“shell 可交互”三个锚点分离出来，供启动 时延分析使用。

第三件是**运行期会话工具**。厂商 Linux 的 getty 只在开机后的短窗口内可用，该工具在 同一次抓取里完成“引导—抢登录窗口—执行命令”的连续动作，使两侧系统能在同一份静态 ELF、同一探针下取得可比数据。

3.7.2　镜像通路：以 U-Boot UMS 取代串口文件传输

镜像通路的设计直接由介质与带宽决定。控制台是 115200 8N1 且被 shell 占用，把约 16 MB 的 FIT 映像经由该通道传输既受带宽限制、又缺乏可靠的流控手段，因此方案排除串口传文件， 其依据同样是这一路带宽与流控条件。取而代之的是 U-Boot 的 USB 大容量存储（UMS）模式：在 `soph#` 提示符下执行 `ums 0 mmc 0`，SoC 的 USB 设备控制器把整张 SD 卡以 U 盘形式交给主机，随后宿主机以普通块设备语义读写该卡。

这一转换带来两个直接收益。其一是**路径分工清晰**：FAT 启动分区（`/dev/sda1`，本板为 256 MiB）承载 FIT 映像，用 mtools 的 `mcopy`/`mdel` 直接写入与回收；ext4 根分区 （`/dev/sda2`）承载探针与测试程序，用 `debugfs` 写入并顺带设置执行位。二者都不需要 主机提权、不需要板上具备网络，也不需要把文件先落到板上再由板上复制。其二是 **可自动化**：整个动作序列为“软复位

回到 U-Boot → 启动 UMS → 写入映像 → 中止 UMS → `fatload` 并 `bootm`”，全部可由宿主脚本驱动，因而可以在无人值守条件下反复执行。

实践中还积累了两条必须写明的边界（设备节点残留与空间统计滞后），见 5.8 节。

3.7.3　系统切换：以小核为中介的暖复位与双系统对照

SG2002 的复位是多层结构：上电复位、系统硬复位与软件可控的复位配置寄存器分属不同 层次，且它们只作用于状态单元，不改变存储器内容。本项目的切换思路没有沿用“内核 `reboot(2)` 直接触发固件复位”的路径，而是把小核当作板级复位的执行者：大核侧通过 `ap_mailbox` 上的 `cmdqu` 命令队列把复位请求交给 C906L 固件，由固件按 FSBL 的序列 操作复位控制器与安全子系统寄存器，完成整芯片软复位。

这一设计的原因有二：其一，大核侧没有物理内存直写通道，把寄存器序列放在内核里做成受控接口，比在用户态暴露裸物理内存更安全；其二，小核固件在复位过程中不随大核一起被 清空，因而复位前后它仍可保留必要的板级状态。

该原语带来的直接效果是：shell 中一条命令即可回到 U-Boot，物理按键与断电不再是流程的 一部分。其代价（软复位只重启 SoC、5 V 供电轨不掉电）写在 5.8 节，并成为 执行器侧兜底要求的依据。

在双系统对照方面，同一张卡上同时放置本项目的 FIT 与厂商 Linux 的 FIT，由 U-Boot 以 `fatload` + `bootm` 选择载入对象；Linux 侧则经由串口 getty 的登录窗口执行同一份静态 探针程序。由此，两侧的对比数据在同一块板、同一份可执行文件与同一套工具下取得，避免 了把根文件系统差异误读为内核差异。

3.7.4　执行器通路：SG2002 PWM 的编程模型

SG2002 提供四个 PWM 控制器（PWM0–PWM3），基址自 `0x03060000` 起、各占 4 KiB， 每个控制器带四路独立输出，共十六路。时基取自 100 MHz 与 148.5 MHz 二者之一（默认 前者），内部计数器为 30 位，最高输出约 50 MHz、最低约 0.09 Hz。每一路只由两个计数 寄存器描述波形：`PERIODn` 给出周期拍数，`HLPERIODn` 给出**低电平**拍数；占空比由两者相减得到， 硬件不提供独立的占空比寄存器。输出分为两种模式：`PWMMODE`＝0 为连续输出， `PWMMODE`＝1 则按 `PCOUNTn` 输出固定脉冲数，结束后 `PWMDONE` 置位且 `PULSECOUNTn` 可读回实际拍数。四路之间还可进入同步模式（`SHIFTMODE`＝1），以 `SHIFTCOUNTn` 为 每一路设定相位差。运行期修改参数需经 `PWMUPDATE` 动态装载；而重新使能一路时，必须令 `PWMSTART` 走一次 0 → 1，否则计数器与状态寄存器不会复位。这套模型与 Linux `/sys/class/pwm` 的四处落差单独列在 5.8 节，它正是不宜直接复用其他平台 PWM 驱动的原因。

本项目在这一阶段的定位是：把电机 PWM 交由底盘控制器产生，SG2002 侧只负责下发速度 目标（经 UART1 或经小核命令链路转发）。取舍依据在实测特性：开环 PWM 的死区在空载时为 13%–14%（对应指令量 19–21），带载时 抬升到 16%，转速与指令呈明显非线性，接口工作量不是决定因素。因此本模块的这部分工作以硬件模型分析与接口映射分析为主，而非以驱动合并为目标。

3.7.5　从网卡上电到板上 SSH

板上 SSH 的价值在于为单一串口之外增加一条登录通路，其成立依赖一条完整的使能链。 链的底端是无线网卡本身：AIC8800 挂在 SD1 上，与前述 UART3 争用焊盘，因此引脚复用 一旦写错，无线链路会先于网络协议栈失效。其上依次是固件下载、接口启用、WPA2 关联、 地址分配与 `sshd` 启动。

围绕这条链，本项目处理了若干与本板强相关的缺陷：构建期的固件抓取需要离线缓存， 否则无外网的机器无法出包；关联被固件拒绝或处于忙状态时不再把设备打死，早期启动 窗口内的连接尝试因此可用；信道表原先只覆盖 2.4 GHz，补全后才连得上 5 GHz 热点； WPA 握手需要熵，用芯片 TRNG 补齐启动熵之后，关联不再因熵不可用而失败。

使能链最隐蔽的一处故障在并发而不在协议：压测显示无线链路在累计传输约 9 MiB 后会 静默失效，接口仍为 `RUNNING`、地址仍在，但连通性全无。定位与修复见 5.8 节； 修复后同样的压测在 60 MiB、60 轮下零错误，与厂商 Linux 的表现对齐。

需要指出的是，交付阶段网络并不能被当作可依赖的前提。现场热点为移动热点，主机随时可能离场，因此本项目把“串口可独立完成全部操作”定为硬约束：镜像经 UMS 写入、探针经串口启动执行、板卡经小核原语软复位。网络与 SSH 在此定位为加速项而非必要项。

3.8　软件界面 （可选）

3.8.1　X 主要模块界面

第 4 章　实验测试

4.1　测试说明

本项目所有对外结论都建立在同一组测试条件上，因此口径先说清楚：**同一块板、同一份符合 Linux ABI 的静态可执行文件**，在 StarryOS 与厂商 Linux 两条基线上各测一次；性能类指标另做“优化前/优化后”的同板对照，并尽量在**同一次上电、同一份探针、交替顺序**下重复测量，以抵消本板上“启动后第一次大分配明显更快”这类时间漂移。凡两侧口径不能对齐的指标（例如 启动时间里的脚本段），一律拆开分别报告；Linux 侧缺测的指标标为“未测”，不拿别处的数字 补位。探针都是自研 C 程序，两侧编译同一份源码；内核侧另有 `/proc/fault_attrib` 作为 分段计数的出口。各条工作线的探针清单、验收门与实测结果见下文对应小节。

4.2　测试结果

总体上，本项目在三类指标上分别取得了不同的位置，读表时应当分开看：**固定成本类**（系统调用入口、进程创建与 exec、缺页、文件读写）从落后一到两个数量级收敛到同一量级， 其中缺页与存储写回已基本追平或反超；**带宽与总量类**（内存带宽、冷启动、连续写吞吐） 本来就领先或持平，优化后仍然领先；**确实还没追平**的只有上下文切换与文件读路径的每页 固定开销两处，两者的归因与后续方案见 第 5 章。以下按应用层、系统层内核、 实时小核、板级 I/O 四条线分别给出结果。

以下测试用于验证视觉推理工作线的阶段时延与系统级收益。系统调用、内存、存储、网络等实验结果由本章其他小节按统一测试原则继续补充。

4.3　StarryOS 与 Linux 视觉性能对比

在使用正式 640×640 网球模型完成三球任务的实测中，StarryOS 的单帧摄像头等待、JPU 解码和 VPSS 处理合计为 6.087 ms，TPU 输入 TDMA 搬运为 1.116 ms，TPU 前向推理为 41.418 ms，检测后处理为 1.629 ms，其他视觉开销为 0.189 ms，单帧视觉总时延为 50.441 ms，平均检测置信度为 90.83%。与早期依赖 CPU 完成图像转换和缩放的处理方式相比，当前链路将 JPEG 解码、图像预处理和神经网络计算分别交给 JPU、VPSS 和 TPU，并通过 ION 物理缓冲区在硬件模块之间共享图像数据，显著减少了大图复制、CPU 计算和调度等待开销。

4.3.1　测试结果概述

本次测试在同一 SG2002 硬件平台上，采用相同的 640×640 网球检测模型和基本一致的视觉处理流程，对 StarryOS 与 Linux 基线进行了单帧平均性能对比。两套系统的主要处理链路均可概括为：摄像头采集 MJPEG 图像、JPU 硬件解码、VPSS 硬件预处理、TPU 输入搬运、TPU 前向推理以及 YOLO 检测后处理。

表 StarryOS 和 Linux 基线在相同测试条件下单帧性能对比表

指标	StarryOS	Linux 基线	差值（StarryOS − Linux）	对比 cha ru 结果
摄像头等待	5.489 ms	0.051 ms	+5.438 ms	Linux 更低
JPU 解码	1.645 ms	11.033 ms	−9.388 ms	StarryOS 更低
VPSS	1.643 ms	1.865 ms	−0.222 ms	StarryOS 更低
摄像头/JPU 等待 + VPSS	8.776 ms	12.949 ms	−4.173 ms	StarryOS 更低
TPU 输入 TDMA 搬运	1.114 ms	0.971 ms	+0.143 ms	Linux 更低
TPU 推理	41.309 ms	39.721 ms	+1.588 ms	Linux 更低
后处理	1.632 ms	0.449 ms	+1.183 ms	Linux 更低
其他视觉开销	0.194 ms	0.140 ms	+0.054 ms	Linux 更低
视觉总时延	**53.024 ms**	**54.230 ms**	**-1.206 ms**	**StarryOS 低约 2.22%**
单帧平均置信度	**93.13%**	**93.62%**	**-0.49 个百分点**	**基本接近**

从总量上看, StarryOS 的单帧视觉总时延为 **53.024 ms,** 比 Linux 基线的 54.230 ms 减少 1.206 ms, 按 Linux 基线计算低 2.22%。这个结果表明, 经过摄像头、JPU、VPSS 和 TPU 驱动优化后, StarryOS 的成功视觉帧处理性能已经与 Linux 处于同一水平，并在本轮测试中略有领先。分阶段结果仍有快慢差异：StarryOS 在图像解码和预处理阶段取得的优势，抵消了其在 TPU 推理、后处理等阶段的额外开销。

4.3.2　各项指标的含义以及计算方式

视觉总时延的计算公式如下:

```
视觉总时延
  = 摄像头等待
  + JPU 解码
  + VPSS
  + TPU 输入 TDMA
  + TPU 推理
  + 后处理
  + 其他视觉开销
```

系统采用分段插桩方式测量单帧视觉链路。用户态使用单调时钟，在摄像头、VPSS、TPU 运行时和后处理函数调用前后记录时间；StarryOS 内核在 JPU 解码、VPSS 硬件启动与完成中断、TPU 命令触发与中断恢复位置补充纳秒级时间戳。每个成功完成的网球视觉帧生成一组阶段数据，测试结束后计算单帧平均值:

$$\overline{T} = \frac{1}{N}\sum_{i=1}^{N} T_i$$

总体探针布局如下:

```
t0：开始处理一帧
  |
  ├─ t1：调用摄像头/JPU ioctl
  |    ├─ k1：内核开始JPU解码
  |    └─ k2：JPU解码完成
  ├─ t2：摄像头/JPU ioctl返回
  |
  ├─ t3：提交VPSS ioctl
  |    ├─ v1：进入VPSS内核接口
  |    ├─ v2：启动VPSS硬件
  |    └─ v3：VPSS完成中断
  ├─ t4：VPSS ioctl返回
  |
  ├─ t5：开始导入TPU输入张量
  ├─ t6：输入TDMA完成
  |
  ├─ t7：调用CVI_NN_Forward()
  ├─ t8：CVI_NN_Forward()返回
  |
  ├─ t9：开始解析输出张量
  ├─ t10：NMS及坐标恢复完成
  |
  └─ t11：本帧视觉处理结束
```

指标	指标含义	计算方式	计时范围	探针位置
摄像头等待	视觉线程请求下一幅可用图像后，等待压缩帧就绪并取得该帧的时间	$T_{\text{camera}} = (t_2 - t_1) - (k_2 - k_1)$	等待新帧、取得最新帧及摄像头接口附加开销	用户态在 `t1`、`t2` 记录摄像头/JPU ioctl 墙钟，内核在 `k1`、`k2` 记录 JPU 解码时间，两者相减得到摄像头等待残差
JPU 解码	将 MJPEG 压缩帧提交给 JPEG 处理单元并取得 YUV422P 解码结果的时间	$T_{\text{JPU}} = k_2 - k_1$	JPU 提交、硬件解码、完成等待及状态检查	StarryOS 内核 `decode_jpeg_into_ion()` 中，`k1` 位于 `decode_planar_into()` 调用前，`k2` 位于解码完成及输出状态检查后
VPSS	完成 YUV422P 到 RGB 的颜色空间转换、缩放、letterbox 填充及 RGB planar 排布的时间	$T_{\text{VPSS}} = t_4 - t_3$；其中 $T_{\text{submit}} = v_2 - v_1$，$T_{\text{hardware}} = v_3 - v_2$，$T_{\text{return}} = (t_4 - t_3) - (v_3 - v_1)$	请求校验与硬件启动、硬件处理、中断唤醒及 ioctl 返回	用户态在 `t3`、`t4` 记录 ioctl 墙钟；内核在 `v1` 记录请求入口、`v2` 记录硬件启动、`v3` 记录完成 IRQ
摄像头/JPU 等待 + VPSS	从请求一帧图像到获得可输入 TPU 的 RGB planar 图像为止的前端视觉时延	$T_{\text{front}} = (t_2 - t_1) + (t_4 - t_3)$	从请求图像到获得模型输入图像	由摄像头/JPU ioctl 墙钟与 VPSS ioctl 墙钟相加，无独立硬件探针
TPU 输入 TDMA 搬运	在模型正式计算前，将 VPSS 输出准备到 TPU 输入张量所需位置的时间	$T_{\text{TDMA}} = t_6 - t_5$	将 VPSS 输出导入 TPU 输入张量	`t5` 位于 `CVI_NN_SetTensorWithAlignedFrames()` 调用前，`t6` 位于该调用返回后

表 11: 视觉链路性能指标定义与测量探针（上）

指标	指标含义	计算方式	计时范围	探针位置
TPU 推理	从提交模型命令流到前向推理完成并返回的时间	$T_{\text{forward}} = t_8 - t_7$	命令提交、模型计算、内部访存、中断等待及运行时返回	`t7` 位于 `CVI_NN_Forward()` 调用前，`t8` 位于该调用返回后；内核另在命令触发、完成 IRQ 和任务恢复处设置诊断探针
后处理	解析 TPU 的 FP32 输出，执行置信度过滤、NMS 和检测框坐标恢复的时间	$T_{\text{post}} = t_{10} - t_9$	输出解析、阈值过滤、NMS 及检测框坐标恢复	`t9` 位于 `get_detections()` 调用前，`t10` 位于 `nms()` 与 `correct_yolo_boxes()` 完成后
其他视觉开销	无法归入上述主要阶段的剩余视觉处理时间	$T_{\text{other}} = (t_{11} - t_0) - [(t_2 - t_1) + (t_4 - t_3) + (t_6 - t_5) + (t_8 - t_7) + (t_{10} - t_9)]$	状态整理、时间戳记录、函数边界及取整残差	由单帧视觉外层墙钟扣除五个已明确计时的主要区间得到，底盘和机械臂动作不计入 `t11 - t0`
视觉总时延	当前串行模式下一帧图像从请求到产生检测结果的平均响应时间	$T_{\text{vision}} = t_{11} - t_0 = T_{\text{camera}} + T_{\text{JPU}} + T_{\text{VPSS}} + T_{\text{TDMA}} + T_{\text{forward}} + T_{\text{post}} + T_{\text{other}}$	从请求图像到得到检测结果，不含底盘和机械臂动作	`t0` 位于单帧视觉处理开始处，`t11` 位于检测结果形成处；逐帧生成 `StageTimingSample` 后累计求平均
单帧平均置信度	成功检出帧中，被选为追踪目标的网球框置信度均值	$\overline{C} = \frac{100}{N_d} \sum_{i=1}^{N_d} s_i$	检出帧中面积最大网球框的置信度统计	用户态 `handle_detections()` 选择面积最大的检测框，`ConfidenceTotals::record()` 累计其置信度

表 12: 视觉链路性能指标定义与测量探针（下）

其中，StarryOS 的“摄像头等待”采用摄像头/JPU ioctl 墙钟减去成功 JPU 解码时间得到，因此它是一个残差指标，不等同于纯 USB 传输时间，还可能包含等待新帧、JPEG 临时复制、锁操作和系统调用开销。VPSS 的提交时间表示内核接收请求到硬件启动之间的时间，其中包含参数校验、设备锁等待和寄存器配置，并非单纯的任务排队时间。TPU 输入 TDMA 搬运和 TPU 推理均采用用户态 API 墙钟时间，其中同时包含运行时封装、驱动调用、硬件执行、中断等待和返回用户态的开销。

StarryOS 的 JPU、VPSS 和 TPU 可以通过内核时间戳进一步区分硬件执行与软件调度开销；Linux 基线受厂商接口限制，摄像头采用 `poll() + VIDIOC_DQBUF` 墙钟，JPU 采用 `CVI_VDEC_SendStream() + CVI_VDEC_GetFrame()` 墙钟，VPSS 采用 `CVI_VPSS_SendFrame() + CVI_VPSS_GetChnFrame()` 墙钟。因此，单独的摄像头等待和 JPU 解码存在一定的接口边界差异，“摄像头/JPU 等待 + VPSS”和视觉总时延更适合评价系统级性能。所有均值只统计成功完成完整视觉链路的网球帧.

4.3.3　表格结果解释

StarryOS 在图像前端取得了最明显的优势。其“摄像头/JPU 等待 + VPSS”为 8.776 ms，比 Linux 的 12.949 ms 少 4.173 ms，降低约 32.23%。进一步分解可见，StarryOS 的摄像头等待比 Linux 多 5.438 ms，但 JPU 解码少 9.388 ms，VPSS 又少 0.222 ms。也就是说，StarryOS 前端的净优势几乎全部来自 JPU 路径；该优势不仅补偿了摄像头等待的劣势，还为后续阶段留下了约 4.173 ms 的时延余量。

进入 TPU 阶段后，Linux 更占优势。StarryOS 的 TDMA 比 Linux 多 0.143 ms，TPU 推理多 1.588 ms，后处理多 1.183 ms，其他视觉开销多 0.054 ms，这四部分合计使 StarryOS 比 Linux 多消耗约 2.968 ms。最终，前端节省的 4.173 ms 抵消了后端多出的约 2.968 ms，形成约 1.205 ms 的净优势，与报告总时延差 1.206 ms 基本一致，剩余 0.001 ms 来自逐项四舍五入。

因此，本轮数据支持的结论是：StarryOS 依靠更短的专用 JPU 前端路径，在后端仍略慢于 Linux 的情况下实现了略低的单帧总时延；该结果不代表所有驱动阶段均快于 Linux。

4.3.4　StarryOS 快于 Linux 的原因分析

1)面向固定工作负载的专用内核通路

Linux 基线使用标准 V4L2 摄像头接口以及 CVITEK Multimedia Framework 中的 VDEC、VPSS 等通用组件。其优势是接口完整、兼容性强，但一次处理会经过 `poll()`、`VIDIOC_DQBUF`、VDEC 码流提交、VDEC 输出帧获取、VPSS 送帧和取帧等多层框架接口。

StarryOS 则针对当前固定的 SG2002、UVC MJPEG 4:2:2 摄像头和 640×640 模型构建了专用数据路径。UVC 后台任务负责持续组帧，视觉线程通过专用 ioctl 获取最新帧；JPU 直接将解码结果写入预先分配的 ION 物理缓冲区；VPSS 再直接读取该缓冲区。专用路径减少了通用媒体框架的对象管理、队列切换和多次 API 往返，因此在固定工作负载下可以获得更低的控制路径开销。

2) 最新帧覆盖策略避免队列积压

StarryOS 摄像头驱动采用异步采集和“只保留最新完整帧”的策略。当摄像头产生图像的速度高于推理速度时，新帧可以覆盖尚未消费的旧帧，前台不会沿着历史队列逐帧追赶。这种设计降低了机器人闭环控制中的帧龄，也避免了排队时延持续增长。

Linux V4L2 同样具备异步环形缓冲能力，但其基线程序按照 DQBUF、处理、QBUF 的标准队列语义消费帧。两者的目标并不完全相同：Linux 更强调通用缓冲管理，StarryOS 更强调机器人控制所需的最新状态。因此，StarryOS 的设计更适合“允许丢旧帧、不能积压”的实时控制任务。

3) JPU 解码结果直接进入共享物理缓冲区

StarryOS 的 JPU 输出直接写入同时具有 CPU 虚拟地址和硬件物理地址的 ION 缓冲区，后续 VPSS 使用同一物理缓冲区，不需要把完整的 640×480 YUV 图像复制到普通用户态缓冲区。JPU、VPSS 和 TPU 之间以物理地址和共享内存衔接，是 StarryOS 前端获得低时延的重要基础。

Linux 的 VDEC/JPU 同样使用硬件缓冲区，并非纯 CPU 拷贝实现；但是 Linux 计时范围明确包含 `CVI_VDEC_SendStream()` 和阻塞式 `CVI_VDEC_GetFrame()` 的完整 SDK 墙钟时间。该通用

VDEC 通道还涉及输入码流排队、输出帧管理以及框架同步。StarryOS 的 `jpu_decode_us` 取自更专用的内核解码路径，因而报告值明显更低。

需要强调的是，两端使用的是同一块 SG2002 JPU 硬件。1.645 ms 与 11.033 ms 的巨大差距不应直接解释为 JPU 硬件算力差异，更合理的解释是驱动路径、框架开销、排队方式以及计时边界共同造成了差异。前端合计 8.776 ms 对 12.949 ms，比单独的 JPU 数字更适合支撑系统级结论。

4)VPSS 专用离线任务开销较小

StarryOS 通过 `/dev/cvi-vpss0` 的专用离线任务一次完成 YUV422P 到 RGB、缩放、letterbox 和三平面输出，并通过 IRQ 等待硬件完成。实测 1.643 ms 与 Linux 的 1.865 ms 只相差 0.222 ms，说明两端 VPSS 硬件执行能力基本一致，StarryOS 的任务配置和中断返回路径没有引入明显额外开销。StarryOS 略快可能来自更短的专用 ioctl 路径，也可能处于测量波动范围内，不宜将 0.222 ms 解释为硬件性能差异。

5)内核调度链路已经过针对性简化

StarryOS 早期 TPU 路径曾存在提交线程、worker 和完成等待线程之间的多次调度交接。经过优化后，推理任务改为更直接的串行提交，TPU IRQ 唤醒后的调度位置也得到改善。此前分段计时表明，worker 排队和 IRQ 恢复造成的几十毫秒延迟已经被消除。虽然当前 TPU 墙钟仍略慢于 Linux，但这些改动使 StarryOS 不再因通用调度链路而出现数量级上的劣势，并为本轮总时延追平 Linux 提供了基础。

4.3.5　StarryOS 慢于 Linux 的可能原因

1) 摄像头等待受到请求相位影响

StarryOS 的摄像头等待为 5.489 ms，而 Linux 只有 0.051 ms。Linux 的计时范围是从调用 `poll()` 到 `VIDIOC_DQBUF` 完成；当程序请求时 V4L2 队列中已经存在完整帧，`poll()` 和 DQBUF 可以近乎立即返回。该数值更多表示“消费者请求时缓冲区是否已有帧”，而不是摄像头完成一帧传输只需 0.051 ms。

StarryOS 的视觉线程通过带 `last_sequence` 的专用 ioctl 请求序号更新的帧。如果当前最新序号已经消费，调用会等待后台 UVC 任务发布更新帧，因此更容易计入下一帧到达前的剩余时间。这种语义能够避免重复处理同一帧，但会将帧周期相位显式反映在摄像头等待字段中。

因此，5.489 ms 对 0.051 ms 主要体现缓冲语义和计时位置差异，不能据此判断 Linux USB/UVC 驱动的数据传输速度比 StarryOS 快一百倍。若要公平比较摄像头子系统，应进一步统一“USB 帧完成时间戳—消费者取得时间戳”的定义，并同时报告帧率、帧龄、丢帧率和序号跳变。

2) TPU 用户态墙钟仍包含约 2 ms 软件开销

StarryOS 的 TPU 推理为 41.309 ms，比 Linux 多 1.588 ms，约慢 4.00%。内核分段测试已经显示，StarryOS 的 `fire → IRQ` 硬件命令流时间约为 39.4 ms，与 Linux 本轮 39.721 ms 的整个 TPU 推理量级非常接近。这说明 SG2002 TPU、模型命令流和主频并不存在明显的硬件算力差距。

StarryOS 从约 39.4 ms 的硬件完成时间增加到 41.309 ms 的用户态 `CVI_NN_Forward()` 墙钟时间，中间还包含运行时提交、系统调用、IRQ 后任务恢复、驱动收尾以及返回用户态等开销。Linux 的官方驱动采用成熟的 dma-buf、异步 submit/wait、硬件 IRQ 和等待队列机制，其任务生命周期和运行时配合更完整，因而可能在完成通知和返回用户态方面更紧凑。

此外，TPU 命令流会与 DDR、VPSS、USB 等模块共享内存系统。即使 TPU 与 FAB 分别工作在相同的 700 MHz 和 900 MHz，测试瞬间的 DDR 争用、缓存状态和中断负载仍可能造成 1～2 ms 的墙钟波动。因此，在没有同轮 `fire → IRQ`、IRQ 恢复、运行时返回和 DDR 带宽数据之前，不能把全部 1.588 ms 都归因于 TPU 驱动本身。

3)TDMA 差距较小，可能来自 DMA/cache 路径

StarryOS 的 TPU 输入 TDMA 为 1.114 ms，Linux 为 0.971 ms，相差 0.143 ms。绝对差值很小，可能同时受到定时器粒度、调度抖动和内存状态影响。从架构上看，Linux 官方 TPU 驱动使用 dma-buf 获取物理地址，并通过体系结构 DMA cache 同步接口维护一致性；StarryOS 当前采用 ION 零拷贝、coherent-uncached 映射，并将 cache ioctl 主要作为同步屏障。

Linux 的成熟 DMA 映射和运行时协作可能减少提交与同步开销，而 StarryOS 的专用 ABI、文件描述符解析、物理地址校验和完成等待仍会贡献少量固定成本。不过，0.143 ms 只占总时延约 0.27%，当前不应作为首要优化目标，除非多轮测试稳定复现并能由内核分段计时定位到具体步骤。

4)非缓存映射可能增加 CPU 后处理读取成本

StarryOS 后处理为 1.632 ms，Linux 为 0.449 ms，StarryOS 多 1.183 ms。两端的 `tpu.rs` 已核对为完全一致，均直接读取运行时管理的 FP32 输出张量，再执行阈值筛选、NMS 和坐标恢复。因此，该差异更可能来自运行环境，而不是后处理算法源代码。

一个值得重点验证的内核原因是输出张量的缓存属性。StarryOS 当前明确采用 coherent-uncached 映射策略；若 TPU 输出张量在 CPU 侧也通过非缓存映射读取，那么 CPU 扫描约 42,000 个 FP32 元素时会产生较高的内存访问开销。Linux 通常采用可缓存的用户映射，并在设备与 CPU 转换所有权时执行 DMA invalidate；完成一次同步后，CPU 可以利用数据缓存高速解析输出。该差异能够合理解释“TPU 硬件时间接近，但 StarryOS CPU 后处理明显更慢”的现象。

这一解释目前仍属于高可信度假设，而不是已完成的因果证明。要确认它，需要分别记录输出张量映射属性、cache invalidate 时间、纯顺序扫描带宽，以及不执行 NMS 时的张量遍历时间。

5) CPU 调度、频率和样本内容也会影响后处理

即使源代码相同，CPU 主频与动态调频策略、编译工具链、内存分配器、缓存冷热状态、系统中断负载和线程调度仍会影响 Rust 后处理时间。后处理耗时还与超过置信度阈值的候选框数量有关；不同测试过程中摄像头采到的目标位置和背景略有变化，可能使 NMS 的输入数量不同。

因此，下一轮应同时输出每帧原始候选数、阈值过滤后候选数、NMS 保留数、CPU 主频和上下文切换次数。在这些数据缺失时，不能只凭 1.632 ms 与 0.449 ms 就断言 StarryOS 的 CPU 或调度器整体慢于 Linux。

4.3.6　对置信度结果的解释

StarryOS 的平均置信度为 93.13%，Linux 为 93.62%，差值为 0.49 个百分点。两端使用同一模型，VPSS 输出格式和模型输入布局也已按同一标准实现，因此该差距更可能来自实际采样帧不同、球在画面中的大小与位置不同、曝光变化以及 letterbox 输入的逐像素微小差异，而不是操作系统直接改变了神经网络精度。

0.49 个百分点相对于当前场景波动较小，可以认为两端输出质量基本一致。但是，若要形成正式精度结论，仍需固定同一批输入图像或保存同一批 MJPEG 帧，在两个系统上离线回放，并计算 Precision、Recall、mAP50 和 mAP50-95。实时场景下的平均置信度只能作为一致性参考。

4.3.7　综合结论

本轮测试表明，StarryOS 通过专用 UVC/JPU 驱动、最新帧策略、ION 共享物理缓冲区以及紧凑的 JPU—VPSS 数据通路，在图像前端建立了约 4.173 ms 的优势。该优势抵消了 StarryOS 在 TPU 输入、TPU 推理、CPU 后处理和其他开销方面合计约 2.968 ms 的劣势，使其最终视觉总时延比 Linux 少约 1.206 ms，即快约 2.22%。

从系统架构角度看，StarryOS 的优势来自针对固定机器人工作负载进行跨层协同和路径裁剪；Linux 的优势则来自成熟的 DMA/cache 管理、TPU 官方驱动、等待队列和通用运行时生态。当前结果说明 StarryOS 已经在成功视觉帧的平均延迟上追平并略微超过 Linux，但领先幅度较小，尚不足以宣称在所有负载和统计口径下全面优于 Linux。

后续优化应优先统一两端摄像头和 JPU 的时间戳边界，拆分 TPU 的硬件执行、IRQ 恢复和运行时返回，验证 TPU 输出张量的缓存属性并优化 CPU 后处理，并在相同图像回放、相同温度和相同系统负载下进行多轮测试，同时报告平均值、P50、P95、P99、失败率和任务墙钟。VPSS 已接近硬件能力，不再作为主要压缩对象。完成这些工作后，才能对两种系统的架构效率作出更严格、可复现的结论。

应用层的对比以端到端帧到指令延迟为准；系统层的三条线各自回答不同的问题——固定开销 收敛到多少、实时性是否可承诺、被测过程是否可重复——因此数据分组给出，不并入同一个总分。

4.4　内核优化的测试说明

4.4.1　本线特有的测试口径

全篇统一的评测口径（同一块板、同一份符合 Linux ABI 的静态可执行文件、两侧各测一次、未测项显式标注）已在“测试说明”中给出，本节不重复；内核这条线在此基础上另有三条 特有约定，因为它的对照组是“优化前/优化后”，而不同次上电之间的读数本身会漂移。

项	内容
板卡	AKA-00-SG2002（LicheeRV Nano 血统），CPU 为 C906（riscv64，单应用核），内存 236 MiB
对照系统	厂商 Linux 5.10.4，卡上 `linux.sd`，同板同卡同一次上电周期内交替启动
用户态	同一份静态链接 riscv64 ELF。所有探针都是自研 C 程序，两侧编译同一份源码
计时	板端 `rdtime`（时基 1 MHz）+ 主机侧串口逐行时间戳；微基准取 20–100 轮中位
口径	每个指标三列：Linux 实测／优化之前／优化之后

表 13: 内核优化的评测平台与对照口径

三条特有约定如下：

1. **冷启动受控 A/B**。内核“优化前/后”的对比尽量在**同一次上电、同一份探针、交替顺序** 下做（例如两两交替各测两遍取均值），以抵消“启动后第一次大分配明显更快”这类时间漂移。
2. **探针本身有代价**。 热路径上的分段计时是每次系统调用多 12 次取时间戳，实测平白增加 约 1.2 μs。凡是“优化前/后”的对照数字，探针都已撤掉；带探针的中间过程只在归因时出现，并注明口径。
3. **分段计账取快照之差**。 内核侧的分段数字由 `/proc/fault_attrib` 的计数器给出，取 “工作负载前后的快照之差”折算为每次调用的平均值；该口径只反映被测量路径自身，不含探针与被测负载之外的干扰。

4.4.2　探针与验收门

探针	测什么	对应的优化族
`syscost2`	九个系统调用的单次开销（含 `invalid_syscall` 对照）	K1 系统调用返回路径
`sa1`	返回路径的开关分段（单步 / ptrace / seccomp / trace）	K1
`sp2`	`fork`+`exit`+`waitpid` 与 `fork`+`execve`+`waitpid` 的墙钟	K2、K3
`fk1`/`fk2`	`fork` 内部分段（页表复制 / 首次入队 / 建线程）	K2
`faultcost`	匿名首触成本 vs 区域大小（1/4/16/64 MiB）	K4、K5
`faultattrib`	缺页的 plan/prepare/apply/TLB 分段	K4、K5
`popcost`	逐页缺页与整段物化的对照（动手前的上界实验）	K5
`touch64`	冷启动 64 MiB 首触单点（受控 A/B 口径）	K5
`bloat`	预取窗口的膨胀验证（短流 / 跨步 / 随机 / `MADV_DONTNEED`）	K5
`cowtest`	`fork` 后逐页改写父子各自正确、`MAP_POPULATE` 写读校验	K5
`read_bench`	不同缓冲大小下的读吞吐	K6
`read_syscall`	`open`+`read`+`close` 的分档中位耗时	K6、K7
`rp1`/`rp2`	warm/fresh 缓冲读，以及 `mmap`/读/`munmap` 三段拆解	K5、K6
`mmc`	TLB 一致性自检（固定地址反复映射-填-拆除-再映射）	K6
`block-rw-bench`	ext4 顺序写与 `fsync`（多档位）	K7
`/proc/fault_attrib`	内核侧分段计数、索引插入位置、回收队列长度	K2、K4、K5

表 14: 本项目的探针清单。`/proc/fault_attrib` 是内核侧计数出口，所有分段数字都由“工作负载前后快照之差”取得

每一处改动上线前都过同一组门：

层	判据	适用族
宿主单测	文件系统与内存相关 crate 的宿主回归全绿（内核 crate 收尾时 261 项）	全部
板端功能	标准 harness 全项 + 功能用例（`ls`/`head`/`grep`/`sh -c`/退出码）	全部
数据正确性	同一文件多种分块读的校验和一致，且与主机侧直接导出的文件逐字节相同	K3、K6
内存不泄漏	mmap/touch/munmap 反复折腾后可用内存稳定；预取窗口另有膨胀验证	K5
一致性	TLB 一致性自检不匹配页数为 0（改 TLB 语义的改动必须跑）	K6
压力	多二进制 `exec` 压力、`fork`/`exec` 风暴、泄漏检查与调度探针全过	K1、K2
可回退	动语义的改动都留运行期回退开关（`/proc/icache_flush`、`/proc/fault_around`）	K1、K5

表 15: 验收门。凡改动动到对外语义（指令缓存、TLB、访问时间、地址空间退休），都额外做了对应的专项自检

板端跑一组微基准（两侧同一份命令）：

```
# StarryOS 侧（登录后的 shell）
cd /root/probes
./syscost2                          # 系统调用入口
./sp2                               # 进程创建与 exec
./faultcost 64                      # 64 MiB 首触
./read_syscall /bin/busybox 1 100
./rp1 && ./rp2                      # warm/fresh 读与三段拆解
./mmc                               # TLB 一致性自检
cat /proc/fault_attrib              # 内核侧分段（读前读后各取一次，相减即"每次调用"）
```

4.5　内核优化的测试结果

4.5.1　总体对照

指标	Linux	优化前	优化后	优化后/ Linux
`getpid`（空系统调用）	422.5 ns	10560 ns	1631 ns	3.9×
`pipe_wr`（4 KiB 往返）	4489.2 ns	46819 ns	12576 ns	2.8×
管道 ping-pong	26 µs	445.7 µs	421.1 µs	16.2×
`fork`+`exit`+`waitpid`	338.1 µs	7497 µs	6461 µs	19.1×
`fork`+`execve`+`waitpid`	2407 µs	43537 µs	22359 µs	9.3×
上下文切换	10.97 µs	226.8 µs	220.4 µs	20.1×
双线程 `sched_yield` 交接	6.9 µs	171 µs	112.5 µs	16.3×
64 MiB 匿名首触（默认 4 KiB）	7899 ns/页	1302123 ns/页	44120 ns/页	5.6×
64 MiB 首触（显式大页）	5683 ns/页	22216 ns/页	8014 ns/页	1.41×
ext4 `open`+`read`(1 KiB)+`close`	42 µs	10320 µs	454 µs	10.8×
8 MiB 顺序写 + `fsync`	790 ms	7961 ms	931 ms	1.18×
8 MiB 页缓存顺序写	34 MiB/s	2.41 MiB/s	42.43 MiB/s	我方更快
缓存命中读（warm）	671.6 MB/s	374 MB/s	474 MB/s	1.42×
缓存命中读（fresh）	271.1 MB/s	50.2 MB/s	62.8 MB/s	4.32×
单次大读 803 KB	277.2 MB/s	60.9 MB/s	76.2 MB/s	3.64×
冷启动到可交互 shell	33.2 s †	12.3 s	10.8 s	我方更快

表 16: 内核优化的总体结果。Linux 列为同板实测；“优化后/Linux”小于 1 表示本侧更快。上下文切换与管道往返本轮只做了归因、未改行为，故仍落后一个数量级（见 5.3 节）；† 末行两侧的启动锚点不同，只作量级参考，口径见 4.5.7 节

4.5.2　系统调用与进程间通信

指标	Linux 5.10.4	优化前	优化后
`getpid`（空系统调用）	422.5 ns	10560 ns	1631 ns
`pipe_wr`（4 KiB 往返）	4489.2 ns	46819 ns	12576 ns
双线程 `sched_yield` 交接	6.9 µs	171 µs	112.5 µs
管道 ping-pong	26 µs	445.7 µs	421.1 µs
上下文切换	10.97 µs	226.8 µs	220.4 µs
`memcpy` / 顺序读带宽	1.024 / 1.488 GB/s	0.984 / 1.420 GB/s	0.981 / 1.428 GB/s

表 17: 返回路径瘦身前后的对照。前三行是“调用体很轻”的指标，收益最直接；后三行主要由调度与内存主导，与归因一致

`getpid` 从 25.0 倍收到 3.9 倍、`pipe_wr` 从 10.4 倍收到 2.8 倍，`sched_yield` 交接下降约三分之一。有两个口径必须写明：其一，带探针时会低估收益——同一镜像 在带 5 个系统调用探针时 `getpid` 显示为 3 050 ns，而撤掉探针后是 1631 ns，也就是 说真实收益是 6.5 倍；其二，`getpid` 的复测区间是 1233–1631 ns，上表取最差值，最好的 1233 ns 对应 2.9 倍。剩下的固定成本构成与“为什么停手” 见 5.3 节。

4.5.3　进程创建与 exec

指标	Linux 5.10.4	优化前	优化后
`fork` + `exit` + `waitpid`	338.1 µs	7497 µs	6461 µs
`fork` + `execve` + `waitpid`	2407 µs	43537 µs	22359 µs
└ 其中 execve 内核侧	未测	32.3 ms	**4.5 ms**
上下文切换	10.97 µs	226.8 µs	220.4 µs
双线程 `sched_yield` 交接	6.9 µs	171 µs	112.5 µs
子地址空间退休	未测	3570 µs	410 µs

表 18: 进程与 exec 面的实测。`fork_exec_exit` 收掉 49%，exec 内核侧收掉 86%；`fork_exit` 只降 14%，原因见 5.3 节 的归因

`fork_exit` 只降 14%，看起来与前面几处改动不成比例：其剩余时间主要落在“切到一个新任务 再切回来”的 11–14 次上下文切换与调度热路径的固定开销上，页表复制只占小头。因此 `fork` 的下一步与上下文切换是同一件事，两者必须一起做。

4.5.4　缺页与地址空间

64 MiB 匿名首触	Linux	优化前	优化后
默认 4 KiB 路径，每页	7899 ns/页	1302123 ns/页	**44120 ns/页**（29.5×）
显式大页路径，每页	5683 ns/页	22216 ns/页	8014 ns/页

表 19: 缺页路径的最终位置。默认路径相对自身优化前是 29.5×，相对 Linux 从 165 倍收到 5.6 倍；显式大页路径只差 1.41 倍

分档看“成本不再随区域膨胀”这件事最直观（ns/页）：

区域	优化前	优化后（索引）	优化后（窗口）
1 MiB	154179	168918	134962
4 MiB	307024	151250	63939
16 MiB	587627	153875	57658
64 MiB	1302123	162213	**44120**

表 20: 优化前每页成本随区域线性上升（154 µs → 1 302 µs）；修掉页索引后四条基本持平，窗口批量物化再整体下移一档

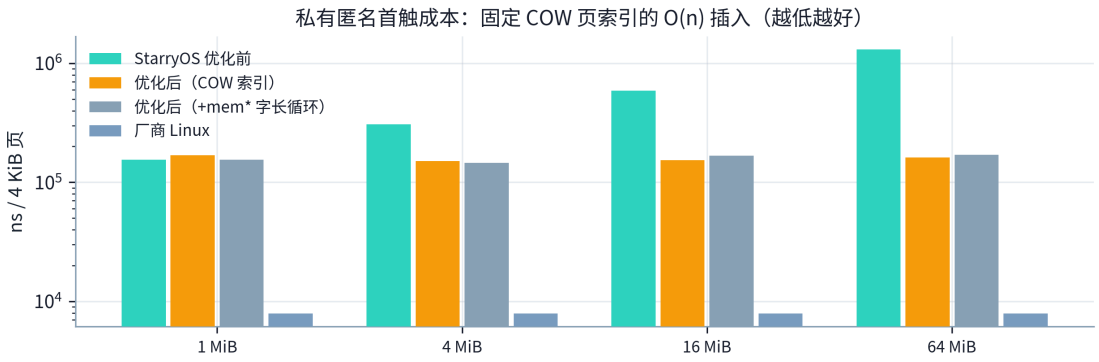


图 4: 优化前/后的首触成本随区域大小的变化，以及与 Linux 的对照。优化前四条随区域线性上升，优化后四条变平（对数轴）

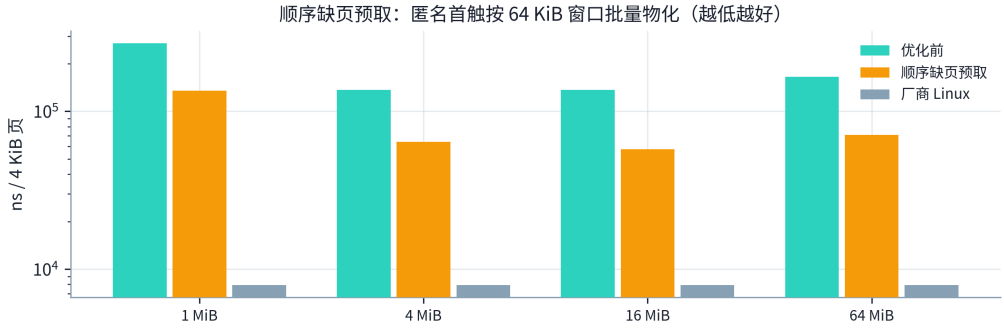


图 5: 顺序缺页预取的同板冷启动受控 A/B（两遍均值）。四种区域大小一致下降约 2.0–2.4 倍，显式大页路径不变

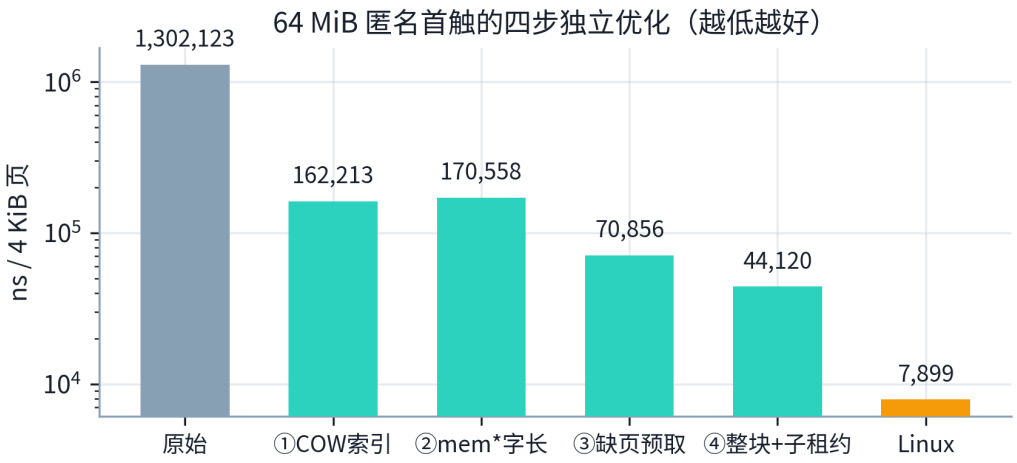


图 6: 64 MiB 匿名首触的逐级优化历程，以及与 Linux 的对照。四个台阶分别是页索引、字长内存操作，以及窗口批量物化里的“窗口预取”与“帧整块 + 子租约”两步（对数轴，越低越好）

4.5.5　文件读路径

指标	Linux 5.10.4	优化前	优化后
缓存命中读（warm）	671.6 MB/s	374 MB/s	474 MB/s
缓存命中读（fresh）	271.1 MB/s	50.2 MB/s	**62.8 MB/s**
单次大读 803 KB	277.2 MB/s	60.9 MB/s	76.2 MB/s
ext4 `open`+`read`(1 KiB)+`close`	42 µs	10320 µs	454 µs

表 21: 读路径的最终位置。`open`+`read`+`close` 改善 22 倍，读吞吐改善约 1.25 倍；fresh 一列仍差 4.3 倍，差距在每页固定开销上

同一份文件三种分块方式（4 KiB、64 KiB、`md5sum` 默认）读出的校验和完全一致，且与在 主机侧从板子根分区直接导出的同一文件逐字节相同。每轮换一块新 mmap 缓冲时，整轮 803 KB 被拆成三段：`mmap` 1.5–1.8 µs/页、读（首触 + 拷贝）从 49.68 µs/页降到 44.55 µs/页、`munmap` 从 29.51 µs/页降到 23.39 µs/页， 整轮合计从 80.7 µs/页降到 69.75 µs/页。

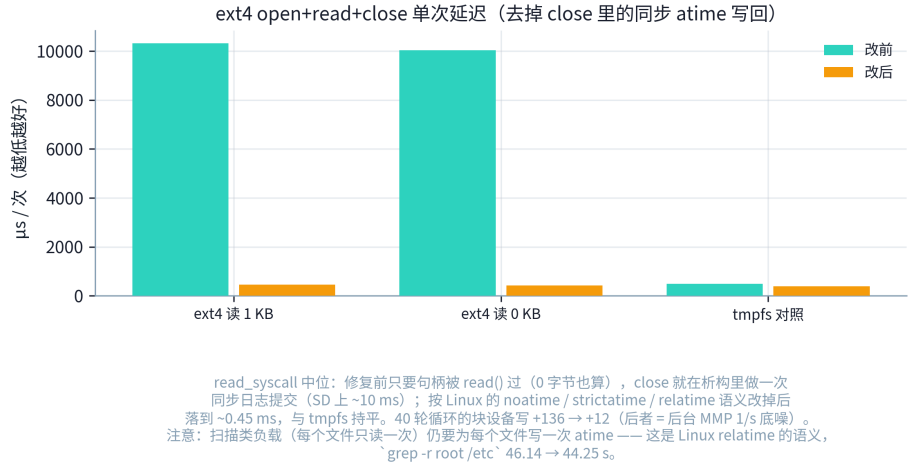


图 7: ext4 `open+read+close` 的中位耗时（对数轴）：优化前 10 320 μs、优化后 454 μs、同板 Linux 42 μs

4.5.6　存储写回与元数据

8 MiB 档，`fsync` 耗时	Linux ms	优化前 ms	优化后 ms
扇区写（512 B）	814	7 961	1 050
页写（4 KiB）	790	7961	**931**
硬件最大块（1 MiB）	857	7 636	926
规划器切分（1 MiB）	793	7 754	934

表 22: 写回批量化前后的 `fsync` 耗时。四个档位全部落到与 Linux 同一水平（±20%），不再有“档位越大越慢”的现象

指标	Linux 5.10.4	优化前	优化后
8 MiB 顺序写 + `fsync`	790 ms	7961 ms	**931 ms**
8 MiB 页缓存顺序写	34 MiB/s	2.41 MiB/s	**42.43 MiB/s**
ext4 `open+read`(1 KiB)+`close`	42 μs	10320 μs	**454 μs**
`fsync`(2 MiB)	790 ms	未测	2000 ms

表 23: 存储面的最终位置。写 + `fsync` 差 1.18 倍、顺序写吞吐反超、`open+read+close` 差 10.8 倍；`fsync`(2 MiB) 仍差 2.5 倍，属未完成项

吞吐提升有一条必须写清的边界：**页缓存容量把吞吐提了 17.6 倍，但“写完 + `fsync`”的端到端时间几乎不变**。因为 8 MiB 现在一次性回写，而这块 SD 卡约 1 MiB/s 的顺序写速度才是真正的底——上限调到 20 MiB 档实测反而只有 1.0–1.4 MiB/s。所以这一族解决的是“每次小批量回写带来 的额外事务与抖动”，不是“卡本身慢”。

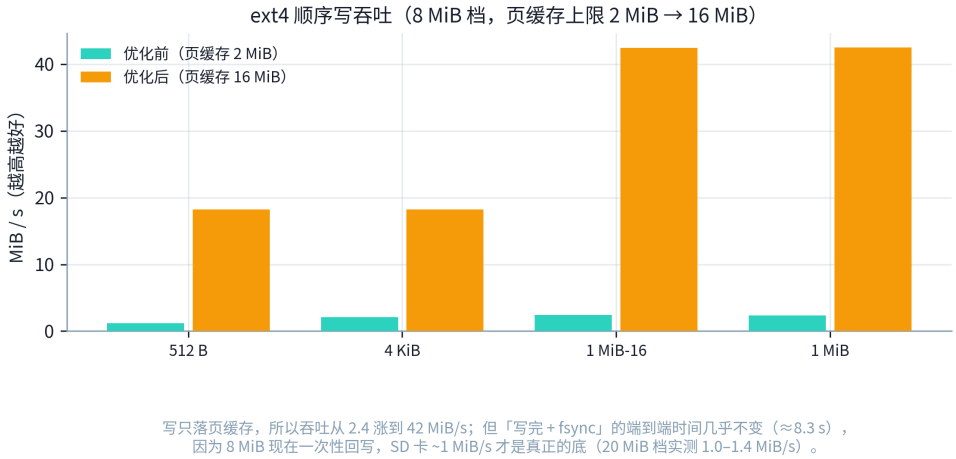


图 8: 页缓存上限提升前后的 ext4 顺序写吞吐（8 MiB 档）。吞吐提高 17.6 倍，但“写完 + `fsync`”的端到端时间不变——底在这块 SD 卡本身

4.5.7　启动时间

启动时间必须分段报，否则会把结论说反。用同一个主机侧探针（同一条串口、同样的逐行时间戳）把启动切成三段，两侧只在“就绪锚点”上不同——本侧取 shell 提示符，Linux 取 `getty` 的 `login:` 提示符：

段	Linux 5.10.4	优化前	归属
U-Boot `fatload`	1.04 s	1.45 s	镜像大小差异：两侧都是约 10.7 MiB/s
`bootm` → 首个用户态输出	1.2 s	**9.01 s**	**这一段才是内核**：内核初始化 + 设备探测
首个用户态 → 可交互提示符	**约 32 s**	4.2 s	rootfs 与 init 脚本
合计（`bootm` → 提示符）	33.2 s	13.21 s	端到端我方更快 2.5×

表 24: 启动分段对照。能归给内核的是中间那一段：本侧 9.01 s 对 Linux 1.20 s，落后 7.5 倍；而端到端本侧更快，主要因为 Linux 的 init 脚本很重

这张表给出两个不能混淆的结论：其一，**端到端我方更快**（13.21 s 对 33.2 s），但这主要来自 Linux 自己的脚本重，不能当成“内核更快”的证据；其二，**内核段落后 7.5 倍**（9.01 s 对 1.2 s），这一段全是内核初始化 与设备探测。本轮只动了一处固定等待，上电到用户态标记从 10.81 s 降到 9.61 s、到 shell 可交互从 12.3 s 降到 10.8 s； 其余静默块与不动的理由见 5.3 节。

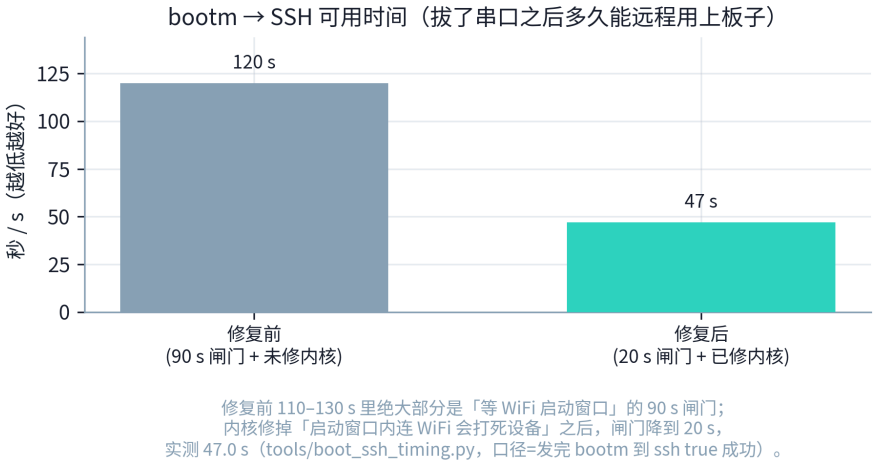


图 9: `bootm` → SSH 可用的时间。优化前绝大部分是“等无线设备启动窗口”的固定闸门（内核修复后闸门从 90 s 降到 20 s）

内核部分给出的是每次操作固定成本的收窄幅度与残余差距；实时性一侧无法由同一组负载 回答，需要专门的控制链路对照。

4.6　小核链路的测试说明与方法

这一条工作线的每组数字都按同一套方法取得，这套方法本身就是它的产出之一，因为它决定了“哪些结论可以被承诺”。

第一，**判据必须机器可判定**。每个实验都写成“一次运行给出一个布尔量与一个数值”，不用肉眼看日志判断成败：链路握手看“拔线冻结、插回恢复”，协议自检看“步数与异常计数”， 看门狗看“停喂心跳后停车计数是否递增”，堵转保护看“双轮按住是否触发一次并锁存”。

第二，**口径必须显式声明“包含什么、不包含什么”**。例如控制周期延迟要写明是否含串口发送 时间、是否含底盘板的执行时间；启动与登录的时间要写明锚点是“内核就绪”“首个用户态输出”还是“可交互”。凡两侧口径不能对齐的指标一律拆开报，不把差异记到被测对象头上。

第三，**所有板级数字三次重复取中位数，基线版本冻结并记录**。每一条滞后于现实的结论都要 能追溯到当次运行的原始日志；一旦发现结论与日志不符，以日志为准并更正结论。

判据与门禁也写进流程：如果测出底盘板自身已有链路超时，就优先改底盘板而不是动小核；如果命令路径的 p99 远小于机构容差，那么“确定性”就不成立为优化理由，叙事要改成“系统无关与链路所有权”。这套门禁在本文档里也确实改变过一次结论——见 5.7 节 关于“度量假象”的记述。

4.7　小核链路的测试结果

4.7.1　跨核往返延迟与稳定性

观测	实测值	含义
往返延迟（发送 + 等待指定应答）	274 / 277 / 299 µs	旧轮询实现下界 1000 µs
往返延迟分布	min 53 / p50 54 / p90 266 / max 413 µs	双峰：未入睡命中 / 中断唤醒
稳定性	`tx=rx=irq=12 dropped=0`（连续 5 次）	无丢包、无落空
共享区版本	`version=3 header=192`，Ring A/B 偏移 320 / 4416	与驱动常量逐项对应
小核 tick	5000.00 µs（−0 ppm），24 307/24 307 个间隔为 1 tick	两核时间基准一致，无需比例补偿

表 25: 跨核通道的实测。往返延迟的双峰形状说明“未入睡即命中”与“经中断唤醒”是两条不同的路径，p50 比旧轮询实现的下界低一个数量级

4.7.2　底盘链路接管的验收结果

阶段	命令 / 模式	实测结果	判据
链路握手	`0x61`，mode=1, kind=6	INIT 4→6、ACK 4→6、24→36 字节、帧错误 0；拔线冻结、插回恢复	小核独立完成收发
协议自检	`0x61` + mode=2, kind=7	5/5 步、异常 0、ACK 10、60 字节、帧错误 0	全程零速度指令
周期控制	`0x62`，mode=3, kind=8	默认 100 ms；每 5 拍回读 RPM；`dt_max = 周期/5`	闭环与大核负载解耦
心跳看门狗	`0x64` 设窗口（默认 500 ms）	停喂心跳后 `wd_stops` 逐次 +1，窗口内停车	大核失效后能停
堵转保护	kind=10	双轮按住触发 1 次并锁存；单轮差速不触发；改目标解除	不新增硬件即能“卡住就停”

表 26: 底盘链路接管的三阶段与两层保护的验收结果。每一行的判据都是布尔量，可以脚本复核

4.7.3　控制周期的确定性：大核直控与小核接管的 A/B 对照

要证明小核的价值，唯一有说服力的形式是**同机、同负载、同链路的对照**。A 组由大核用户态按固定周期写 `/dev/ttyS1`，用工具测量每一拍的实际间隔；B 组由小核按 tick 计数下发，kind=9 给出“恰好一个周期 / 多一个 tick / 多两个 tick”的分布。两组负载相同：空载、3 个 `yes` 打满大核 CPU，以及卡上真实的感知流水线（`akars-tennis-validator`，它不占用 `/dev/ttyS1`，所以两组合并跑）。

负载	A 组：大核直控 p99 / max	B 组：小核接管 kind=9	说明
空载	+0.21 ms / +0.22 ms	236/236 落在 20 tick（min=mean=max）	控制周期 100 ms
3 × `yes` 打满 CPU	**+1.54 ms / +1.75 ms**	227/227 落在 20 tick	同刻 uart 往返涨到 3 670 µs
视觉 validator 流水线	+0.947 ms / +0.950 ms	200/200 落在 20 tick	长尾取决于负载是否“持续就绪”
带载（20%，压地）	149 拍 p99 = 100 331 µs / max = 100 333 µs	4 802/4 802 = 100%（20 tick）	与空载同口径复测

表 27: A/B 对照。大核直控的控制周期偏差随负载恶化到约 8 倍，小核接管在全部负载下 min = mean = max，样本 100% 落在期望 tick 刻度；带载一行为 20% 占空比、车压地工况



图 10: 跨核往返延迟分布（中断驱动）与旧轮询实现的下界。所有条形按同一尺度绘制；中断驱动的 p50 比轮询下界低一个数量级



图 11: A/B 对照：大核直控（A 组）相对 100 ms 控制周期的 p99 与 max 偏差；小核接管（B 组）在全部负载下 min = mean = max

第三轮把负载换成真实感知流水线后出现了一个反直觉但重要的结果：A 组的 p99 偏差只有 0.947 ms、max 为 0.950 ms，**比 3 个 `yes` 反而更小**。原因不难解释：`yes` 是“永远就绪”的纯计算竞争者，会把用户态控制进程从时间片上挤开；视觉流水线在文件读与 TPU ioctl 上会阻塞，反而给调度器留下了放行窗口。也就是说，**大核直控的长尾取决于负载是否持续就绪，而不在于它“像不像视觉”**。小核一侧则与负载无关：200/200 仍全部落在 20 tick。带载（车压地、20% 占空比）复测同样成立。

4.7.4　控制周期档位与代价

控制的稳定性建立在“小核节拍不受大核负载影响”上，因此周期本身可以压得很短。把控制周期从 100 ms 一路压到 5 ms，各档位的节拍精度如下：每档只有切换瞬间的一个过渡样本落在 100 ms 刻度，稳态不抖动。



图 12: 控制周期可调范围与节拍精度：5–1000 ms 各档位实测落在期望 tick 刻度的样本比例

代价只有一条：目标速度从“大核写入”到“线上生效”要等一个控制周期（100 ms 周期下平均 50 ms、最坏 100 ms）。把控制周期压到 5 ms（1 个 tick）后，这一项降到**平均 2.5 ms、最坏 5 ms**，裸板实测 2 994/2 995 个样本落在 1 tick 刻度；代价是链路上 `SET_SPEED` 帧涨到 200 帧/秒，底盘板能不能吃下这个速率仍需带车验证。

小核部分给出确定性与失效兜底；而以上三组数据都依赖同一条前提：板子可被反复刷写、反复上电，且取数过程可脚本化。最后一条线给出这一前提的实测代价。

4.8　上板工作流与启动锚点的实测

板级 I/O 层不主张相对厂商 Linux 的性能优势，其交付物是“可反复暖重启、可无人值守 刷写与被测”的板卡与工作流。下表给出镜像通路与三个启动锚点的实测用时；所有数字都在 同一块板、同一条串口上取得，取法与内核侧各章的探针一致（主机侧逐行时间戳）。

通路／锚点	用时	说明
UMS 写入 16 MB FIT（mtools ＋ `sync`）	约 4.4 s	宿主侧块设备写入，无需板上网络、无需提权
U-Boot `fatload` 16 MB FIT	约 1.4 s	按同卡实测 10.7 MiB/s 折算
`bootm` → 首个用户态输出	10.81 → **9.61** s	缩短启动期等 DHCP 的阻塞宽限期
`bootm` → shell 可交互	12.3 → **10.8** s	同上
`bootm` → SSH 可登录	120 → **47** s	含本队自建的无线自联闸门（90 → 20 s），属脚手架而非内核指标
串口直传镜像	不采用	115200 8N1、shell 与控制台同路、无可靠流控

表 28: SG2002 上板工作流的实测用时。前两行是通路效率，中间两行是可交互性；其中“首个用户态输出”才是与内核可比的口径，其余各项均包含本项目自建的启动脚手架

读这张表要分清两件事。“首个用户态输出”是唯一能归给内核的锚点，它的分段构成、与 厂商 Linux 的逐段对照以及启动期那一处固定等待的优化，放在 4.5 节 给出；而“shell 可交互”与“SSH 可登录”两行各自包含本队搭的启动脚手架（无线自联 闸门、探针启动脚本），只能说明这块板在当前配置下的实际可用时间，不能当作内核指标。4.5.7 节 给出的端到端对照也遵循同一口径，即把“内核段”与“用户态与脚本段”分开报。

第 5 章　实现难点说明

本章记录“改动之前发生了什么”：每一处优化都是先被数据指出来的，而这些定位过程本身就是本项目在方法上的主要产出。写成难点而不是流水账，还有一重作用——它解释了方案为什么长成现在这样，也让后续接手的人不必重复同样的排除工作。按应用层视觉链路、系统层内核、实时小核、板级 I/O 四条线分别展开；其中被数据否掉的假设与尚未完成的工作也一并列出，因为“不做什么”与“做什么”同样需要证据。

以下内容归纳视觉推理链路中的代表性实现难点。它不穷举项目全部难题，其他内核子系统的定位、修复与验收过程可在本章并列扩展。

5.1　视觉推理链路优化难点

本文讨论的正式视觉链路以当前 640×640 网球模型为准。输入摄像头输出 640×480、Motion JPEG（MJPEG，运动 JPEG）压缩图像，JPEG 图像的亮度分量采样因子为 `0x21`，即 4:2:2 色度采样。模型输入张量为 `1×3×640×640`、8 位无符号整数、RGB planar、NCHW（Number–Channel–Height–Width，批次—通道—高度—宽度）布局；模型输出张量为 `[1,5,8400,1]` 的 32 位浮点数。正式模型文件为 `model/yolov8n_tennis_v2.cvimodel`.

缩写、数据格式与关键概念

缩写或术语	全称	本项目中的含义
USB	Universal Serial Bus	摄像头与 SG2002 之间的数据总线
UVC	USB Video Class	USB 摄像头的视频类协议及数据包格式
MJPEG	Motion Joint Photographic Experts Group	摄像头连续输出独立 JPEG 图像帧的压缩视频格式
JPU	JPEG Processing Unit	SG2002 片上 JPEG 硬件解码单元
YUV422P	YUV 4:2:2 Planar	Y、Cb、Cr 三平面存储，水平方向色度二分之一采样
VPSS	Video Processing Subsystem	完成颜色空间转换、缩放、边框填充和输出排布的硬件模块
CSC	Color Space Conversion	YUV 与 RGB 等颜色空间之间的转换
DMA	Direct Memory Access	硬件在不经 CPU 逐字节复制的情况下访问内存
ION	专有分配器名称	为多媒体与 TPU 提供物理连续、设备可访问的共享内存
TDMA	Tensor Direct Memory Access	TPU 的张量数据搬运单元
TIU	Tensor Instruction Unit	TPU 中执行卷积、激活等张量运算的计算单元
TPU	Tensor Processing Unit	SG2002 神经网络硬件加速器
IRQ	Interrupt Request	硬件完成后通知 CPU 的中断请求
MMIO	Memory-Mapped Input/Output	通过内存映射寄存器控制设备
ABI	Application Binary Interface	用户态与内核态之间稳定的数据结构、命令号和返回语义
ioctl	Input/Output Control	用户态通过文件描述符向设备驱动提交控制请求的接口
NCHW	Number–Channel–Height–Width	模型输入的四维张量排列顺序
NMS	Non-Maximum Suppression	去除同一目标重复检测框的非极大值抑制算法
IoU	Intersection over Union	两个检测框交集面积与并集面积之比
latest-frame	最新帧语义	消费者只处理最新完整帧，允许淘汰过时帧而不积压历史队列
letterbox	等比例缩放加边框填充	保持原图纵横比，将剩余区域填充成模型输入尺寸
zero-copy	零拷贝	大尺寸解码图像在设备间共享同一物理缓冲区；本项目并非所有数据完全无复制
W1C	Write One to Clear	向状态位写入 1 才能清除的中断寄存器语义

5.1.1　优化前的处理链路与问题模型

早期处理流程可概括为：前台应用同步等待摄像头图像，将压缩图像或 YUV 图像取回普通用户态内存，由中央处理器完成 JPEG 解码后的颜色转换、缩放、通道重排和 letterbox，再把处理结果复制到 TPU 输入张量，调用模型前向推理，最后在 CPU 上进行候选框解析与 NMS。该流程能够完成功能验证，但没有充分使用 SG2002 的 TPU, JPU 和 VPSS，也没有形成适合实时闭环控制的缓冲与调度模型。

早期观测数据为：视频采集约 73 ms、CPU 预处理约 56 ms、TPU 推理约 56 ms、后处理约 14 ms，单帧总处理时间约 189 ms。需要说明的是，四个分项直接相加为 199 ms，因此该早期数据只能作为瓶颈量级证据，不能与后续精确分段计时进行严格算术对比。不过它足以说明三个结构性问题：第一，采集是同步阻塞的；第二，约 56 ms 的像素级预处理消耗了有限的 C906 CPU 资源；第三，图像在普通缓冲区、设备缓冲区和 TPU 张量之间存在多次大块复制。单纯提高某个模块的吞吐量不一定缩短“看到目标—产生控制决策”的响应时间。例如，摄像头可以持续产生 30 帧每秒，但若应用严格按照先进先出顺序处理，每帧推理耗时又高于帧周期，队列就会积压，机器人实际处理的是越来越旧的图像。类似地，VPSS 硬件实际只运行 1.5 ms，但若等待线程在完成中断后 40 ms 才重新获得 CPU，则用户观察到的 VPSS 时间仍接近 42 ms。因此，优化必须同时覆盖数据面和控制面：

1. 数据面优化关注 JPEG、YUV 和 RGB 数据如何在 UVC、JPU、VPSS 与 TPU 之间流动，目标是减少 CPU 像素计算与大图复制；
2. 控制面优化关注任务如何入队、等待、被中断唤醒和恢复执行，目标是消除排队、错误唤醒策略和不必要的线程交接；
3. 契约优化关注每个设备真实支持的像素格式、步长、地址范围和张量布局，目标是在加速的同时保持检测结果正确；
4. 可观测性优化关注时间戳边界和异常统计，目标是区分硬件慢、调度慢、相位等待和统计重复；
5. 稳定性优化关注损坏 JPEG、位流耗尽、超时、摄像头复位以及错误码语义，目标是避免单帧故障演化成整机卡死。

最终正式链路如下：



图 13: 优化后的异构视觉推理数据通路

该架构按照职责划分内核态与用户态工作：内核负责设备生命周期、物理缓冲区访问、异步采集、硬件提交、中断和异常恢复；用户态负责模型契约校验、设备组合、检测后处理和机器人策略。这样既保留了策略迭代的灵活性，也避免让用户态重复实现硬件驱动职责。

5.1.2　异步采集与最新帧调度机制

原始实现采用同步采集模式，视觉线程只有在需要执行推理时才向摄像头请求图像，并持续阻塞至通用串行总线视频类（USB Video Class，UVC）数据包被组装为一幅完整图像。在这种模式下，图像采集、JPEG 解码、图像预处理和张量处理器（Tensor Processing Unit，TPU）推理严格串行，摄像头无法在视觉线程执行推理期间准备下一帧。如果请求时刻刚好错过上一幅图像，视觉线程还可能等待接近一个完整的摄像头帧周期。若使用先进先出队列保存所有采集帧，当图像生产速度高于推理速度时，又会形成历史帧积压，使机器人基于过时图像进行运动决策。因此，本系统没有追求处理摄像头产生的每一帧，而是以降低图像帧龄为目标，使每次视觉决策尽可能使用最新的完整图像。



图 14: 异步采集与最新帧发布机制

为此，摄像头驱动在 `cvi_usb_camera.rs` 中引入了 `AsyncCapture` 和 `LatestFrameStore`。`AsyncCapture` 负责管理后台采集任务的启动、停止、等待和消费状态。用户态通过 `CVI_CAMERA_IOCTL_START_ASYNC` 启动后台采集线程，驱动会检查其运行状态，避免重复启动多个线程并发访问同一个 UVC 端点。设备关闭或用户态发出 `CVI_CAMERA_IOCTL_STOP_ASYNC` 请求时，驱动设置停止标志并唤醒相关等待者，使后台任务能够有序退出。这样，图像采集从前台视觉线程中分离出来，使摄像头可以在前台执行解码、预处理和推理时准备下一幅图像。

`LatestFrameStore` 采用双缓冲机制保存 MJPEG 图像。两个 `Vec<u8>` 缓冲区分别承担“已发布缓冲区”和“生产者写入缓冲区”的角色：消费者只能读取当前已发布缓冲区，而后台线程始终向另一个非活动缓冲区写入新数据。其写入索引可以表示为 `published_index XOR 1`。只有当一幅 JPEG 图像完成组装并通过完整性检查后，驱动才统一更新帧元数据、递增帧序号并切换已发布索引。在此之前，消费者仍然可以安全读取上一幅完整图像。因此，双缓冲将正在接收的可变数据与已经发布的稳定快照分离，既防止消费者读取到半帧，也避免在整个 USB 数据包组装过程中长时间持有互斥锁。每一幅成功发布的图像都带有单调递增的帧序号 `sequence`。

为了避免后台采集线程持续占用单核 C906，当前实现还增加了“一帧预取”背压机制。后台任务只在尚未发布任何图像，或者消费者已经通过 `mark_consumed(sequence)` 确认取得当前图像后，才开始下一次采集。该机制采用容量上限为一幅已发布图像的有界生产者—消费者协议，禁

止生产者无限超前运行。用户态取得一幅有效图像后，驱动更新 `last_consumed_sequence` 并唤醒生产者；生产者随后可以在前台处理当前图像期间采集下一帧。该设计既实现了图像采集与视觉处理的前后台重叠，又限制了 UVC 线程对 USB 主机栈和中央处理器（Central Processing Unit，CPU）资源的持续占用，同时避免形成历史帧队列。完整图像在发布前还需要经过 JPEG 完整性校验。`capture_frame()` 最多执行八次底层采集，每次调用 `uvc_capture_one_frame_profiled()` 后，都会检查图像开头是否包含 JPEG 图像起始标志（Start of Image，SOI）`0xFF 0xD8`，末尾是否包含图像结束标志（End of Image，EOI）`0xFF 0xD9`，以及图像长度是否达到 `MIN_VALID_JPEG_BYTES=4096`。任一条件不满足时，驱动分别累计 `invalid_soi`、`invalid_eoi` 或 `invalid_too_small`，同时更新总无效帧数和重试次数，并重置 UVC 帧标识符（Frame Identifier，FID）的连续性状态。这样可以防止损坏帧的残留包状态影响下一次组帧。只有连续八次采集均失败时，驱动才向上层返回错误。

经过上述改造，摄像头子系统形成了由后台异步采集、JPEG 双缓冲、帧序号同步、最新帧发布、一帧背压和异常帧恢复共同组成的完整机制。其中，双缓冲负责保证生产者和消费者并发访问时的数据一致性；最新帧策略负责消除历史图像积压；帧序号和等待队列负责实现可靠同步；一帧背压用于控制后台线程的资源占用；JPEG 完整性校验则负责在进入硬件解码器之前拦截损坏图像。该方案使视觉线程能够获得尽可能新且完整的摄像头图像，并为后续 JPU 解码、VPSS 预处理和 TPU 推理提供稳定的数据输入。

5.1.3　JPU 硬件解码优化

摄像头输出的 MJPEG 码流由一系列相互独立的 JPEG 图像组成。早期处理链路如果 CPU 完成 JPEG 解码，就需要在单核 C906 上执行熵解码、反量化、逆离散余弦变换（Inverse Discrete Cosine Transform，IDCT）和颜色分量重建等计算密集型操作。这些任务不仅占用大量处理器时间，还会与设备驱动、检测后处理和机器人控制状态机竞争唯一的大核资源。为降低 CPU 负载，系统将 JPEG 解码迁移到 SG2002 内置的 JPEG 处理单元（JPEG Processing Unit，JPU）。驱动通过 `decode_planar_into()` 将经过完整性校验的 JPEG 码流提交给 JPU，由专用硬件完成码流解析和像素重建。CPU 仅负责配置寄存器、提交任务、等待完成中断、检查硬件状态以及解析输出格式，不再逐像素执行 JPEG 解码算法。实测 JPU 解码阶段的平均时延约为 1.6～1.7 ms，说明 JPEG 解码已经从主要的软件计算负担转变为时延较低的固定功能硬件任务。



图 15: JPU 硬件解码与异常恢复流程

JPU 的输出格式并不能由应用程序任意指定，而是首先取决于输入 JPEG 的色度采样方式。JPEG 图像的帧起始标记（Start of Frame，SOF）记录了各颜色分量的水平和垂直采样因子。当前摄像头输出的 JPEG 中，亮度分量 Y 的采样因子为 `0x21`，表示水平方向采用二比一的亮度—色度采样，而垂直方向不进行色度下采样，因此实际格式为 4:2:2。JPU 按照这一采样关系自然输出三

平面的 YUV422P 数据：640×480 图像的 Y 平面占 307,200 字节，Cb 和 Cr 平面各占 153,600 字节，总数据量为 614,400 字节。NV12 则属于 4:2:0 半平面格式，要求色度在水平和垂直方向都减半，并将 U、V 分量交织存储。若把当前 JPU 输出的 4:2:2 三平面数据直接解释为 NV12，将同时造成色度采样比例和内存布局错误。因此，正式链路保留 JPU 的 YUV422P 输出，再由视频处理子系统（Video Processing Subsystem，VPSS）完成正确的色度重采样、颜色空间转换和模型输入格式生成。对于非 4:2:0 JPEG，驱动中的 `CVI_CAMERA_IOCTL_GET_LATEST_NV12_FRAME` 会拒绝执行不符合输入采样格式的转换。

为了减少解码后大尺寸图像的复制，系统使 JPU 直接将 YUV422P 输出写入 Linux ION 内存分配器提供的物理连续缓冲区。用户态通过 `CameraIonFrameRequest` 向内核传入目标 ION 文件描述符、缓冲区偏移、可用容量、上一帧序号以及请求超时时间。内核函数 `latest_yuv_to_ion()` 根据文件描述符查找对应的 ION 分配对象，检查 `buffer_offset + capacity` 是否超过分配范围，并计算 JPU 可以访问的总线物理地址。完成参数校验后，JPU 以该物理地址作为解码输出基址，依次写入 Y、Cb 和 Cr 三个平面。解码成功后，驱动向用户态返回各平面的偏移量、长度和步长，以及图像宽高、像素格式、帧序号、采集时间戳和 `jpu_decode_us` 等信息。这样，用户态和 VPSS 可以直接根据驱动返回的元数据访问各个平面，不需要重新推导地址或猜测输出布局。

JPU 运行期间，目标 ION 缓冲区的访问所有权属于硬件设备。用户态在 ioctl 返回之前不得读取或修改相应区域，驱动也只有在 JPU 完成解码并执行必要的内存同步后，才将缓冲区重新交还给用户态和后续 VPSS 使用。这种所有权转换是保证 DMA 数据一致性的必要条件，而不只是应用层约定。如果 CPU 与 JPU 同时访问同一目标缓冲区，即使双方使用的物理地址完全正确，仍可能出现图像撕裂、部分平面尚未写完或缓存内容与物理内存不一致等问题。当前链路通过同步 ioctl 语义明确了设备写入和用户态读取的先后关系，从而保证 VPSS 接收到的是一幅完整、稳定的 YUV422P 图像。

需要说明的是，当前链路并非严格意义上的端到端零复制。`latest_yuv_to_ion()` 在取得最新 JPEG 后，会将几十 KB 的压缩码流复制到临时 `Vec<u8>` 中，然后释放 `LatestFrameStore` 的状态锁，再取得 JPU 状态锁并提交硬件解码。这次压缩数据复制是为了缩短最新帧存储锁的持有时间，同时避免摄像头状态锁和 JPU 状态锁以不同顺序嵌套而形成 ABBA 死锁。如果在整个 JPU 解码期间持续持有最新帧锁，后台 UVC 线程将无法发布下一帧，异步采集与推理重叠也会失去意义。当前实现真正消除的是大尺寸未压缩图像复制：614,400 字节的 YUV422P 图像由 JPU 直接写入源 ION 缓冲区，VPSS 直接读取该物理缓冲区；VPSS 生成的 RGB planar 图像也直接写入目标 ION 缓冲区，再由 TPU 使用。因此，更准确的技术表述是“大图像数据通路零拷贝或低拷贝”，而不是整个处理链路完全不存在数据复制。

总体而言，该项优化围绕 JPU 建立了一条包含输入校验、物理缓冲区输出、DMA 所有权管理、ABI 校验、位流边界保护、异常帧丢弃、有界重试和硬件复位的完整解码路径，覆盖范围超过单纯的软硬件解码替换。JPU 只负责按照 JPEG 原始采样格式重建像素，不负责将任意 4:2:2 图像直接转换成 NV12 或模型所需的 RGB 格式；损坏的 JPEG 也只能被识别、丢弃和恢复，无法由驱动补回已经缺失的 USB 数据。后续若继续优化，应优先研究压缩码流缓冲区的安全共享和跨设备缓冲区所有权协议，并保持像素格式标注与真实数据布局一致。

5.1.4　ION 共享物理缓冲区

在原始视觉处理链路中，不同硬件模块通常分别维护自己的输入和输出缓冲区。图像从一个模块传递到另一个模块时，需要先复制到普通用户态内存，再复制到下一个设备能够访问的缓冲区。对于 640×480 的 YUV422P 图像，单帧数据量为 614,400 字节；对于 640×640 的 RGB planar 图像，单帧数据量达到 1,228,800 字节。如果 JPU、视频处理子系统（Video Processing Subsystem，VPSS）和张量处理器（Tensor Processing Unit，TPU）之间均通过中央处理器（Central Processing Unit，CPU）复制整幅图像，单帧将产生数 MB 的额外内存读写。这些复制不仅直接占用单核 C906 的执行时间，还会与硬件加速器竞争双倍数据速率同步动态随机存取存储器（Double Data Rate Synchronous Dynamic Random-Access Memory，DDR SDRAM）的带宽，进而间接延长 JPU、VPSS 和 TPU 的硬件执行时间。



图 16: 基于 ION 的跨设备共享缓冲区布局

为减少大尺寸图像在不同设备之间的重复复制，系统建立了两级 ION 共享物理缓冲区。ION 是内存分配器的专有名称，并非英文首字母缩写。其中，source ION 作为 JPU 的输出缓冲区和 VPSS 的输入缓冲区，destination ION 作为 VPSS 的输出缓冲区和 TPU 的输入缓冲区。各硬件模块通过同一缓冲区的物理地址直接衔接，用户态主要负责传递 ION 文件描述符、缓冲区偏移、有效长度、平面步长、图像格式和时间戳等元数据，不再通过普通 `Vec<u8>` 在模块之间复制完整的 YUV 或 RGB 图像。由此，图像数据的传递方式从“CPU 读取并重新写入”转变为“硬件在共享物理内存上按顺序交接”.用户态在 `VpssRgbPipeline::open()` 中为 source ION 分配 `SOURCE_CAPACITY=2×1024×1024` 字节的物理缓冲区，并使用 `akars-jpu` 作为分配标签。该容量明显大于当前 640×480 YUV422P 图像所需的 614,400 字节，用于容纳内存页对齐、硬件平面对齐、不同 JPEG 采样格式以及未来可能扩展的输入分辨率，同时为内核的偏移和容量检查提供明确上限。当前图像在 source ION 中的有效布局由 JPU 驱动返回：Y 平面起始于 `source_paddr + y_offset`，分辨率为 640×480、步长为 640 字节；Cb 平面起始于 `source_paddr + cb_offset`，分辨率为 320×480、步长为 320 字节；Cr 平面起始于 `source_paddr + cr_offset`，其尺寸和步长与 Cb 平面相同。各平面的偏移、长度和步长均由摄像头/JPU ioctl 明确返回，而不是由用户程序根据固定公式硬编码。即使驱动以后改变平面对齐方式，用户态也可以继续依据应用二进制接口（Application Binary Interface，ABI）提供的元数据构造 VPSS 任务。destination ION 的容量根据模型输入尺寸计算，即 `output_width × output_height × 3`，并向 4 KiB 内存页边界对齐。对于 640×640 模型输入，有效图像载荷为 1,228,800 字节。VPSS 按照 RGB planar 格式依次写入红、绿、蓝三个颜色平面：R 平面从偏移 0 开始，G 平面从偏移 `640×640` 开始，B 平面从偏移 `2×640×640` 开始，每个平面的行步长均为 640 字节。该内存布局与模型要求的批次、通道、高度、宽度（Number, Channel, Height, Width，NCHW）张量布局一致，因此不再需要 CPU 执行通道交织转平面、

通道顺序调整或逐像素重排。VPSS 完成写入后，TPU 运行时可以直接使用该缓冲区作为模型输入数据来源。

ION 缓冲区由 `IonAllocation::allocate()` 统一申请。分配过程使用直接存储器访问（Direct Memory Access，DMA）一致性内存堆，同时取得 ION 文件描述符、实际分配容量和设备可访问的物理地址。用户态随后通过 `mmap()` 建立共享虚拟地址映射，并设置 `PROT_READ | PROT_WRITE` 和 `MAP_SHARED` 属性，使用户态在确有需要时能够访问同一批物理页。`IonAllocation` 对象析构时负责执行 `munmap()` 并关闭文件描述符，从而保证正常退出和异常返回路径均不会遗留映射关系或长期占用物理内存。文件描述符、用户虚拟地址和设备物理地址分别服务于对象管理、CPU 访问和硬件访问，但三者对应的是同一个 ION 分配对象。SG2002 的部分多媒体设备和 TPU 寄存器只能接收 32 位物理地址，因此共享缓冲区必须位于低 4 GiB 物理地址范围内。如果分配器返回超过 `0xFFFF_FFFF` 的物理地址，而驱动在写入设备寄存器时直接截断高 32 位，硬件将访问错误的内存区域。这类错误可能表现为静默数据损坏、总线访问异常、设备超时或无法稳定复现的推理错误。为此，内核 ION 路径使用 `alloc_coherent_pages_dma32()` 分配低地址物理页，从分配阶段保证缓冲区满足 DMA32 约束。设备提交路径还会检查物理基址与有效长度相加后是否溢出 32 位可寻址空间。该约束不能通过提交时简单地把 64 位地址强制转换为 32 位来替代，而必须保证内存分配器、驱动 ABI 和硬件地址寄存器宽度保持一致。

共享物理缓冲区并不意味着多个处理单元可以在任意时刻同时访问同一片内存。低拷贝链路必须配合明确的 DMA 数据所有权状态机。source ION 初始由用户态管理，提交 JPU 任务后转为 JPU 独占写入状态；JPU 完成解码并产生完成中断后，缓冲区所有权转交给 VPSS，VPSS 在该阶段只读取 Y、Cb 和 Cr 平面；VPSS 完成读取后，source ION 才能重新用于下一幅图像。destination ION 同样从空闲状态开始，提交 VPSS 任务后由 VPSS 独占写入；VPSS 完成后，缓冲区转交给 TPU 或张量直接存储器访问（Tensor Direct Memory Access，TDMA）单元读取；只有 TPU 前向推理结束后，destination ION 才能被下一帧覆盖或重新使用。通过 source ION 和 destination ION 的两级共享，系统消除了 YUV422P 解码图像与 RGB planar 预处理图像在设备之间的 CPU 整图复制。JPU 将 614,400 字节的 YUV422P 结果直接写入 source ION，VPSS 直接读取该缓冲区；VPSS 再将 1,228,800 字节的 RGB planar 结果写入 destination ION，TPU 通过物理地址或运行时导入机制读取该结果。CPU 不再承担大图像搬运，只需设置设备参数、传递缓冲区描述信息并维护所有权状态。这不仅减少了 CPU 执行时间，也降低了 CPU 对 DDR 的额外访问，为 JPU、VPSS 和 TPU 保留了更多有效内存带宽。

总体而言，ION 优化建立了统一的跨设备缓冲区表示方式，并围绕该缓冲区实现地址范围校验、DMA32 约束、映射属性管理、ABI 元数据传递和设备所有权同步，其作用远超物理连续内存分配本身。它使 JPU、VPSS 和 TPU 能够在同一条物理数据通路上顺序处理图像，避免 CPU 成为大图像传递的中间节点。该机制既是降低单帧视觉时延的重要基础，也是后续实现多缓冲流水线和设备间异步并行的前提。

5.1.5　扩展 VPSS 驱动，直接接收 JPU 输出的 YUV422P

项目早期的视频处理子系统（Video Processing Subsystem，VPSS）驱动主要围绕 NV12 输入格式设计。原有源帧抽象只包含亮度 Y 平面和交织 UV 平面两个物理地址，适用于 4:2:0 半平面图像。然而，当前摄像头输出的运动联合图像专家组（Motion JPEG，MJPEG）采用 4:2:2 色度采

样，经过 JPEG 处理单元（JPEG Processing Unit，JPU）解码后得到的是 Y、Cb、Cr 三个平面相互独立的 YUV422P 图像。二者在平面数量、色度采样比例和内存布局上均不兼容，因而在 JPU 输出与 VPSS 输入之间形成了格式断点。如果继续使用原有 NV12 接口，只能将 YUV422P 错误标记为 NV12、由中央处理器（Central Processing Unit，CPU）执行一次 YUV422P 到 NV12 的全帧转换，或者放弃 VPSS 并继续使用 CPU 完成颜色转换和缩放。第一种方案会产生错误图像，后两种方案则会引入整图复制和软件预处理开销。



图 17: JPU 输出与 VPSS 输入格式衔接

为消除这一格式断点，内核 VPSS 驱动新增了 `Yuv422PlanarFrame` 数据结构和 `SourceFrame::Yuv422Planar` 源帧类型，同时使用 `RgbPlanarFrame` 和 `DestinationFrame` 描述模型输入端的三平面 RGB 目标图像。用户态与内核态共同定义 `VpssRunYuv422pRgb` 请求结构，并通过 `CVI_VPSS_IOCTL_RUN_YUV422P_RGB` 提交离线硬件任务。该接口直接表达“JPU 输出的 YUV422P 三平面图像经过 VPSS 转换为 RGB planar 三平面图像”的数据契约，不再依赖驱动根据缓冲区数量或格式编号进行隐式推断。由此，JPU 输出可以直接作为 VPSS 输入，不需要在两者之间插入 CPU 颜色格式转换。

YUV422P 输入需要按照 CV181x/SG2002 官方视频缩放器的格式定义配置。驱动将输入格式设置为 `SCL_FMT_YUV422`，为 Y、Cb 和 Cr 分别配置独立的物理基址，并按照 4:2:2 采样关系设置颜色分量。亮度分量采用 8 位采样，水平方向的色度下采样因子为 2，垂直方向的色度下采样因子为 1。对于 640×480 输入图像，Y 平面的行步长为 640 字节，Cb 和 Cr 平面的行步长均为 320 字节；由于 4:2:2 只在水平方向降低色度分辨率，两个色度平面的高度仍然是 480 行。驱动必须使用 JPU 接口实际返回的 `stride_y` 和 `stride_c`，不能假设三个平面的步长都等于图像宽度。格式扩展的关键并不只是增加第三个物理地址字段，还包括采样因子、平面顺序、步长和颜色空间转换参数的一致配置。如果把垂直色度采样因子错误设置为 2，VPSS 会把 4:2:2 输入按照 4:2:0 读取，只处理一半的色度行，最终产生垂直方向的颜色错位；如果交换 Cb 和 Cr 地址，则会产生明显的红蓝通道反转；如果将色度步长错误设置为 640，硬件又会跨越正确的色度行边界。为了保证图像内容正确，驱动需要同时配置输入格式、三个物理地址、两个步长、水平和垂直采样关系，以及与输入颜色范围匹配的颜色空间转换矩阵。

用户态和内核态之间通过固定的应用二进制接口（Application Binary Interface，ABI）描述一次完整 VPSS 任务。当前 `VpssRunYuv422pRgb` 的 ABI 版本为 1，结构体大小固定为 248 字节。请求中包含源、目标 ION 文件描述符，以及 Y、Cb、Cr 三个输入平面的偏移；同时包含源图像宽高、亮度步长和色度步长。几何变换部分描述源裁剪区域、目标内容区域以及完整输出画布的宽高；输出部分描述 R、G、B 三个平面的偏移和步长；控制部分包含边框颜色、任务超时、输入

帧序号和时间戳；返回部分则包含执行状态、中断计数、排队时间、启动时间、完成时间、硬件执行时间、输出序号、输出时间戳，以及 IMG、AXI、SCALER 和输出直接存储器访问（Output Direct Memory Access，ODMA）等硬件模块的状态寄存器快照。将这些字段集中在一次 ioctl 请求中的目的，是让一个离线 VPSS 任务完整描述输入格式、颜色转换、裁剪、缩放、letterbox 填充和输出布局。如果将上述处理拆分为多个驱动接口，就需要多次系统调用、多次硬件任务配置以及额外的中间缓冲区；每个中间阶段还会增加新的设备同步点和缓冲区所有权转换。当前设计使 VPSS 能够在一次硬件调度中完成模型所需的全部图像预处理，既缩短控制路径，也避免生成用户态不需要的中间图像。

VPSS 属于直接存储器访问（Direct Memory Access，DMA）主设备，因此在写入硬件寄存器之前，驱动必须严格验证请求中的缓冲区和几何参数。内核首先检查 ABI 版本和结构体大小是否匹配，再通过源、目标 ION 文件描述符解析受信任的物理内存对象。随后验证 Y、Cb、Cr 三个源平面以及 R、G、B 三个目标平面的地址范围是否完全位于对应 ION 对象内，检查各平面的偏移、长度和物理地址计算是否发生整数溢出，并确认宽度、高度和步长符合硬件约束。驱动还需要保证源裁剪区域没有超出 640×480 输入图像，目标内容区域没有超出输出画布，RGB 三个输出平面之间不存在非法重叠，并且目标缓冲区容量足以容纳完整输出。这些边界检查虽然位于性能关键路径中，但不能为了减少少量软件开销而省略。错误的物理地址、步长或平面长度可能使 VPSS 越过合法缓冲区，通过 DMA 覆盖内核数据、其他进程内存或其他设备正在使用的物理页面。项目的优化原则是减少整图复制、缩短任务层级和合并硬件处理步骤，而不是绕过内存安全检查。与硬件执行时间相比，这些整数范围和布局校验的耗时很小，却是保证长期稳定运行和驱动安全性的必要条件。

完成参数校验后，VPSS 在一次离线硬件任务中完成模型输入预处理。硬件首先从 source ION 读取 Y、Cb 和 Cr 三个平面，并使用 BT.601 有限范围颜色转换矩阵，将 YCbCr 数据转换到 RGB 颜色域；随后按照目标内容区域执行比例缩放，将转换后的图像放置到完整输出画布中；未被有效图像覆盖的区域使用请求指定的边框颜色填充；最后由 ODMA 按照 R、G、B 三个独立平面的形式，将结果写入 destination ION。整个过程中不需要 CPU 逐像素执行 YUV 到 RGB 转换、双线性缩放、边框填充或通道重排。

总体而言，该项优化完成了 JPU 与 VPSS 之间的原生格式衔接，使 YUV422P 三平面图像能够通过 ION 物理缓冲区直接进入 VPSS，并在一次硬件任务中完成颜色空间转换、letterbox、缩放和 RGB planar 输出。它不仅扩展了一个新的像素格式枚举，还系统解决了三平面地址表达、色度采样配置、步长管理、ABI 定义、DMA 边界校验、输出通道验证、设备生命周期管理和工作模式裁剪等问题。最终形成了“JPU YUV422P—VPSS RGB planar—TPU NCHW 输入”的完整硬件预处理链路，消除了原有格式断点以及 CPU 全帧颜色转换和布局重排开销。

5.1.6　修复 VPSS 完成后的调度长尾

早期测试中，视频处理子系统（Video Processing Subsystem，VPSS）的单次调用墙钟时间为 44～47 ms，因此一度被认为是颜色空间转换和缩放效率较低。加入内核分段时间戳后发现，VPSS 硬件从任务启动到完成中断实际只需要 1.4～1.7 ms，其余数十毫秒主要发生在硬件完成之后：中断处理程序已经确认任务结束，但提交任务的用户线程没有及时恢复运行。如果只测量 ioctl 入口和出口，调度等待就会被错误地统计为 VPSS 硬件执行时间。为区分排队、硬件执行和完成返回三

个阶段，VPSS 应用二进制接口（Application Binary Interface，ABI）增加了 `queue_enter_ns`、`hardware_start_ns`、`hardware_done_ns` 和 `elapsed_ns` 等时间戳。用户态通过 `split_vpss_timing()` 计算：

```
queue_us       = hardware_start_ns - queue_enter_ns
hardware_us    = hardware_done_ns  - hardware_start_ns
wake_return_us = ioctl_wall_us - queue_us - hardware_us
```

其中，`queue_us` 表示任务从进入驱动到真正启动硬件的等待时间，`hardware_us` 表示 VPSS 实际执行时间，`wake_return_us` 表示硬件完成后等待任务恢复并返回用户态的时间。如果旧内核不支持详细时间戳，或者时间戳不满足单调递增关系，程序会回退到 `elapsed_ns`，并将该样本标记为不可分段，避免产生负数或失真的统计结果。

该问题的根因是单核 StarryOS 中的中断唤醒与任务调度之间存在较长延迟。VPSS 运行期间，提交线程进入等待队列；硬件完成后，中断请求（Interrupt Request，IRQ）处理程序记录完成状态并唤醒等待者。但是，普通唤醒仅将线程重新放入就绪队列，并不保证其立即获得 CPU。当前任务或其他后台线程仍可能继续使用剩余时间片，导致 VPSS 已经完成，而 ioctl 调用线程仍要等待下一个调度周期。由此可见，硬件完成中断的到达并不等于用户线程已经恢复执行。

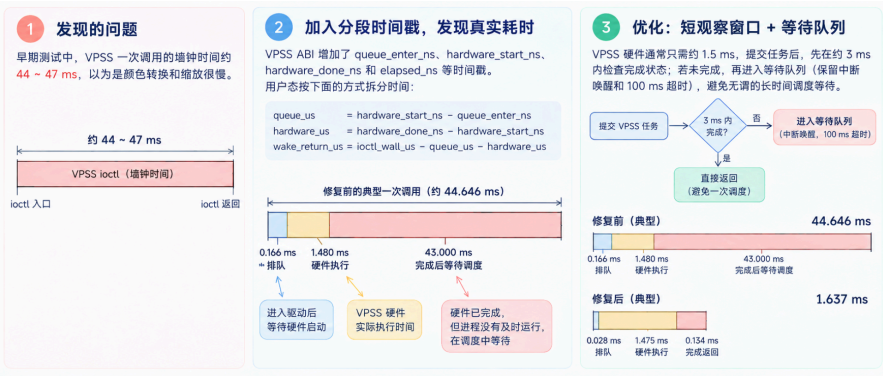


图 18: VPSS 硬件完成与线程恢复时间线

针对正常 VPSS 任务硬件时间为 1.5 ms 的特点，驱动采用“短观察窗口加等待队列”的两阶段完成机制。任务提交后的最初 3 ms 内，调用线程对软件完成状态进行有界观察；如果 IRQ 在该窗口内发布完成状态，线程可以直接执行收尾并返回，从而避免一次长时间片级的休眠与重新调度。如果 3 ms 内任务仍未完成，线程就转入等待队列，并保留中断唤醒和 100 ms 绝对超时，防止硬件异常时无限占用 CPU。该机制采用有界观察；3 ms 窗口根据 VPSS 正常执行时间设定，超过窗口后自动进入休眠，因此能够在常规任务中降低返回时延，在异常任务或较长任务中保持系统公平性。观察循环只检查由 IRQ 路径发布的软件完成标志，不直接读取并清除硬件的写一清零（Write One to Clear，W1C）中断状态。中断状态寄存器仍由 IRQ 处理程序统一清理，避免任务线程与中断处理程序竞争清除同一事件而造成完成中断丢失。

修复前，典型 VPSS 墙钟时间为 44.646 ms，其中硬件执行 1.480 ms，完成后的调度等待接近 43 ms。修复后，典型墙钟时间降至 1.637 ms，可分解为 0.028 ms 排队、1.475 ms 硬件执行和 0.134 ms 完成返回，整体下降 96.3%。最终正式测试中，StarryOS 的 VPSS 平均时延为 1.643 ms，与 Linux 基线的 1.865 ms 处于同一水平，说明软件调度长尾已基本消除。

该优化表明，设备调用的墙钟时延不等同于硬件执行时延。分析硬件加速器性能时，应分别记录任务入队、硬件启动、硬件完成、中断处理、任务恢复和用户态返回等阶段。3 ms 短观察窗口适用于当前低并发、固定分辨率的专用场景；如果未来引入更高分辨率或多路 VPSS 任务，硬件时间可能超过该窗口，此时功能仍会通过等待队列正常完成，但窗口参数和调度策略需要重新评估，不能通过无限延长轮询时间来追求低延迟。

5.1.7　模型输入契约校验与正确的 TPU 输入导入

张量处理器（Tensor Processing Unit，TPU）的模型输入不能被简单理解为一段长度相同的字节数据。输入张量的形状、元素类型、颜色通道顺序、内存布局、量化参数、行步长、对齐方式和设备内存类型共同构成模型输入契约。即使缓冲区长度同为 `3×640×640` 字节，如果将蓝绿红（Blue, Green, Red，BGR）误作红绿蓝（Red, Green, Blue，RGB），将高度—宽度—通道（Height–Width–Channel，HWC）误作通道—高度—宽度（Channel–Height–Width，CHW），或者忽略运行时要求的对齐步长，TPU 仍可能正常完成前向计算，但输出结果会稳定地偏离正确值。因此，模型输入格式需要像硬件寄存器接口一样被严格校验。

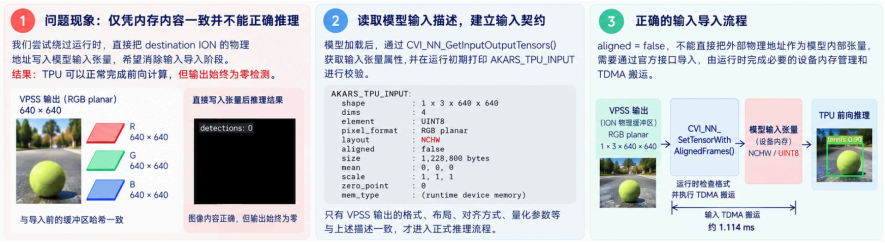


图 19: 模型输入张量契约校验流程

模型加载后，`YoloModel::open()`依次调用 `CVI_NN_RegisterModel()`和 `CVI_NN_GetInputOutputTensors()`，获取模型的输入、输出张量描述。程序随后输出 `AKARS_TPU_INPUT` 诊断信息，包括张量形状、维数、元素格式、元素数量、内存大小、物理地址、内存类型、像素格式、对齐标志、量化比例、零点、均值和缩放系数。只有这些属性与视频处理子系统（Video Processing Subsystem，VPSS）的输出布局一致时，程序才允许进入正式推理流程。该检查可以在运行初期发现模型版本、用户态程序和 VPSS 输出格式之间的不匹配，避免把输入契约错误误判为模型精度问题。

当前正式 640×640 模型的输入契约为：

```
shape        = 1×3×640×640
element      = UINT8
pixel format = RGB planar
layout       = NCHW
aligned      = false
mean         = 0, 0, 0
scale        = 1, 1, 1
zero point   = 0
```

该模型直接接收 8 位无符号整数（Unsigned 8-bit Integer，UINT8）格式的 RGB 三平面数据，不要求 CPU 执行浮点归一化、均值扣除或额外的量化变换。VPSS 因此可以直接生成 `1×3×640×640` 的批次、通道、高度、宽度（Number, Channel, Height, Width，NCHW）输入。但是，`aligned=false` 意味着不能仅凭缓冲区大小一致，就假设运行时允许将任意外部物理地址直接作为模型内部输入

张量。正式路径使用 `CVI_NN_SetTensorWithAlignedFrames()`，由官方运行时按照模型描述导入 VPSS 输出，并通过张量直接存储器访问（Tensor Direct Memory Access，TDMA）完成必要的数据搬运和内部布局准备。

5.1.8　TPU ioctl ABI、ION 地址与完成语义修复

CVITEK 神经网络运行时 `libcviruntime.so` 并不直接控制全部张量处理器（Tensor Processing Unit，TPU）寄存器，而是通过 `/dev/cvi-tpu0` 的输入输出控制（Input/Output Control，ioctl）接口提交命令缓冲区、等待直接存储器访问缓冲区（DMA Buffer，dma-buf）任务完成并查询执行状态。因此，StarryOS 驱动必须准确兼容 Linux/CVITEK 接口的参数结构、字段大小、输入输出方向、文件描述符语义和返回值规则。此类应用二进制接口（Application Binary Interface，ABI）错误通常不会造成编译失败，而可能表现为运行时重复等待、进入兼容路径、产生额外调度开销，甚至出现偶发超时或静默推理错误。

TPU 驱动不再将用户态传入的整数直接解释为物理地址，而是将其作为 ION 或 dma-buf 文件描述符，从内核维护的缓冲区对象中解析真实物理基址。驱动随后检查命令区域和数据区域是否落在合法分配范围内，验证偏移、长度和地址运算不存在越界或整数溢出，并确认最终设备地址位于 TPU 能够访问的低 4 GiB 物理地址范围。用户态参数则通过受控的虚拟内存指针封装进行读取和写回，避免内核直接解引用不可信用户地址。

该修改统一了视频处理子系统（Video Processing Subsystem，VPSS）与 TPU 的内存来源。VPSS 输出的 destination ION 可以继续作为 TPU 输入导入的基础物理对象，不需要先复制到另一个普通用户态缓冲区或独立的 TPU 专用缓冲区。官方运行时仍可根据模型要求执行张量直接存储器访问（Tensor Direct Memory Access，TDMA），但 VPSS 和 TPU 所引用的是同一套受内核管理的 ION 分配对象，从而维持跨设备共享内存链路。兼容性修复中最关键的问题之一是 `CVITPU_WAIT_DMABUF` 的返回语义。ioctl 系统调用的返回值应表示驱动是否成功解析请求并完成等待操作，而 TPU 硬件任务的执行结果应写入用户提供的 `wait_arg.ret` 字段。如果驱动直接将硬件状态作为 ioctl 返回值，官方运行时可能把已完成的任务误判为系统调用失败，进而重复进入等待或错误恢复流程。修复后，只要参数解析和等待过程本身正常完成，ioctl 就返回 0；任务成功、设备错误或超时状态则通过 `wait_arg.ret` 返回。该语义与官方运行时保持一致，消除了由 ABI 误解引起的重复等待。为保证提交任务与完成事件正确对应，驱动使用提交线程标识和单调递增的任务序号建立任务身份，并将已完成任务写入 `DONE_LIST`。等待方只能取得属于自身请求的完成项，防止多个提交者或运行时内部并行操作错误消费彼此的完成事件。虽然当前机器人视觉程序主要以串行方式执行 TPU 推理，但正确的任务关联仍是驱动 ABI 完整性和未来支持并发任务的基础。

TPU 完成中断请求（Interrupt Request，IRQ）可能同时携带正常完成、TDMA 错误或张量指令单元（Tensor Instruction Unit，TIU）错误状态。驱动不能因为收到完成中断就统一报告成功，也不能在清除中断寄存器后丢失原始错误位。当前中断路径会先读取并保存相关状态寄存器，再发布任务完成状态并唤醒等待者，最后按照硬件要求清理中断。这样，设备错误能够随完成项返回给用户态运行时，而不会被普通完成状态覆盖。当前正式路径启用了 `TPU_DIRECT_EXECUTION=true`，采用同步直通执行方式，使提交线程直接完成命令提交、硬件等待和结果返回，减少提交线程、后台 worker 和完成等待线程之间的调度交接。旧的异步 worker 路

径仍被保留，作为 A/B 性能比较、兼容性验证和未来队列化执行的回退方案。两条路径共用相同的用户态 ABI，因此可以在不修改 `libcviruntime.so` 和应用程序的情况下切换执行架构。

总体而言，该项修复统一了用户态运行时、TPU 驱动和 ION 内存管理之间的接口语义，覆盖范围超过单个 ioctl 返回值。通过规范文件描述符解析、物理地址验证、任务身份管理、完成结果返回和错误状态保存，StarryOS 的 TPU 驱动能够按照官方运行时预期完成命令提交与等待，也为后续消除 worker 调度开销和优化 IRQ 完成路径提供了可靠基础。未来若任务需要队列化，可重新启用 worker，而不必重写运行时接口。

5.1.9　TPU 任务分类与四段时间戳计时机制

如果仅记录 `CVI_NN_Forward()` 的总墙钟时间无法准确定位张量处理器（Tensor Processing Unit，TPU）性能问题。一次调用可能同时包含输入张量搬运、任务排队、命令寄存器配置、张量直接存储器访问（Tensor Direct Memory Access，TDMA）执行、张量指令单元（Tensor Instruction Unit，TIU）计算、中断响应、等待线程恢复和用户态返回等多个阶段。因此，即使测得一次调用耗时 45 ms，也无法仅凭总时间判断瓶颈来自模型计算、内存访问还是操作系统调度。为提高统计的可解释性，驱动根据命令缓冲区头部的描述符数量对任务进行分类。当 `bd_desc_count=0` 且 `tdma_desc_count>0` 时，将任务识别为 `tdma_only`，主要对应模型输入数据导入；当 `bd_desc_count>0` 时，将任务识别为 `compute`，表示任务包含 TIU 计算及相关内部数据搬运；其余无法可靠识别的任务归入 `unknown`。通过这种分类，短时延的输入 TDMA 任务不会与长时延的模型计算任务混合统计。

每个 TPU 任务至少记录四个核心时间区间：

```
submit_to_run  = 驱动收到提交请求到任务实际开始执行
fire_to_irq    = 最终启动寄存器写入到硬件完成IRQ到达
irq_to_resume  = IRQ发布完成状态到等待线程恢复运行
done_to_user   = 内核完成收尾到运行时调用返回用户态
```

此外，驱动使用 `other` 记录寄存器准备、状态读取、任务关联和统计更新等无法归入上述区间的少量开销。`fire_to_irq` 的起点经过专门校准，设置在最终 `TDMA_CTRL` 启动寄存器写入之后，排除了更早的描述符地址配置、时钟控制或中断屏蔽操作。这样，该指标才能准确表示 TPU 命令流从真正启动到产生完成中断的硬件执行区间。内核通过 `TASK_TIMING` 分别输出 `all`、`tdma_only` 和 `compute` 三类任务的调用次数、累计时间、平均时间和最大时间。用户态则把 `CVI_NN_SetTensorWithAlignedFrames()` 的墙钟时间记为“TPU 输入 TDMA 搬运”，把 `CVI_NN_Forward()` 记为“TPU 推理”。结合用户态应用程序接口边界和内核任务时间线，可以判断额外时延产生在运行时、系统调用边界还是驱动内部。例如，若内核 TDMA-only 任务为 1 ms，而用户态记录为 3 ms，则其余 2 ms 主要位于运行时封装和系统调用前后；如果 `fire_to_irq` 只有 0.8 ms，但内核任务总时间达到 45 ms，则瓶颈位于排队或任务恢复，与 TPU 硬件及 DDR 带宽无关。

为避免性能观测本身影响测试结果，分段计时使用单调高分辨率时钟，每个任务只保存固定数量的整数时间戳，不在热路径中逐条打印描述符信息。逐帧 TPU 调试输出可以通过运行参数关闭，但累计器仍会在任务结束后输出平均值、最大值和分位数。这样既保留了驱动可观测性，也避免串口日志产生的额外开销被错误计入后处理或其他视觉时延。

当前正式模型的 compute 任务内核平均时间为 40.387 ms，其中 submit_to_run 为 0.002 ms、fire_to_irq 为 39.361 ms、irq_to_resume 为 0.874 ms、done_to_user 为 0.131 ms，其余开销为 0.018 ms。fire_to_irq 占内核 compute 时间的 97.46%，说明早期 worker 排队和任务恢复造成的软件长尾已经基本消除。当前 39～41 ms 的 TPU 推理时间主要来自模型命令流内部的 TIU 计算、TDMA 搬运和存储系统访问，驱动排队与调度延迟已经降至次要部分。

5.1.10　TPU 执行链路重构与调度开销消除

早期 TPU 驱动采用异步 worker 架构。用户线程提交任务后先唤醒 TPU worker，由 worker 配置硬件并等待完成中断；中断请求（Interrupt Request, IRQ）到达后再次唤醒 worker，worker 完成状态处理后再通知原始提交线程。一次串行任务因此包含两次额外的线程调度交接。

阶段	异步 worker 路径	同步直通路径
任务提交	用户线程写入队列	用户线程直接取得 TPU_RUN_LOCK
硬件配置	唤醒 worker 后由 worker 执行	原提交线程直接执行
硬件等待	worker 阻塞等待 IRQ	原提交线程阻塞等待 IRQ
完成处理	IRQ 唤醒 worker	IRQ 唤醒原提交线程
返回用户态	worker 再通知提交线程	原提交线程直接返回
额外调度交接	至少两次	无中间 worker 交接

在多核系统中，worker 可以与提交线程并行运行；但在单核 C906 上，每次任务交接都可能错过当前调度时间片。早期测试中，submit_to_worker 约为 20.899 ms，硬件执行区间约为 20.302 ms，完成后返回用户态约为 5.229 ms。虽然该阶段的硬件统计仍混合了输入 TDMA 与 compute 任务，但已经能够确认软件排队与硬件执行处于同一量级。



图 20: TPU 异步 worker 路径的调度交接

当前正式路径启用同步直通执行，由原始提交线程在内核中完成完整任务生命周期：

```
取得TPU_RUN_LOCK
        ↓
解析命令与ION地址
        ↓
配置时钟、复位和设备寄存器
        ↓
写入最终启动寄存器
        ↓
提交线程阻塞等待IRQ
        ↓
IRQ发布完成状态并唤醒提交线程
        ↓
检查状态、生成完成项并返回用户态
```

提交线程首先取得可睡眠的全局 TPU_RUN_LOCK，确保同一时刻只有一个命令流占用 TPU。随后完成任务分类、地址验证、寄存器配置和硬件启动。硬件执行期间，提交线程在 IRQ 等待点休眠并让出 CPU；IRQ 到达后直接唤醒同一个提交线程，由其完成状态检查并返回用户态。任务上下文不再转交给中间 worker。



图 21: TPU 同步直通执行路径

TPU_RUN_LOCK 必须使用可睡眠锁，不能在持有自旋锁时等待硬件。如果跨越硬件执行阶段持有自旋锁，可能长期禁止调度，并导致单核系统死锁。同步直通仍然采用阻塞式 IRQ 等待，而不是持续忙轮询；其优化对象是任务所有权交接和调度层级。

静态 A/B 测试结果如下：

性能指标	worker 路径	同步直通路径	优化效果
输入阶段	52.830 ms	7.740 ms	降低 85.35%
submit_to_run	22.873 ms	0.002 ms	基本消除
done_to_user	2.958 ms	0.116 ms	降低 96.08%
视觉总时延	100.207 ms	72.042 ms	降低 28.11%
处理帧率	9.96 FPS	13.86 FPS	提高 39.16%

TPU 时钟、模型结构和硬件命令流在两组测试中没有改变，因此性能提升来源于 worker 排队及线程交接的消除，与硬件频率增加无关。驱动仍保留 TPU_DIRECT_EXECUTION 开关和原异步 worker 实现，正式版本将其设置为 true。保留旧路径便于进行 A/B 性能对照，也为未来多进程共享 TPU 或批量任务调度提供兼容回退，而不需要修改现有用户态应用二进制接口（Application Binary Interface，ABI）。

同步直通适用于当前单进程、单模型、单帧串行的机器人视觉负载。长时间 compute 任务持有 TPU_RUN_LOCK 时，其他提交者必须等待，因此该方案不适合直接替代通用多任务调度器。如果未来需要同时运行视觉、语音或其他 TPU 模型，应重新引入公平队列、任务优先级或时间片机制。当前设计属于针对确定性工作负载的执行路径裁剪，主要目标是降低单帧响应时延。

5.1.11　面向短时 TDMA 的中断唤醒与竞态管理

VPSS 完成返回路径优化后，其平均时延由 45 ms 量级下降至 1.6 ms，但端到端日志中的张量直接存储器访问（Tensor Direct Memory Access，TDMA）输入阶段一度上升至 40～45 ms。前级调度瓶颈消除后，后一级原有的中断唤醒长尾变得可见，VPSS 的工作量并未转移给 TDMA。

内核分段时间戳明确显示，问题不在 TDMA 硬件本身：

TDMA 阶段	优化前典型时间	含义
`fire_to_irq`	0.838 ms	TDMA 硬件执行正常
`irq_to_resume`	44.283 ms	中断后等待线程未及时恢复
内核任务总时间	45.253 ms	主要由调度等待构成

StarryOS 常规 `WaitQueue::notify_one()` 按照普通策略将等待任务加入就绪队列。在时间片轮转（Round-Robin，RR）调度下，刚被中断唤醒的 TPU 提交线程可能被放置在运行队列尾部，需要等待其他任务用完时间片。对于硬件执行时间不足 1 ms 的 TDMA 任务，几十毫秒的恢复等待会完全掩盖硬件性能。

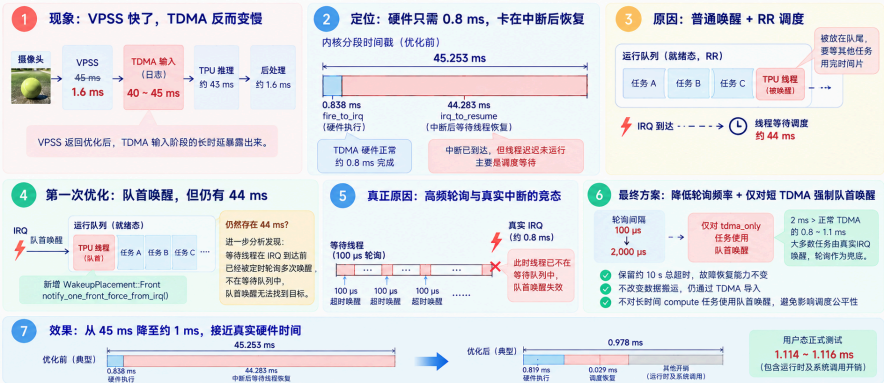


图 22: TDMA 完成中断与短周期轮询竞态

为缩短中断后的恢复路径，调度器新增 `WakeupPlacement::{Normal, Front}`，运行队列支持将被唤醒任务插入队首，等待队列则增加 `notify_one_front_force_from_irq()`。当 TPU 中断处理程序确认当前任务属于 `tdma_only` 时，会将对应提交线程放置到就绪队列前部，使其在中断返回后的下一次调度中优先恢复。第一次引入队首唤醒后，44 ms 量级的长尾仍然存在。进一步分析发现，驱动原先每 100 µs 执行一次超时轮询，而正常 TDMA 在 0.8 ms 后完成。在真实中断到达之前，等待线程已经多次因定时器超时离开 IRQ 等待队列并重新检查状态。当 IRQ 真正到达时，等待队列中可能已不存在对应线程，队首唤醒因而无法找到提升对象。该问题本质上是高频定时轮询与真实硬件中断之间的竞态。为恢复 IRQ 作为正常完成主路径，驱动将 `WAIT_POLL_INTERVAL_US` 从 100 µs 调整为 2,000 µs，同时保持 10 s 的总超时。2 ms 大于正常 TDMA 的 0.8～1.1 ms，因此大多数任务会先由真实 IRQ 唤醒；只有在中断丢失或设备异常时，定时轮询才作为兜底机制检查硬件状态。轮询总次数根据总超时重新计算，因此故障恢复窗口没有缩短。

优化前后的典型时间对比如下：

TDMA 阶段	优化前	优化后	结果
`fire_to_irq`	0.838 ms	0.819 ms	硬件时间基本不变
`irq_to_resume`	44.283 ms	0.029 ms	调度长尾基本消除
内核任务总时间	45.253 ms	0.978 ms	恢复到真实硬件量级
用户态正式测试	45.589 ms	1.114～1.116 ms	包含运行时及系统调用开销

“消除 TDMA 时延”具体指消除多余的调度等待，统计项和数据搬运均被保留。输入数据仍需通过 TDMA 导入模型要求的存储布局，0.8～1.1 ms 的真实硬件时间依然存在；被消除的是 IRQ 到达后 44 ms 量级的不必要调度等待。强制队首唤醒只用于 `tdma_only` 短任务。此类任务的硬件时间不足 1 ms，对一个完整调度时间片极为敏感；compute 任务本身需要 39 ms 量级，如果所有

TPU 完成中断都无条件插入队首，可能破坏系统调度公平性。当前驱动通过 ACTIVE_TDMA_ONLY 识别短输入任务，对其采用强制前插，而长时间 compute 任务保留较保守的普通唤醒策略。

该优化只改变任务在运行队列中的恢复位置，不改变命令内容、缓冲区数据或硬件完成状态。为避免高频中断造成其他线程饥饿，队首唤醒被严格限制在短 TDMA 任务；2 ms 轮询负责处理中断丢失，10 s 总超时则防止设备异常导致永久阻塞。由此，驱动在保持故障恢复能力的同时，将 TDMA 输入路径恢复到接近真实硬件执行时间的水平。。

5.1.12　TPU 驱动优化

TPU 性能不仅取决于模型和标称频率，还受到时钟门控、时钟源选择、分频配置、复位顺序、中断状态清理以及任务生命周期管理的共同影响。日志中显示“700 MHz”并不能单独证明 TPU 实际运行在目标频率，也不能说明片上互连（Fabric，FAB）和存储系统处于匹配状态。为此，项目参考 SOPHGO/CVITEK 官方 SG2002/CV181x 驱动及本地 ref/TPU_ref 资料，引入 Cv181xTpuSoc 管理层，将芯片级时钟与复位控制从通用 TPU 命令执行逻辑中分离。

SoC 初始化按照官方顺序配置 TPU 与 FAB 时钟门控，并管理 TDMA、TPU 和 TPUSYS 复位信号。相关复位采用低有效语义：驱动先清除控制位以断言复位，再置位解除复位，避免在下级模块仍处于活动状态时直接复位上层系统。共享时钟寄存器采用带中断保护的自旋锁执行读—改—写操作，防止不同执行路径并发修改寄存器时覆盖其他位。驱动通过读取 TPLL、FPLL、MIPIMPLL 和 A0PLL 等锁相环（Phase-Locked Loop，PLL）状态，结合时钟多路选择器、旁路状态和从 1 开始计数的分频系数，计算 TPU 与 FAB 的实际工作频率。当前实机结果如下：

时钟或状态	实测值
TPU 频率	700 MHz
FAB 频率	900 MHz
TPLL 频率	1,400 MHz
FPLL 频率	1,500 MHz
MIPIMPLL 频率	900 MHz
TPU 分频寄存器	0x00020009
是否重新应用性能分频	false

perf_div_applied=false 表明启动固件已经将共享 TPLL 和 TPU 分频配置到目标状态，驱动识别到现有配置正确后没有重复改写。共享 PLL 和电压域可能同时服务其他外设，因此不能仅为提高 TPU 频率而擅自修改，否则可能破坏其他设备的稳定性。

TPU 任务按照 prepare → run → finish 三个阶段管理：

生命周期阶段	主要职责
prepare	打开必要时钟、解除复位、清理遗留状态并确认设备可用
run	写入描述符地址，配置动态时钟管理和中断，最终触发命令流
finish	保存完成及错误状态，清理中断并完成设备收尾

驱动将解析得到的实际 tpu_clk_rate 传递给执行层，用于等待和超时计算，避免继续使用未经硬件验证的固定频率常量。发生超时或错误后，设备也可以依据同一生命周期重新清理状态并恢复运行。

为准确测量硬件命令流，fire_to_irq 的起点被移动到最终 TDMA_CTRL 启动寄存器写入时刻，终点则由真实完成中断记录。早期计时点位于描述符地址或其他准备寄存器写入之前，会把软件配置时间混入硬件执行区间。修正后，fire_to_irq 能够较准确地表示从硬件命令流启动到完成中断产生的实际时间。

当前正式模型的 compute 任务平均 fire_to_irq 约为 39.361 ms，Linux 基线中同一模型的 TPU 推理约为 39.721 ms，两者处于相同量级：

指标	StarryOS	Linux 基线
TPU/FAB 工作频率	700/900 MHz	相同硬件配置
核心硬件执行时间	39.361 ms	39.721 ms
差值	-	0.360 ms

该结果说明 StarryOS 不存在明显的 TPU 降频问题，TPU 硬件算力已经与 Linux 基线基本一致。用户态最终观察到的 1～2 ms 差异更可能来自中断恢复、运行时封装、系统调用边界和测试波动，时钟频率配置错误已被实机数据排除。

补齐官方时钟、复位和任务生命周期没有带来数量级的时延下降，是因为启动固件原本已经将 TPU 配置为 700 MHz、FAB 配置为 900 MHz。该项工作的主要价值在于排除时钟和复位配置疑点，保证设备异常后能够可靠恢复，使 StarryOS 与官方 Linux 驱动具备可比较的寄存器生命周期，并为硬件时间测量建立可信边界。

目前 compute 任务 97% 以上的时间位于 fire_to_irq 区间。该阶段包含模型内部 TIU 计算、权重和特征图 TDMA、片上静态随机存取存储器（Static Random-Access Memory，SRAM）与 DDR 访问以及算子间同步。这部分属于模型命令流的真实硬件成本，无法继续通过提高线程优先级、调整运行队列位置或缩短系统调用消除。若需要进一步降低 39 ms 量级的计算时间，应转向模型裁剪、量化、算子融合、编译器调度、片上内存规划和输入尺寸优化。

5.1.13　TPU 输出直读与轻量化检测后处理

早期检测后处理平均为 14 ms，主要开销可能来自输出张量复制、通用格式转换、为全部候选框创建对象、低置信度候选参与排序和非极大值抑制，以及逐帧调试日志。当前模型只包含“网球”一个类别，通用多类别处理流程中存在较多冗余，因此用户态对输出读取和候选框筛选顺序进行了针对性简化。

get_detections() 首先检查 TPU 输出张量的数据格式。如果输出已经是 32 位浮点数（32-bit Floating Point，FP32），程序通过 CVI_NN_TensorPtr() 取得运行时管理的张量地址，并使用 slice::from_raw_parts() 建立只读切片，直接解析 42,000 个浮点元素，不再复制到新的 Vec<f32>。只有输出为量化格式、脑浮点数（Brain Floating Point，BF16）或其他非 FP32 格式时，才回退到 tensor_to_f32() 执行数据转换。

正式模型的输出形状为 [1,5,8400,1]，即 5 个通道和 8,400 个候选位置。各通道含义如下：

通道	数据含义
0	检测框中心横坐标 `cx`
1	检测框中心纵坐标 `cy`
2	检测框宽度 `w`
3	检测框高度 `h`
4	网球类别置信度

`parse_yolov8_output()`按照通道跨度访问数据，而不是把输出错误地解释为 8,400 组连续五元组。多类别兼容路径会在类别通道中寻找最大分数及类别编号；对于当前单类别模型，该过程简化为一次连续的置信度扫描。

程序采用“置信度阈值优先”的处理顺序，默认阈值为 0.5。候选分数只有超过阈值后，程序才读取坐标、计算角点并创建 `Detection` 对象。由于 8,400 个候选位置中的绝大多数属于背景，该策略能够显著减少动态对象数量、排序规模和后续非极大值抑制（Non-Maximum Suppression，NMS）的比较次数。优化重点是在保持检测标准不变的情况下尽早淘汰无效数据，未通过提高阈值改变检测口径。

通过阈值筛选的候选框按置信度降序排列，NMS 只在相同批次和相同类别之间计算交并比（Intersection over Union，IoU）：

```
intersection =
    max(0, min(r1,r2) − max(l1,l2))
  × max(0, min(b1,b2) − max(t1,t2))

union = area1 + area2 − intersection
IoU   = intersection / union
```

当 IoU 超过默认阈值 0.5 时，程序抑制置信度较低的重复框，最终一次性保留有效结果。实际网球场景通常只有一个主目标，经过置信度过滤后进入 NMS 的候选数量较少，因此当前后处理平均时延已下降到 1.6 ms 量级。

模型使用 640×640 输入，而摄像头原始图像为 640×480，因此检测框还需要执行 letterbox 坐标逆映射。当前几何关系为：

参数	数值
缩放比例 `scale`	1
有效内容宽度 `new_w`	640
有效内容高度 `new_h`	480
水平填充 `pad_x`	0
垂直填充 `pad_y`	80

`correct_yolo_boxes()`从模型坐标中减去填充偏移，再除以缩放比例，将预测框恢复到摄像头坐标系，最后裁剪到`[0,639]×[0,479]`范围。如果省略该步骤，检测框的垂直坐标将产生 80 像素偏差，进一步影响网球中心位置、面积比例、转向决策和抓取条件判断。

优化前后的主要变化如下：

优化项目	原始方式	当前方式
FP32 输出读取	复制或通用转换	直接建立只读切片
候选框创建	大量候选提前构造	超过阈值后才构造
排序与 NMS	包含大量低分候选	仅处理有效候选
调试输出	可能逐帧打印	默认关闭详细日志
平均后处理时间	14 ms	1.6 ms

最新测试中，StarryOS 后处理平均为 1.632 ms，Linux 基线为 0.449 ms。两端使用相同的 Rust 解析、阈值筛选、NMS 和坐标恢复逻辑，因此差异更可能来自张量映射缓存属性、CPU 调度、编译环境、候选数量及缓存冷热状态。

5.2　技术路线取舍与回退方案

5.2.1　强制 JPU 输出 NV12

否决原因：摄像头 JPEG 是 4:2:2，JPU 原生三平面输出为 YUV422P；NV12 是 4:2:0 半平面。强制宣称 NV12 会造成采样和布局错误。正确方案是让 VPSS 官方支持 YUV422P 输入并完成 CSC。

5.2.2　普通模型输入直接物理地址绑定

回退原因：虽然源缓冲和张量哈希一致，但直接推理为零检测，说明绕过了运行时对内存类型、stride、命令补丁或缓存所有权的处理。正式路径恢复官方 `CVI_NN_SetTensorWithAlignedFrames()`。

5.2.3　P2 aligned 640 模型

回退原因：输入 TDMA 接近 0，但前向增加到约 80 ms，后处理增加到约 5 ms，共享内存显著膨胀，总时延约 134 ms。局部指标优化导致整体退化。

5.2.4　固定跳帧或降低处理帧数

固定跳帧可以降低平均资源占用，却不会降低被选中帧的 JPU、VPSS 和 TPU 单帧时延，还会增大控制反馈间隔。当前只在算法不需要 YOLO 的红车阶段跳过 VPSS/TPU，属于按需求裁剪，而不是盲目丢弃感知机会。

5.2.5　继续提高 compute 线程优先级

compute 的 97% 以上已经处于 `fire_to_irq` 硬件命令流。提高等待线程优先级只能影响 IRQ 后不足 1 ms 的恢复段，无法消除 39 ms 模型执行。继续修改调度可能破坏 USB 与控制线程公平性，收益有限。

视觉链路的难点集中在同步取帧的重构、加速器的输入契约与完成事件调度三处，其根因都在 设备与驱动的交互边界上；系统层的难点则落在另一类位置——固定开销的实现细节、调度器的 不变式，以及硬件语义与软件假设之间的落差。

5.3　内核优化中的定位方法与难点

这一章记录的是“改动之前发生了什么”：每一族改动都是先被数据指出来的，而这些定位过程 本身就是本项目在方法上的主要产出。把它们写下来还有一重作用——它解释了为什么最终方案 长成现在这样，也让后续接手的人不必重复同样的排除工作。

5.3.1　定位方法：分段计账、标度实验与受控 A/B

三种手法贯穿全部内核优化。第一种是**分段计账**：在内核侧把一条路径切成若干段并累计 耗时，用一个特殊文件（`/proc/fault_attrib`）读出，取“工作负载前后的快照之差”得到每次 调用的平均值。第二种是**标度实验**：固定其它条件、只改一个自变量（区域大小、缓冲区大小、窗口页数），看哪一段的成本随之变化——“随规模变化”的项与“固定”的项对应完全不同的原因。 第三种是**受控 A/B**：同一次上电、交替顺序、各测两遍取均值，用来抵消本板上“启动后第一次 大分配明显更快”这类时间漂移。

这三种手法也解释了为什么本项目的收益集中在“固定开销”上：分段数据反复显示，同一块板上 StarryOS 与 Linux 的差距不在带宽，而在每次操作要付的那一笔固定成本。

5.3.2　系统调用返回路径：一笔 2.17 µs 的冗余刷新

`getpid` 优化前要 10 560 ns，同板 Linux 只要 422.5 ns。更有说服力的是另一组对照：故意传入非法调用号的 `invalid_syscall`（内核里只有“立即返回”这一条路）与 `getpid` 一样 慢——处理体只从当前线程结构里读一个字段，不可能值 10 µs。九个系统调用在优化前呈同一条 水平线，说明它们共享同一笔固定成本。

段	ns/次	说明
陷阱循环整体 `iter`	10 687	用户态进入 → 返回值写回
└ 进入/返回用户态 `enter`	5 558	架构层的一整趟往返
└ 分派与函数体 `handle`	2 654	按调用号查表 + `getpid` 本体
└ ptrace 前置检查 `pre`	264	每次进入都判一次“是否被跟踪”
└ 返回后处理 `post`	439	信号与返回前记账
其它（用户线侧循环、统计口径）	1 772	探针自身与两侧差值

表 40: `getpid` 的分段归因（优化前）。处理体只占 2.65 µs，其余都是路径成本

把最大的一段 `enter` 再拆开，答案落在一个指令上：`fence.i` 占 2 172 ns、真正的寄存器 往返占 2 150 ns、入口校验 267 ns。也就是说，**每次返回用户态都执行一次指令缓存失效**，它自己值 2.17 µs，并且把此后整条内核路径的指令缓存打冷。而内核本来就已经在真正需要的 两处刷（页变可执行、ptrace 改写正文），返回路径上的这一次是纯冗余。

改动之后 `getpid` 落到 1 631 ns，但还剩下 3.05 µs 的账（含探针开销口径）：

段	ns/次	说明
返回用户态往返	1 070	架构地板：已无可摘除的冗余
循环内其它	560	采样与记账
分派与函数体	360	查表 + `getpid` 本体
ptrace 前置 + 返回前信号检查	230	每次陷阱都查一遍的开关
用户侧	840	探针循环本身

表 41: 返回路径瘦身之后 `getpid` 的剩余构成。已经没有单项超过 1.1 µs，剩下的是五处“每次陷阱都查一遍”的布尔开关

剩下的是同一类东西：单步标志 91 ns、ptrace 速查 102 ns、返回前工作 149 ns、seccomp 88 ns、trace 101 ns。要让它们再快，得把这些分散的布尔判断合并成一次位图检查，属于返回 路径的结构性重构，收益上限约 1.5 µs，本轮到此为止。

5.3.3　进程与 exec：账目与两条决定性判据

`fork_exit` 优化前是 7 497 µs。给内核侧与任务侧各加一组分段计数器后，得到的账是：`fork()` 系统调用 4 418 µs（其中 `try_clone` 1 528、首次入队激活 1 724、建线程 591、 其余 575）、子进程 `_exit()` 1 013 µs、子地址空间回收 1 459 µs（由回收线程异步做， 不在 `fork_exit` 的墙钟里）、以及 11–14 次上下文切换（约 2.6–3.2 ms）。

归因给出两条决定性判据，它们把后续方向定死了：其一，**首次入队激活的 1.7 ms 全在守卫 析构**——把它拆成三段后，入队 44 µs、收尾 0.6 µs、定时器 10.5 µs，剩下 1 852 µs 全在“从 IRQ 守卫退出时挑下一个任务”。其二，**这 1.7 ms 不是“子进程占用了 CPU”**——让子进程 在 `fork` 返回后先忙等 T ms 再退出，父侧 `fork()` 的耗时完全不随 T 增长。所以它是切换 本身的成本：切到一个全新任务再切回来。

顺带否掉了两个假设：硬件 ASID 是可用的（已经处于标签模式，不存在“每次切换都全量刷 TLB”），以及子地址空间析构并不在 `fork_exit` 的关键路径上。

`exec` 一侧的分段结论是：`exec_load` 31.7 ms 里 `exec_populate` 占 28.3 ms（88%）， 而重定位只占 0.56 ms、ELF 解析 0.05 ms。再往物化里挖又得到两条否证——文件后端的分段 计数全程为 0，说明段映射走的是 COW；一次 51 轮 exec 压力前后的块层读数增量为零，说明 exec 完全不读 SD 卡。

5.3.4　上下文切换：归因清楚了，但没动

一次 `ctx_switch` 优化前是 226.8 µs，Linux 10.97 µs。该指标是“管道写/读系统调用 + 阻塞 + 唤醒 + 一次切换”的整包成本，而非单条切换指令的耗时：

段	µs/次	说明
管道写（含唤醒）	61	其中唤醒 49：选中等待者 0.35、交付 43–47
└ 入队：就绪树插入	13.6	红黑树插入与虚拟时间更新
└ 入队：其余六段	11.7	实体克隆、`settle_current`、`push_class` 等
调度入口	2.1 × 4–6 次	其中约 1/3 是空转：进去发现不需要切换，仍付全额
调度决策	15.6	选取与虚拟时间维护
裸交接	5	地址空间激活 2.0 + CPU 绑定 0.4 + 汇编交接 2.2

表 42: 一次上下文切换的账。**没有单一热点**：十几处 1–15 µs 的固定开销叠加，其中“每次切换要进 4–6 次调度器、1/3 是空转”是可以立刻回收的一块

这一族在本轮止步于归因。原因是它已经不是“修一处 bug”的规模：要逼近 Linux 的 10.97 µs，必须重构调度热路径（空转入口前置判定、入队合并快照、收紧 `settle_current` 与虚拟时间 更新），三条都动调度器不变式。本轮先把归因做完——这是后续任何改动的前提。

顺带记录一条**负结果**：把“首次入队立刻抢占”改成 Linux 的 `WF_FORK` 语义（用运行期旋钮在同一镜像上做 A/B），只值 `fork_exit` 0.4%、`fork_exec_exit` 0.9% 的改善。原因是那 1.7 ms 里子进程该做的工作本来就要付，抑制抢占只是把时间从 `fork` 挪到 `waitpid`。

5.3.5　缺页：从“87.7% 在 prepare”到一个 $O(n^2)$ 的索引

缺页分段的第一层结果把范围收得很窄：`prepare` 占 87.7%，而 TLB 失效只占 0.07%—— 后者直接否掉了“缺页慢是因为刷 TLB”这个常见猜测。再拆 `prepare`，`pending_insert` 占 719 039 ns/页，是整条路径的 58%。

根因是 COW 页索引（一个按物理帧地址排序的 `vec`）在每次插入时都要扫整个索引统计存活 条目，再叠加有序 `vec` 的搬移，整体退化成 $O(n^2)$；成本因此随区域线性膨胀（1 MiB 154 µs → 64 MiB 1 302 µs/页）。修掉之后成本变平，但**搬移**还在：标度实验显示 `pending_insert` 的每页成本仍随区域增长，位置直方图进一步说明插入位置不是两端的 问题（追加只占 21%），而是“一窗 64 页里每一条插入都要把整条尾巴搬一次”，平均每次 搬 1 930 个条目（62 KB），已经贴着这块板的内存带宽。

这一族里最值得记的一处教训是：**把“整段插入”想简单了会把板子挂死**。第一版实现让板子 在启动过程中完全无输出，连软件复位都发不进去，只能人工按复位键。原因是朴素的“整段落位”没有考虑索引里已经存在的过期墓碑条目：如果某个墓碑的物理地址落在这一段区间内部，整段插入之后索引就不再按物理地址有序，而后续的插入与摘除都靠二分查找定位，于是从“插入慢”变成“状态错乱”。重新实现时的安全设计是三条——整段插入全有或全无、遇到空洞或 区间内墓碑就退回逐页登记、容量按整段预留以免在 IRQ 关闭的临界区里分配内存。

同一族里还发现一处**真正的正确性缺陷**：页索引里已释放帧留下的过期墓碑会遮蔽同址重分配 的新主人，使查询返回“无此身份”，让 fork/COW 在帧回收后走错分支。修复方式是自该位置向前扫过所有仍包含该物理地址的条目、返回第一个活主人（无墓碑时仍只查一条，代价不变）。

5.3.6　读路径：前两条线索为什么只值 10%

缓存命中读优化前的画像是“每页取页两把锁 2.2 µs + 页到暂存 4.3 + 暂存到用户 4.5”。 据此做的两条改动——去掉暂存中转（改成 pin 住页缓存页直接拷给调用方）、把用户内存准备 整段化——分别只把大读吞吐从 374 提到 437、再到 474 MB/s，而“每轮换一块新缓冲”那一列 只从 50.2 走到 51.5 MB/s。也就是说，数据路径与缺页事务的“外壳”都不是瓶颈。

决定性的对照来自同一个探针换不同文件：

探针	中位耗时	说明
`read_syscall /proc/uptime 1 KB`	404 µs	procfs，纯内存
`read_syscall /etc/profile 1 KB`	7777 µs	ext4，连跑两遍都是 7.7–7.8 ms
`read_syscall /bin/busybox 1 KB`	10115 µs	ext4，连跑三遍都是 10.10–10.16 ms
`read_syscall /bin/busybox 0 KB`	10038 µs	**读 0 字节也一样慢**

表 43: 决定性对照。耗时与“读多少字节”无关，与“在哪个文件系统”强相关（差 20 倍），且每次重复都一样（不是冷启动开销）

三条推论把范围收得很窄：与读的数据量无关；ext4 比 procfs 慢 20 倍；不是一次性开销。于是转向“`open`+`read`+`close` 里哪一段固定地做了 I/O”。分段计时把 10 ms 钉在文件对象 析构上；随后四轮探针逐一排除 VFS 节点驱逐、目录项与文件节点析构、ext4 inode 析构。最后用三个“零代码”实验直接指认根因：

实验	结果	说明
40 次 shell `exec 3</bin/busybox` 后立即关闭（**不读**）	0.9 ms/次	只 open+close 不慢
`read_syscall /bin/busybox 0 40`（读 0 字节）	10.0 ms/次	触发条件是“`read` 被调用过”
40 轮前后 `/proc/diskstats`	writes +**136**、sectors +1088	约 1 个 inode 块 + 一次日志提交；tmpfs 上增量为 0

表 44: 三条零代码实验。答案是：文件对象析构只要句柄“被读过”（0 字节 `read` 也算）就在 `close` 时同步更新访问时间，而 ext4 的元数据更新末尾直接做一次同步日志提交

这一族还有一处值得记录的实现陷阱：**用墙上时钟判断“访问时间是否过期”在本板上永远成立**。板上没有 RTC、也没有 NTP，`date` 显示 1970 年，比镜像里文件的 2025/2026 年时间戳还小，拿它套 `relatime` 判据会永远判成“已过期”，退化成每次都写。最终改用单调时钟，并把 “上次持久化时间”记在本来就跨 `open` 存活的页缓存身份里。

5.3.7　其余指标的剩余固定成本

上下文切换与读路径之外，还有两处剩余成本已经被量到具体项上，只是本轮没有动：

项	每页	说明与后续想法
页对象建立 `page_object`	7.3 µs	每页一次 `Arc::new` + 页标识堆分配 → 可把页头内嵌进帧，或做对象池
索引插入（按 run 批量化之后）	约 1 µs	已经贴住内存带宽，继续压需要换数据结构
装页表（`pt.map_page`）	3.8 µs	三级走查；可按叶子表批量安装
帧分配 / 清零	3.9 µs	整块分配后主要是清零，受带宽限制
槽 / 反向映射发布	**未精确量到**	余下 **残差** µs/页（由 40 减 16 再减 5 得到）的嫌疑落在这里：每页一个映射槽 + 一次反向映射更新

表 45: 缺页路径的剩余预算。已量到的四项合计约 16 µs/页；另有约 19 µs/页 尚未精确归因，嫌疑是每页一次的槽与反向映射发布

`munmap` 一侧同样已经拆到底：整轮 23.39 µs/页 里，清页表 10.5、 捕获前像 3.61、摘除映射槽 3.28、其余 4.9。清页表 仍在逐页走查两次，但**这一项必须和“空叶子表回收”一起设计**——页表 crate 里已有的批量清 映射原语不剪空目录，单独换上会让“不断换新地址”的程序持续累积页表页，而项目里还有一个 尚未定位的内存增长问题（见下文），不能只为提速而换。

5.4　被数据否掉的假设

优化过程中有相当一部分工作是“证明某条路不值得走”。下表是六个被实测否掉的假设：

假设	实测	结论
缺页慢是因为刷 TLB	TLB 段只占整条路径的 0.07%	不做；直接省掉了一次针对 TLB 的改动
页缓存容量不够，应只回收“够腾位置”的子集	回收掉的本来就是脏页，顺序写下每页只回写一次	不做
缓存注册表裁剪是 $O(n^2)$，改掉就能提速	反复打开同一文件时身份被复用，裁剪本来就不跑；实测无变化	改动保留（消除真实复杂度），但不计入收益
`fork` 慢是因为“首次入队立刻抢占”的调度策略	改成 `WF_FORK` 语义只值 0.4% / 0.9%	不做：那段时间本来就要付，抑制抢占只是把它挪到 `waitpid`
无线链路“传输到 9 MiB 就失效”是驱动缺陷	真因是周期性全局同步写回与并发文件系统读踩踏	按并发约束修复，而不是改数据面
页索引插入可以对整段简单地一并合入	第一版实现让板子在启动时硬挂	回退后重做：全有或全无 + 不确定就退回逐页

表 46: 被实测否掉的六个假设。最后一行是唯一一次把板子挂死的改动，重做后的设计见上一节

5.5　已经归因清楚、但本轮没有改的项

下表每一项都有完整的归因数据，之所以不做，是因为它们都需要改动核心不变式（调度器、页表结构、返回路径的开关结构），单独立项比夹在其它改动里更安全。

项	归因结论	为什么留到下一轮
上下文切换 220.4 µs	一次切换要进调度器 4–6 次、约 1/3 是空转；入队与虚拟时间维护合计十几处 1–15 µs 固定开销	要重构调度热路径，三条候选都动调度器不变式
`fork` 的 1.7 ms 首次入队重调度	全在“从 IRQ 守卫退出时挑下一个任务”的析构里，且与子进程是否占用 CPU 无关	属于调度策略语义，需要独立排期与专项回归
读路径 fresh 仍差 62.8 对 271.1 MB/s	每页固定成本：页对象 7.3 µs、装页表 3.8 µs，另有约 19 µs/页 尚未精确归因	先补完那 19 µs 的归因（要挂不按 fork 门控的槽发布探针），再决定改法
`munmap` 清页表 10.5 µs/页	批量原语现成，但它不剪空叶子表	必须与“空叶子表回收”一起设计，否则会持续累积页表页
`getpid` 剩余 3.05 µs	返回路径上散着五处“每次陷阱都查一遍”的布尔开关（合计约 530 ns）	收益上限约 1.5 µs，需要重构返回路径的开关结构
定时器与睡眠抖动	时基本身没问题（累积漂移 0.17%），尾部抖动来自唤醒路径	需要一次性定时器 + 精确截止时间队列，改动面覆盖整个定时器子系统
`fsync`(2 MiB) 差 2.5 倍	每次后端写之后都要等一次屏障，粒度保守	屏障下推直接关系掉电一致性，要做专门的掉电测试
启动内核段 9.01 s	三块静默时间：USB 探测 2.1 s、无线固件 1.5 s、ext4 挂载 0.6 s	都牵动设备初始化顺序；本板曾因“启动窗口内控制设备”整机卡死
模型加载约 3 s	模型文件读取 + 张量解析	已归入读路径余项；预读与内存映射交给运行时属应用层改造

表 47: 本轮未做的九项及其理由。它们都已经有可复现的归因数据，接手时可以直接从“改法”开始

5.6　一条尚未定位清楚的问题

项目过程中出现过一次“内存只减不增”的故障：遍历整个 rootfs 时会触发内核内存耗尽并崩溃，表现为回收队列被永久挂住。当时按“整片地址空间被挂住、内存只增不减”的特征加了定位探针，并逐一验证了四个候选假设——结果**四个假设全部被实测排除**，根因至今没有确定。

目前的状态是：触发路径已经不再复现（遍历整个 rootfs 不再崩溃，回收队列不增长）；探针留在内核里（`/proc/fault_attrib` 的回收队列字段），一旦再次出现可以直接取到队列长度与 重试标志；与之相关的一处结构性风险已经识别——页表批量清映射的原语不剪空叶子表，单独 换上会让“不断换新地址”的程序持续累积页表页。这正是上文“不能只为提速而换”的原因。

把这件事写进报告，是因为它直接约束了后续两处改动的做法，也因为它是一个**负结果**：四个合理的假设都被数据否掉了，说明真正的机制在别处，不能靠推理补齐。

内核侧的难点与负结果均已在上面逐项给出，它们全部落在通用子系统的实现内部；单核平台 无法承诺实时这一条，则属结构性问题，需要另一种解法。

5.7　实时小核：通道、兜底与边界

5.7.1　跨核通道的三处硬约束

通道的架构（见 第 3 章）不复杂，真正花时间的是三处与硬件相关的硬约束，它们 都是实机踩出来的。

第一，**mailbox 的 MMIO 必须显式建立非缓存映射**：同一段发送代码用默认的 `iomap` 会让写滞留，表现为“消息写进槽位但小核始终不取、本核也收不到中断”，换用非缓存映射后一次 跑通。共享上下文与负载缓冲同理——C906 的 D-cache 不参与跨核一致性。

第二，**中断上下文只能用 IRQ-save 自旋锁**：最初用睡眠锁会让内核直接以 `validate sleeping lock context failed` 终止；持锁期间也不能调用分配器，因此接收队列 必须是固定容量的环形数组，等待路径必须按“硬中断 → 固定服务线程 → 等待队列”三段式 实现。

第三，**共享结构的每条 cache line 只能有一个写入方**。这是一条实测教训：布局 v2 把两条 环的 head/tail 挤在同一条 64 字节缓存行里，小核整行回写时会**顺带用旧副本覆盖掉大核写的 `tail`**，大核按被污染的 tail 记账，于是“忽多忽少”。这条现象当时被误读成“小核 tick 慢 2.19%”——真正的原因与更正过程见 5.7.5 节。把头部按写入方拆成三条独立 cache line、`SHM_VERSION` 从 2 升到 3 之后，问题消失。

5.7.2　失效兜底：小核挂了谁停车

小核接管链路之后，看门狗与堵转保护都跑在它身上，于是出现一个新的失效面：**小核自己挂了，这套保护就全没了**，而这时唯一还能动作的是大核。兜底因此要回答两件事：大核怎么确认小核已经死了，以及怎么在小核仍占着 UART1 的情况下把线抢回来。

判据用两条，缺一不可。其一是“小核 PC 恒为 0”（`c906l_kick status` 的只读取证，退出码 3）；其二是“控制周期数在 2 秒内不再前进”（读 kind=8），后者覆盖“小核还活着但控制任务卡死”以及“读不到上报”这两种情况。演练时，活着的那一轮第一条不成立、第二条计数前进 30，判定为未死（不误触发）；`hold` 掉核心之后两条同时成立，判定为已死。

动作序列是：先让驱动放掉小核侧对 UART1 的占用（即便小核已死、该命令必然超时，回填的 `mode=0` 仍表示大核驱动侧已释放），再把 A18/A19 的引脚复用切回大核 UART1，并补发 `INIT` 与“零速 + 停车 + 复位”序列，把车停住。裸板演练里真的把核心停掉后跑通了整条链，随后 `c906l_kick kick` 让小核重新启动、`cmdqu_test uart 1` 把链路交回。

边界必须一并说明：这是**事后兜底**，动作延迟由大核调度决定，不是小核那种固定节拍；真正“毫秒级且不依赖任何软件”只能靠硬件急停，而本项目“不新增组件”的前提把它排除了。另外，抢回之后小核不能再碰 UART1，必须显式“kick + 交回”才能恢复；抢回瞬间 `uart-stats` 里残留的 `mode=3` 是小核死前的陈旧快照，不能当作“它还活着”的证据。同一轮排查中还定性了 一个独立故障：这块板**第一次上电时小核偶发不被释放**（PC 恒 0），用同一段释放序列在几秒后 重放即可恢复，于是把它固化成了现场急救工具。

5.7.3　小核还能做什么，以及为什么现在没做

厂商把小核定位成“感知前端加慢速外设协处理器”：小核固件里有 gpio、i2c、uart、pinmux、cif、jenc、sensor、rgn 驱动与 isp、vi、vcodec、camera、audio 任务，官方 sensor_quick_startup 文档甚至要求在 boot 阶段由小核完成摄像头 sensor 的 I2C 初始化序列。把小核的能力清单与本车实际硬件逐条对照，就能看清哪些能接、哪些接不了，以及各自卡在哪一步。

最典型、收益也最大的一条是**摄像头**。本车的相机是 USB（UVC，1e45:8022）摄像头，挂在大核的 DWC2 USB 主机上，节点为 /dev/cvi-usb-camera0；而小核固件里**完全没有 USB 驱动**，它拿不到这台相机的控制权。厂商给小核准备的摄像头通路是另一条：MIPI CSI 加 sensor 加 ISP 加 VI。要走这条路有两个前提，本车都不满足：其一，需要换成 CSI 接口的 sensor 模块并重新布线；其二，要把 sensor、ISP、VI 的时钟、pinmux 与缓冲所有权交给小核，还要补上厂商快速 启动通路里那段“大核先分配 ION 缓冲”的握手——StarryOS 侧没有这段握手，小核的摄像头分支 因此拿不到有效缓冲（实测那块 ION 窗口是静态残留）。考虑到感知链路 89% 的时间花在解码与 缩放上，这一条确实是“最值得做”的扩展，但目前卡在**硬件形态不匹配**上，因此先记为不做。

不涉及换硬件的候选有三类，各自卡在不同的地方。第一类是**把大核已经跑通、却仍需重复的 实时工作搬过去**：机械臂 UART2 已经能由小核直发（kind=11 回报），但位置读回（PRAD）目前仍由大核工具做，要把它也做成小核闭环，先得定下“/dev/ttyS2 归谁”的交接约定。第二类是**新增传感器输入**：如果之后加装光电门、限位开关或 IMU，让小核做带时间戳的固定 周期采样是天然合适的，但前提是先确认这些信号接在 SG2002 侧还是底盘板侧，并解决 GPIO 的 pinmux 归属。第三类是**需要新增触点的安全硬件**：小核的 driver/gpio 是现成的，做毫秒级 急停只差一个外部触点（排针上 A15 是唯一纯 GPIO），但“不新增组件”是本项目的既定前提。

还有几类工作被明确排除，排除理由分两类——不划算或不必要：电机 PWM 直驱在小核 侧只有寄存器定义、没有现成驱动，而且会丢掉底盘板给出的 RPM 闭环；音频与 ISP 的其余功能块 与拾球场景无关；WiFi 与 SD 在小核固件里没有驱动，且 SD1 焊盘与 UART3 复用冲突，接管会 直接掉 WiFi；时间同步协议（SYS_CMD_SYNC_TIME）已经在协议里预留，但当前小核 tick 与名义值一致（5000.00 μs、0 ppm），优先级排在上述功能之后。

候选工作	厂商预期 / 现成能力	预期收益	为什么没做	启动条件
MIPI CSI 摄像头 + ISP	小核有 `cif`/`sensor`/`isp`/`vi`/`jenc` 全套驱动	感知链 89% 时间在解码与缩放	本车是 USB UVC 摄像头，小核无 USB 驱动	换 CSI 模块，明确 pinmux 归属
机械臂 UART2 读回闭环	UART2 直发已通（kind=11）	抓取动作全程跑在小核节拍上	需先约定 `/dev/ttyS2` 交接	明确交接协议并复测残差
GPIO 急停 / 碰撞触点	小核 `driver/gpio` 现成，A15 是唯一纯 GPIO	毫秒级、不依赖大核	需要外部触点，与“不新增组件”冲突	允许新增微动开关
IMU / 光电门周期采样	小核可做带时间戳的固定周期采样	更稳的采样节拍与时间基准	车上没有这些传感器，引脚归属未定	加装传感器并确认归属
电机 PWM 直驱	小核只有寄存器定义、无 PWM 驱动	省掉一层转发	会丢掉底盘板的 RPM 闭环，不划算	仅当底盘板被移除时再评估
WiFi / SD	小核固件无驱动	—	SD1 焊盘与 UART3 复用冲突，接管会掉 WiFi	—

表 48: 小核侧候选工作的取舍。把“值得搬”的判断标准归纳成四条——驱动就绪、不与大核现有链路冲突或有明确交接、不新增硬件、收益可量化——四条同时满足才会排期

5.7.4　负结果与取证纪律

报告同样记录被否定的设想与踩过的坑，因为它们决定了方案的边界。

第一，**不要把推理或图像预处理搬到小核**。感知链路的瓶颈在 CPU 侧软件 JPEG 解码，不在 NPU，更不在小核：同一张 1279×1706 的图上，软件解码中位数约 829 ms、resize 约 76 ms、 TPU 前向仅 **41.4 ms**、编码约 710 ms，预处理占总时间的 89%，TPU 只占 7.7%。小核没有 TPU 访问也没有对应驱动，正确的做法是用大核上的 JPU 硬解通路解决解码瓶颈。

第二，**不建议做 noMMU 版 StarryOS**。去掉 MMU 等于放弃 Linux ABI 的价值（fork/COW、按需分页、`mprotect`、PIE/ASLR、用户指针校验），成本是月级而收益为负；小核真正需要的 只是几百 KB 的实时固件。

第三，一条与“系统能力”直接相关的**硬解缺陷**：走内核 JPU 的采帧路径（cmd4）会在连续解码 若干秒后把 JPU 永久锁死，现场是 `[JPU] Timeout! status=0x4`（既非 DONE 也非 ERROR），之后每次返回 EIO 直到重启；四轮里首次超时前成功 39–388 帧，而同一轮的原始 MJPEG 路径毫发无伤。把同一条解码路径换成卡上文件的离线重放（完全不经过 USB）后，仍然在第 3 次 解码时以同一句报错锁死——所以“USB 抢总线”可以划掉，问题在 JPU 的状态机。这条负结果 同时说明了为什么长稳数据要按“先硬解、后软解”的顺序采，以及为什么视觉流水线必须把“JPU 锁死”当成一种可观测的失效模式。

第四，一条**取证纪律**：StarryOS 的写只进页缓存、没有周期性回写，板端写出的文件必须先 `sync` 才算存下来。受控实验里，不 `sync` 就硬复位，卡上留下的文件“长度正确、内容全零”（哈

希与同长度全零数据完全一致），带回宿主侧的文件其实是空壳；而 `sync` 是一张“到此 为止”的快照，它只保住自己之前写入的内容，所以脚本里的 `sync` 必须放在最后一次写入之后。

5.7.5　方法：一个把结论改判的例子

早期测量给出“小核 tick 比标称慢 2.19%”，看起来像时钟配置错误。先引入独立于两个核的 第三方参考——主机墙钟给板端串口逐行打时间戳——测得宿主与主机相差 +55.6 ± 12.3 ppm，排除宿主；再改用直方图而不是平均值观察间隔，发现绝大多数相邻间隔恰好是一个 tick，只有极少数是多个 tick，而“每 tick 偏慢”不可能产生这种形状；最后定位到真正的缺陷： 共享头部的一条 cache line 被小核整行回写覆盖，大核按被污染的 tail 记账。把头部按写入方 拆成三条 cache line、版本号从 2 升到 3 之后，复测得到每 tick 5000.00 µs、24 307/24 307 个间隔都正好一个 tick、两分钟零推迟。**结论从“时钟慢 2%”改判为“度量假象”**，而这一次 更正顺带修掉了跨核共享内存的一条硬规则。

这个例子说明“先归因、再改动”在异构场景里更值钱：如果当时直接去调 PLL 分频，既不会 解决问题，还会引入一处真正的时钟错误。

5.7.6　边界与可复用产物

这条工作线到此封盘，能力与边界都已经有实板证据。已验收的能力包括：跨核驱动（`/dev/cvi-rtos-cmdqu`、指定应答等待、`poll`/`epoll`、共享缓冲 `mmap`）、小核接管底盘 链路（握手、命令自检、周期控制、心跳看门狗、堵转保护）、大小核节拍 A/B 对照、控制周期 可调到 5 ms、小核失控后的判定与接管，以及冷启动“小核没被释放”的现场急救。

仍未完成或尚未验证的有：5 ms 控制周期下 200 帧/秒的链路负载需要带车验证；“抢回后底盘 确实收到 STOP”需要带车验证；小核外设归属与 pinmux 仲裁、C906L 是否带 MMU，受限于 TRM 未给出小核章节，仍属未决问题。

可复用的产物除驱动与固件外，还包括一套板端与宿主侧工具（跨核自检、排线协议探针、采帧 与 JPEG 解码分段计时、冷启动统计、上卡与刷写脚本）以及几条可以约束后续工作的硬规则：跨核共享内存必须显式非缓存映射；共享结构的每条 cache line 只能有一个写入方；任何布局改动必须递增版本并两侧同时重建；板端产出的证据文件必须先 `sync`。这些规则与工具都已用 实板数据检验过，可以直接用于后续的异构协同工作。

小核侧的难点在通道约束与失效兜底；而四条线共用的一块板本身也有它自己的难点：焊盘复用、介质通路的边界，以及无线链路的并发缺陷。

5.8　板级 I/O 的难点：三处硬约束与一处并发缺陷

本层的工作看起来只是“接通外设”，但真正花时间的是四件事：一路串口够不够用、介质通路 的边界在哪里、软复位到底复位了什么、以及无线链路为什么会静默失效。下面按这四件事 分别说明难点与结论。

5.8.1　单路控制台的代价：UART 焊盘是复用的

SG2002 在设备树里把 UART0–UART3 全部置为可用，但这并不意味着可以随手取一路当第二控制台：这些控制器的焊盘是可复用的，而候选焊盘各有代价。实测与反编译共同确认，UART1

落在 JTAG 引脚（0x03001064/68，FSEL＝6）上，占用后调试口不可用；UART2 落在 IIC0 焊盘（0x03001070/74，FSEL＝2）上；UART3 则与 SD1 共用焊盘（0x030010D4/D8， FSEL＝5），一旦切过去，挂在 SD1 上的无线网卡随即失效。三路候选各有一个不可接受的 代价，这就是本项目把控制台策略定为“只保留一路、把工具做厚”的直接原因：增加一路串口 在本板上并非配置问题——三路候选分别占用 JTAG、IIC0 与 SD1 焊盘，任何一路都要牺牲 调试口或无线链路。

这条约束还解释了后面两处设计的动机：因为只有一路串口，所以镜像不能走串口（见下一小节）；也因为只有一路串口，所以“板子自己能回到可刷写状态”必须由软件原语保证， 而不能指望第二条带外通道。

5.8.2　介质通路的两条边界：节点残留与空间统计

UMS 把整张卡交给主机之后，有两条边界必须写进流程，否则会以“偶发失败”的形式出现。

其一，**UMS 退出后块设备节点仍可能残留在主机侧**。因此判定“UMS 已就绪”应当以能否读到 FAT 目录为准，而不能以设备节点是否存在为准——按节点存在就写，会在节点尚未就绪时拿到一次写入失败，或者在退出后写到一个已经失效的设备上。

其二，**mtools 对剩余空间的统计与实际簇占用并不一致，写满后报错具有滞后性**。因此 需要维护“启动分区容量约容纳十六份 FIT”这一硬约束，并在写入前滚动删除最旧的映像；依赖工具返回的剩余空间来判满，会在最后一两份映像上失败。

这两条边界都属于“试出来才知道”的类型：它们不会让流程完全不可用，而是让它在长时间 无人值守运行后突然失败，因此被固化成脚本里的前置检查，而不是留在文档里靠人记。

5.8.3　软复位复位了什么：只复位 SoC，不复位执行机构

小核暖复位原语让“回到 U-Boot”变成一条命令，但它的代价在实测中暴露出来：软复位只 重启 SoC，5 V 供电轨不掉电，因此下游执行机构会保持最后一条指令，亦即**“主控重启”并不等于“车已停下”**。车在运动时如果只做软复位，底盘板仍会按最后一条速度指令继续 驱动轮子。

这条边界随后被写入失效分析，成为对执行器侧兜底要求的依据：任何“从软件侧恢复板子”的 动作，都必须在动手之前先把速度指令清零，或者在设计上让执行机构具备独立的超时停车 能力。后文实时小核那条工作线里的心跳看门狗与堵转保护，正是按这条约束设计的。

5.8.4　PWM 的四处语义落差

若按 Linux 的 /sys/class/pwm 语义把 SG2002 的 PWM 暴露给用户态，落差集中在四处：

属性表中的 duty_cycle 以“有效电平的时长”为语义，而硬件只给低电平拍数，需一次 取反并做极性归一；属性以纳秒为单位，硬件以时基拍计数，任何取值都要经过整除与取整，抖动虽只有一拍 却不可忽略；**周期与占空比必须成组生效，否则中间态会输出一段错误波形；**固定脉冲与四路同步两种模式在四个标准属性里没有对应表达，只能另开门路。

这四点正是不宜直接复用其他平台 PWM 驱动的原因，也是本项目把电机 PWM 留给底盘控制器 （而不是在 SG2002 侧直接产生波形）的工程依据之一。

5.8.5　无线链路的并发缺陷：累计传输约 9 MiB 后静默失效

使能链最隐蔽的一处故障不在协议而在并发。压测显示，无线链路在累计传输约 9 MiB 后会 静默失效：接口仍为 `RUNNING`、地址仍在，但连通性全无，`sshd` 亦无法建立会话。定位后 确认其触发条件为“周期性的全局同步写回与并发文件系统读相互踩踏”，读路径在不允许阻塞 的上下文中发起块 I/O，被块层拒绝后又被压成致命错误；修复后同样的压测在 60 MiB、60 轮下零错误，与厂商 Linux 的表现对齐。

这条缺陷的价值有两重。其一是它解释了“接口看起来完全正常、数据面已死”这种最难排查的 现场表现；其二是它把“读路径不该在不允许阻塞的上下文里发起 I/O”这条与文件系统相关 的约束钉了下来——5.3 节 中“文件对象析构路径不做同步 I/O”的结论，与它是同一 类问题。

第 6 章　总 结

本项目的总结按四条线给出：应用层视觉推理与运动控制链路、系统层内核优化、实时小核、 以及板级 I/O 与上板工作流。四条线共用同一块板、同一份用户态 ELF 与同一套评测口径， 因此它们的结论可以放在一起读：应用层把端到端帧到指令延迟压到与 Linux 相当，内核层把 每次操作的固定成本从落后一到两个数量级收敛到同一量级，实时小核补上了单核平台无法承诺 的实时性与故障兜底，板级 I/O 则让这些结论能在真机上被反复测出来。

6.1　视觉推理链路优化总结

本项目针对 SG2002 单核异构视觉平台，对网球机器人推理链路进行了从采集、解码、图像预处理、共享内存、张量导入、神经网络执行到检测后处理的跨层优化。摄像头侧采用后台异步 UVC 组帧、单调帧序号和一帧在途背压，在不形成历史队列的前提下提供最新完整帧；JPU 侧根据 JPEG 4:2:2 采样格式将 MJPEG 直接解码为 YUV422P，并通过三平面 ION 接口把结果写入设备共享物理内存，同时通过 JPEG 完整性校验、更新帧重试、解码器复位和 1,024 字节位流尾部安全区解决损坏帧与页边界预取问题。

在图像预处理阶段，StarryOS VPSS 驱动由原来的 NV12 输入扩展为符合官方寄存器语义的 YUV422P 三平面输入，并在一次离线任务中完成 BT.601 颜色空间转换、等比例缩放、640×640 letterbox 和 RGB planar 输出。JPU 输出的 source ION 与 VPSS 输出的 destination ION 均采用 DMA32 物理连续分配，完整 YUV 和 RGB 图像不再经过 CPU 大块复制。通过排队、硬件启动和硬件完成时间戳发现，早期约 45 ms 的 VPSS 墙钟中真实硬件仅约 1.5 ms，其余为中断后调度等待；采用 3 ms 有界完成观察与 IRQ 兜底后，VPSS 平均时延降至约 1.6 ms。

TPU 路径首先修正 ION 文件描述符解析、`WAIT_DMABUF` 返回语义、任务完成队列和错误状态传递，使官方 CVITEK 运行时与 StarryOS 内核 ABI 对齐。随后按描述符数量区分 TDMA-only 输入任务与 compute 任务，并将任务生命周期拆分为提交排队、硬件命令流、中断恢复和返回用户态四个区间。针对单模型串行负载，提交线程改为持有可睡眠运行锁直接执行 `prepare/run/wait/finish`，删除提交线程与 TPU worker 之间的两次额外调度交接。对于硬件仅约 0.8 ms 的输入 TDMA，调度器增加 IRQ 队首唤醒；同时将 100 µs 定时轮询改为 2 ms，避免定时器在真实 IRQ 到来前移除等待者。最终 TDMA 内核任务约 1 ms，过去约 44 ms 的 IRQ 后等待被实际消除，而非从统计中隐藏。

驱动还依据 SOPHGO 官方实现补齐 CV181x TPU/FAB 时钟门控、低有效复位顺序、PLL/mux/divider 频率解析及 `prepare/run/finish` 生命周期。实机确认 TPU 工作于 700 MHz、FAB 工作于 900 MHz，且固件已经设置正确分频。将 `fire` 时间点校准到最终启动寄存器后，正式 640×640 模型的硬件命令流约 39.4 ms，占内核 compute 墙钟的 97% 以上，说明软件排队已不再是 TPU 主瓶颈，剩余优化必须面向模型结构、编译和存储访问，而不能继续依靠提高线程优先级。

在正式 640×640 模型下，StarryOS 最新单帧平均结果为：摄像头等待 5.489 ms、JPU 1.645 ms、VPSS 1.643 ms、TPU 输入 TDMA 1.114 ms、TPU 推理 41.309 ms、后处理 1.632 ms、其他视觉开销 0.194 ms，总时延 53.024 ms，平均置信度 93.13%。同硬件 Linux 基线总时延为 54.230 ms。StarryOS 依靠专用摄像头/JPU/VPSS 前端取得约 4.173 ms 优势，抵消了 TPU 与后

处理约 2.968 ms 的劣势，最终低约 1.206 ms，即本轮约快 2.22%。这一结果说明当前 StarryOS 已在成功视觉帧平均时延上追平 Linux，并验证了“固定工作负载专用数据通路、硬件共享缓冲区、精确 IRQ 调度和模型契约校验”这一优化路线的有效性。

应用层的结论止于端到端延迟；系统层三条线的结论各不相同，因此分别小结：内核部分回答 固定成本收窄到何处，小核部分回答实时性与兜底是否可承诺，板级 I/O 部分回答结论本身 能否被反复验证。

6.2　内核优化总结

内核这条线只做一件事：把“每次都要付的固定成本”降下来。做完之后，它与厂商 Linux 的 相对位置可以概括为四条。

第一，**“每次调用都要付”的固定成本已系统性下降，但还没有到齐**。空系统调用从 25.0 倍 收到 3.9 倍，4 KiB 管道往返从 10.4 倍收到 2.8 倍，exec 从 18.1 倍 收到 9.3 倍，ext4 `open+read+close` 从 246 倍收到 10.8 倍。

第二，**缺页这条最深的坑基本填平**。64 MiB 默认路径从 1302123 ns/页 降到 44120 ns/页（29.5×），与 Linux 的差距从 165 倍收到 5.6 倍，而且成本已经 **不再随区域大小增长**——后者比倍数本身更重要，因为它意味着这条路径可以随负载规模扩展。 这一族的四步改进彼此独立，任何一步都可以单独回退。

第三，**带宽型与总量型指标本来就领先或持平，优化后仍然领先**。8 MiB 页缓存顺序写从 2.41 MiB/s 提到 42.43 MiB/s（Linux 34 MiB/s），8 MiB 写 + `fsync` 从 7961 ms 降到 931 ms（Linux 790 ms），冷启动到可交互 shell 为 10.8 s（Linux 约 33.2 s）。这些指标 据此判定，差距来自每次操作的固定成本，而同类带宽指标两侧接近。

第四，**剩下的差距集中在两处，且都已被精确定位但没有动**。一是上下文切换（220.4 µs 对 10.97 µs），根因是调度热路径上十几处 1–15 µs 的固定开销叠加； 二是文件读的每页固定成本（fresh 仍差 62.8 对 271.1 MB/s），根因是每页一次的映射槽与反向映射发布、以及页对象建立。两者都需要改动核心不变式，因此本轮只做完归因、把方案与前置条件写进 5.3 节，而不是在收尾阶段动它们。

方法上的收获与数字同样重要。这一阶段没有一次收益来自“按经验调优”：指令缓存刷新、页索引 的 $O(n^2)$、`close` 里的同步元数据提交，都是先被分段数据指出来的；而被否掉的六个假设（TLB 失效、页缓存回收策略、注册表裁剪、`WF_FORK` 抢占抑制、无线链路的协议层、 整段插入的朴素实现）同样来自数据。这套“先归因、再改动、每步留回退”的做法，是后续接手 这项工作时最应该继承的部分。

6.3　实时小核工作线总结

这条工作线把 SG2002 上那颗“存在但未被使用”的 RISC-V 小核做成了可管理的第二个执行体，并用它回答了一个与算力无关的问题：单核通用平台能不能承诺实时与冗余。结论是能，但代价与收益都需要说清。

收益不在算力，而在**确定性与故障兜底**。底盘控制周期的抖动从“由负载决定”变成“与负载 无关”：大核直控在 3 个纯计算负载下 p99 恶化到 1.54 ms、最坏 1.75 ms，而小核接管在同一 负载下 min = mean = max，样本 100% 落在期望的 tick 刻度；真实感知流水线作为负载时，大核直控的长尾反而比纯计算负载小（0.947 ms），说明长尾取决于“负载是否持续就绪”，而不是“负载像不像视觉”。冗余一侧同样有实板证据：大核停喂心跳约 500 ms 后小核自主停车，小核自己被停掉后大核侧的两条只读判据能在 2 秒内确认并抢回链路补发停车序列。

这条工作线同时给出了小核的边界：它的算力低于大核、没有 TPU 访问、外设面窄到不含 USB、 SD 与以太网，因此正确定位是“只接实时与兜底职责”——底盘链路所有权、固定周期采样、 停止与看门狗、上电快速就绪；推理、图像预处理与电机闭环仍留在大核与底盘板上。按这个 定位评估的候选扩展（MIPI CSI 摄像头、机械臂读回、GPIO 急停、传感器周期采样）与其启动 条件，已在 5.7 节 逐条列出；其中收益最大的摄像头前移卡在“本车是 USB 摄像头、 小核无 USB 驱动”这一硬件形态不匹配上。

对整车而言，这条线补上的是**可承诺性**，而非性能指标：无论大核上跑着什么负载、是否正在崩溃重启，“停车”这个动作始终有一处不依赖大核的执行者。这一点与内核优化那条线互补——后者把每次操作的成本降到接近 Linux，前者把不能靠降成本解决的时序问题交给另一颗核。

6.4　板级 I/O 与上板工作流总结

这一层不主张相对厂商 Linux 的性能优势，它的交付物是**可反复暖重启、可无人值守刷写与被测的板卡与工作流**。做到这一步靠的是三条判断：控制台只保留一路、把工具做厚（因为可用的 备用 UART 分别压在 JTAG 与 SD1 焊盘上）；镜像走 U-Boot 的 UMS 而不是串口（因为控制台 带宽有限且缺乏可靠流控）；系统切换借道小核（因为大核侧没有物理内存直写通道）。三条 判断合起来，使“回到 U-Boot、刷入新镜像、跑一遍探针”变成一条全可由宿主脚本驱动的动作 序列，不需要物理按键、不需要板上网络、也不需要主机提权。

网络与 SSH 在这一层里的定位是加速项而非必要项。现场热点是移动热点，主机随时可能离场， 因此把“串口可独立完成全部操作”作为硬约束；无线链路本身也修掉了两处与本板强相关的缺陷（关联被固件拒绝时的设备状态、以及周期同步写回与并发文件读踩踏导致的静默失效），后者 修复后在同一压测下与厂商 Linux 对齐。

对本项目而言，这一层的价值在于**让其余各章的结论可信**：只有板子能被反复刷写、被反复上电、被脚本稳定地取数，“相对 Linux 的持平与领先”才是可复现的结论，而不是一次性的 现场结果。执行器侧的一条边界也在这里被钉下来——软复位只重启 SoC、不停执行机构，因此 任何从软件侧恢复板子的动作都必须先把速度指令清零；这条约束直接决定了实时小核那条线里 心跳看门狗与堵转保护的设计。

参考